PROJECT-BASED STUDY GUIDE

# The Fundamentals of Machine Learning

Part I of *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*,
learned by building one realistic, end-to-end project:
**HomeLens**

| | |
|---|---|
| **Book chapters covered** | 1 · The ML Landscape 2 · End-to-End Project 3 · Classification 4 · Training Models 5 · Support Vector Machines 6 · Decision Trees 7 · Ensembles & Random Forests 8 · Dimensionality Reduction 9 · Unsupervised Learning |
| **Project** | HomeLens: valuation, screening and market segmentation of 20,640 California census districts |
| **What you get** | A complete, tested Python project (11 runnable steps), every output and figure in this guide produced by that code, plus a challenge bank of 100+ questions |
| **Tool versions used** | Python 3.13 · scikit-learn 1.9 · pandas 3.0 · NumPy 2.5 · Matplotlib 3.11 |

This guide is an independent study companion. It explains the concepts of the book in its own words and uses original code. Read it next to the book: the book gives you the author's full treatment; this guide makes you apply every idea until you can defend it.

# Contents


**How to use this guide** 5

- The three-pass method . . . 5
- What "ready for Part II" means . . . 5

**The HomeLens project** 6

- Success criteria agreed with the stakeholders . . . 6
- The data dictionary . . . 6
- Repository layout . . . 6
- Professional conventions used everywhere . . . 7

**Roadmap: book chapter → project step** 8

**Chapter 1 · The Machine Learning Landscape** 9

- 1.1 Concepts you must own . . . 9
- 1.2 Framing HomeLens: the checklist every project starts with . . . 10
- 1.3 Project step 1 — hands on . . . 11
- 1.4 Hands-on experiments (do these) . . . 13
- 1.5 Challenge questions . . . 13

**Chapter 2 · End-to-End Machine Learning Project** 15

- 2.1 Concepts you must own . . . 15
- 2.2 Project step 2 — look at the data and protect the test set . . . 16
- 2.3 Project step 3 — pipelines, models, fine-tuning, error analysis . . . 21
- 2.4 Hands-on experiments . . . 26
- 2.5 Challenge questions . . . 26

**Chapter 3 · Classification** 28

- 3.1 Concepts you must own . . . 28
- 3.2 Project step 4 — hands on . . . 29
- 3.3 Hands-on experiments . . . 34
- 3.4 Challenge questions . . . 34

**Chapter 4 · Training Models** 36

- 4.1 Concepts you must own . . . 36
- 4.2 Project step 5 — hands on . . . 37
- 4.3 Hands-on experiments . . . 45
- 4.4 Challenge questions . . . 45

**Chapter 5 · Support Vector Machines** 47

- 5.1 Concepts you must own . . . 47
- 5.2 Project step 6 — hands on . . . 48
- 5.3 Hands-on experiments . . . 52
- 5.4 Challenge questions . . . 52

**Chapter 6 · Decision Trees** 54

6.1 Concepts you must own . . . 54
6.2 Project step 7 — hands on . . . 55
6.3 Hands-on experiments . . . 59
6.4 Challenge questions . . . 60

**Chapter 7 · Ensemble Learning and Random Forests** 61

7.1 Concepts you must own . . . 61
7.2 Project step 8 — hands on . . . 62
7.3 Hands-on experiments . . . 66
7.4 Challenge questions . . . 66

**Chapter 8 · Dimensionality Reduction** 68

8.1 Concepts you must own . . . 68
8.2 Project step 9 — hands on . . . 69
8.3 Hands-on experiments . . . 73
8.4 Challenge questions . . . 74

**Chapter 9 · Unsupervised Learning Techniques** 75

9.1 Concepts you must own . . . 75
9.2 Project step 10 — hands on . . . 76
9.3 Hands-on experiments . . . 81
9.4 Challenge questions . . . 82

**Chapter 10 · Integration: the final model, the one-shot test, and shipping** 83

10.1 The decision trail . . . 83
10.2 Tuning the final regressor . . . 83
10.3 The one-shot test evaluation . . . 84
10.4 Where is the model weak? Segment analysis . . . 85
10.5 The premium screener . . . 86
10.6 Shipping: artefacts, model card, inference, tests . . . 86
10.7 Defend your model: the sceptic's interview . . . 88

**Appendix A · Master cheat sheets** 89

A.1 Algorithm comparison (Part I) . . . 89
A.2 Metrics cheat sheet . . . 89
A.3 Symptom → diagnosis → remedy . . . 90
A.4 Formula sheet (know these cold) . . . 90

**Appendix B · The mixed challenge round** 91

**Appendix C · Bridge to Part II (Neural Networks and Deep Learning)** 93

Suggested next step: a bridge experiment . . . 93
Mastery checklist . . . 93
A two-week plan . . . 94

Capstone extensions . . . . . 94

**Appendix D · Glossary** **95**

Closing note . . . . . 95

# How to use this guide

Part I of the book is the foundation for everything that follows. Neural networks (Part II) are *one more family of models* trained with the same ideas you meet here: a loss function, gradient descent, regularisation, a validation set, a pipeline that prevents leakage. If those ideas are solid, deep learning is an extension; if they are shaky, deep learning feels like magic and fails for reasons you cannot diagnose.

This guide therefore does not walk through the chapters one by one in isolation. It makes you **build one project** in which every chapter solves a real problem that appears naturally, in the same order a professional would meet it. You will see each idea *fail or succeed on real data*, which is how understanding becomes durable.

## The three-pass method

| Pass | What you do | Goal |
|---|---|---|
| 1 · Read and run | Read each chapter's concept section, then read the code blocks and outputs. Run `./run_all.sh` or run each step with `python tools/run_step.py steps/stepXX_*.py`. | Recognise every idea and see it work. |
| 2 · Predict and break | Before reading each OUTPUT block, cover it and predict the result. Then change one thing (a hyper-parameter, a feature, the split) and predict again. Use the *Hands-on experiments* at the end of each chapter. | Build intuition for cause and effect. |
| 3 · Defend | Answer the *Challenge questions* aloud without notes. Then do the final *Defend your model* review in Chapter 10 as if a sceptical colleague were interrogating you. | Be able to explain, derive and justify. |

> **The five abilities you are training**
> **Explain** — say what an algorithm does and why, in plain words. **Derive** — write the key equation and say what each symbol means. **Implement** — code the core of it from scratch at least once (Chapter 4 does this). **Debug** — recognise under/overfitting, leakage and bad metrics from symptoms. **Decide** — choose an algorithm, metric and validation scheme for a new problem and justify it.

## What "ready for Part II" means

- You can say, without hesitation, what the **training**, **validation** and **test** sets are for, and why touching the test set early invalidates it.
- You can write the gradient-descent update rule, explain the effect of the learning rate, and explain why features must be scaled.
- You can explain the **bias/variance trade-off** and name at least four levers that move a model along it.
- You can choose between precision, recall, F1, ROC-AUC and PR-AUC for a given business cost, and set a decision threshold.
- You can explain why **ensembles** (bagging, boosting, stacking) work, and why a single decision tree is unstable.
- You can describe **PCA** geometrically and algebraically (SVD), and say when dimensionality reduction helps and when it hurts.
- You can wrap everything in a **Pipeline** so cross-validation is honest. In Part II this becomes `tf.data` + Keras models, but the discipline is identical.

# The HomeLens project

You have just joined **HomeLens**, a (fictional) real-estate analytics start-up. The sales team currently estimates neighbourhood values by hand from spreadsheets. They want a data product that can, for any census district in California:

1. **Estimate the typical house value** (a regression model) so analysts can rank and compare districts.
2. **Flag premium districts** (a classifier) so brokers can quickly screen where high-value listings are likely, tuned for a precision the business can trust.
3. **Discover market segments and unusual districts** (unsupervised learning) so that marketing can target regions and data engineers can find suspicious records.

This is a deliberately realistic brief. It is **not** a toy: the data has missing values, hard caps, heavy tails, a categorical field, geographic structure and noisy labels. It is the California housing dataset used throughout Part I of the book (1990 census block groups, 20,640 rows).

## Success criteria agreed with the stakeholders

| Deliverable | Business question | Metric | Acceptance bar |
|---|---|---|---|
| Valuation model | How much is a typical house worth here? | RMSE and MAE in dollars; share within ±20% | Must clearly beat the naive baseline (predict the mean); error estimate must be trustworthy |
| Premium screener | Is this district in the top price quartile? | Precision (cost of a false lead) and recall; ROC-AUC | Precision ≥ 0.85 with the best possible recall |
| Segmentation / anomaly detector | Which markets exist? Which rows look wrong? | Silhouette, BIC, business sense | Interpretable regions; flagged rows reviewable by a human |

## The data dictionary

| Attribute | Meaning | Type / quirk |
|---|---|---|
| longitude, latitude | Location of the district | Numeric; strong geographic structure |
| housing_median_age | Median age of houses (years) | Capped at 52 |
| total_rooms, total_bedrooms | Totals over the whole district | Heavy right tail; 207 missing bedroom values |
| population, households | People and households in the district | Heavy right tail |
| median_income | Median household income (units of $10,000) | Strongest predictor; capped near 15 |
| ocean_proximity | Category: <1H OCEAN, INLAND, NEAR OCEAN, NEAR BAY, ISLAND | Categorical; ISLAND has only 5 rows |
| median_house_value | **Target**: median house value ($) | Capped at $500,001: about 4.7% of rows |

## Repository layout

**PROJECT TREE**

```
homelens/
├── data/housing.csv                  # raw data, never edited
├── src/homelens/                     # the reusable library (imported by every step)
│   ├── config.py                     # paths, RANDOM_STATE, constants
│   ├── data.py                       # load, stratified split, label construction
│   ├── features.py                   # custom transformer + preprocessing pipelines
│   ├── tasks.py                      # ready-made arrays for chapters 3-9
│   ├── utils.py                      # plotting style, RMSE, results registry
│   └── predict.py                    # inference on new districts
├── steps/                            # ONE script per learning step (runnable)
│   ├── step01_framing.py             #  Ch1  landscape, baselines, under/overfitting
│   ├── step02_eda.py                 #  Ch2  look at the data, split, explore
│   ├── step03_pipeline_models.py     #  Ch2  pipelines, CV, fine-tuning, error analysis
│   ├── step04_classification.py      #  Ch3  metrics, thresholds, multiclass
│   ├── step05_training_models.py     #  Ch4  GD from scratch, regularisation, softmax
│   ├── step06_svm.py                 #  Ch5  kernels, C and gamma, SVR
│   ├── step07_trees.py               #  Ch6  CART, Gini, instability
│   ├── step08_ensembles.py           #  Ch7  voting, bagging, forests, boosting, stacking
│   ├── step09_dimensionality.py      #  Ch8  PCA, kernel PCA, t-SNE
│   ├── step10_unsupervised.py        #  Ch9  k-means, DBSCAN, GMM, semi-supervised
│   └── step11_final_model.py         #  all  final model, test evaluation, shipping
├── tests/test_homelens.py            # automated checks of the data/feature code
├── artifacts/                        # saved models + model_card.json
├── reports/figures, reports/outputs  # every figure / printed output in this guide
├── tools/                            # step runner and PDF builder
├── requirements.txt   run_all.sh
```

## Professional conventions used everywhere

- **The test set is locked.** It is split off once, stratified, with a fixed seed, and is first used in Step 11. Everything before that uses cross-validation or an internal hold-out taken from the *training* set.
- **Everything that learns lives inside a Pipeline.** Imputers, scalers, encoders, even k-means feature generators are fitted on training folds only. This is how leakage is avoided.
- **Reproducibility.** One `RANDOM_STATE`, one `config.py`, a `requirements.txt`, and a runner that regenerates every number in this guide.
- **Baselines before models.** A model is only good relative to a dumb alternative.
- **Tests and a model card.** Data code is unit-tested; the shipped model has a documented purpose, metrics and limitations.

**How to set up and run**

`pip install -r requirements.txt`, then from the project folder run `./run_all.sh` (about 10 minutes on two CPU cores) or run a single step: `python tools/run_step.py steps/step05_training_models.py`. The runner executes the script block by block (blocks are separated by `# %% name` comments) and stores each block's printed output; steps are also ordinary Python scripts. After Step 11, `PYTHONPATH=src python -m homelens.predict` scores three example districts, and `pytest -q` runs the tests.

# Roadmap: book chapter → project step

Use this table as your map. The *proof* column is what you should be able to demonstrate before moving on.

| Book chapter | Project step | What the step teaches | Proof you have learned it |
|---|---|---|---|
| 1 · ML landscape | Step 1 | Problem framing, metric choice, baselines, under/overfitting, why data quantity matters | You can predict the shape of a train-vs-validation curve and say which lever to pull |
| 2 · End-to-end project | Steps 2 and 3 (+ 11) | EDA, stratified split, imputation, scaling, encoding, custom transformers, pipelines, CV, randomised search, error analysis, one-shot test evaluation | You can build a leak-free pipeline from memory |
| 3 · Classification | Step 4 | Accuracy trap, confusion matrix, precision/recall, thresholds, ROC vs PR, multiclass, multilabel | You can pick a metric and a threshold from a business cost |
| 4 · Training models | Step 5 | Normal equation, batch/stochastic/mini-batch GD, learning curves, Ridge/Lasso/ElasticNet, early stopping, logistic and softmax regression | You can implement GD and logistic regression with NumPy |
| 5 · SVMs | Step 6 | Large margin, soft margin (C), kernel trick, gamma, SVR, scaling and complexity | You can predict what C and gamma do to a boundary |
| 6 · Decision trees | Step 7 | CART, Gini, regularisation, instability, regression trees | You can compute a Gini split by hand |
| 7 · Ensembles | Step 8 | Voting, bagging/OOB, random forests, boosting, stacking; model bake-off | You can explain why each ensemble works and its failure mode |
| 8 · Dimensionality reduction | Step 9 | Curse of dimensionality, PCA (SVD), kernel PCA, t-SNE | You can compute PCA with `svd` and interpret loadings |
| 9 · Unsupervised learning | Step 10 | k-means, choosing k, DBSCAN, GMM, BIC, anomaly detection, semi-supervised | You can choose and validate a clustering |
| Integration | Step 11 | Final tuned model, honest test evaluation, segment analysis, shipping artefacts | You can defend the model to a sceptic |

# Chapter 1 · The Machine Learning Landscape

**Project step 1** — frame the HomeLens problem, choose a metric, build baselines, and *watch* underfitting and overfitting happen. Code: `steps/step01_framing.py`.

## 1.1 Concepts you must own

### What machine learning is

Machine learning is the practice of getting a computer to improve at a task *from data* instead of from hand-written rules. Tom Mitchell's definition is the one to memorise: **a program learns from experience E with respect to task T and performance measure P if its performance on T, as measured by P, improves with experience E.** For HomeLens: T = estimate a district's typical house value; E = the 16,512 training districts with known values; P = RMSE in dollars. If you cannot name T, E and P for a project, you do not yet have a machine-learning problem.

ML shines where hand-coded rules are too long, too brittle or unknown (spam filtering, image recognition), where the environment changes (retrain instead of rewrite), and where it can reveal structure in data humans have not noticed (*data mining*). It is the wrong tool when a few exact rules solve the problem, or when you have no relevant data.

### A taxonomy of ML systems

Systems are classified along three independent axes. Being able to place any project on all three is the first step of framing it.

| Axis | Options | HomeLens example |
|---|---|---|
| **Supervision** (what the training data contains) | **Supervised** (labels given: regression, classification) · **Unsupervised** (no labels: clustering, anomaly detection, dimensionality reduction, association rules) · **Semi-supervised** (few labels, many unlabelled) · **Self-supervised** (labels generated from the data itself) · **Reinforcement** (an agent acts, receives rewards) | Valuation and screening = supervised; segmentation = unsupervised; Chapter 9 shows a semi-supervised experiment |
| **Learning mode** (does it learn incrementally?) | **Batch / offline** (train on all data, then deploy; retrain periodically) · **Online** (learn from a stream in small steps; tune the *learning rate*; also used for data too big for memory = out-of-core) | Batch: the market changes slowly, so retrain on a schedule |
| **Generalisation strategy** | **Instance-based** (memorise examples, compare new ones with a similarity measure, e.g. k-NN) · **Model-based** (fit a parametric model and predict with it) | Model-based (linear models, trees, boosting) |

**Regression** predicts a number; **classification** predicts a class. Many algorithms have both versions (trees, SVMs, forests), and some have a misleading name (*logistic regression* is a classifier).

### The main challenges: "bad data" and "bad algorithm"

| Challenge | Symptom | Remedy |
|---|---|---|
| Insufficient training data | Learning curves still falling; high variance | Collect more; use simpler/regularised models; data augmentation. Banko & Brill showed that with enough data very different algorithms converge, but data is not always cheap |
| Non-representative data / **sampling bias** | Good validation score, bad production score | Make the sample resemble the population; use stratified sampling; check who is missing (the classic example is a poll that only reaches people with landlines) |
| Poor-quality data | Outliers, noise, missing values, wrong units | Clean, impute, clip or discard; know *why* values are missing (HomeLens: 207 missing bedroom counts) |

| Challenge | Symptom | Remedy |
|---|---|---|
| Irrelevant features | Noise columns reduce accuracy | **Feature engineering**: selection, extraction (e.g. rooms per household), creating new features |
| **Overfitting** | Tiny training error, large validation error | Simplify the model, constrain it (**regularisation**), get more data, reduce noise |
| **Underfitting** | Large error on both sets | Use a more powerful model, better features, less regularisation |

**Overfitting in one sentence**
A model overfits when it learns the *noise and quirks of the training sample* instead of the signal. Complex models (high degree polynomials, deep unrestricted trees, huge networks) have enough freedom to memorise. **Regularisation** is any constraint that reduces that freedom; **hyper-parameters** (like regularisation strength) are settings of the *learning algorithm*, chosen before training and never learned by it.

### Testing and validating

The only way to know how a model will behave on new cases is to try it on new cases. Split the data: the **training set** fits the parameters; the **test set** estimates the *generalisation error* once, at the end. If you pick hyper-parameters by repeatedly measuring on the test set, you will have tuned to it, and your estimate will be optimistic. So introduce a third set:

| Set | Used to | How often |
|---|---|---|
| Training | Fit the model's parameters | Constantly |
| Validation (dev) set, or cross-validation folds | Compare models, choose hyper-parameters | Many times |
| Test | Report the final generalisation error | **Once**, at the very end |

With a small dataset a single validation set is noisy, so use **k-fold cross-validation**: split the training data in $k$ folds, train on $k-1$, validate on the remaining fold, rotate, and average. It costs $k$ times more compute but uses the data efficiently. After choosing the best settings, retrain on the full training set.

**Data mismatch.** If your training data is plentiful but unlike production data (e.g. web photos vs phone-app photos), a validation set drawn from the training source misleads you. Make the validation/test sets from *production-like* data, and carve out a **train-dev set** from the training source: if the model does well on train-dev but badly on validation, the problem is *data mismatch*, not overfitting.

**No Free Lunch theorem.** No model is universally best. Wolpert showed that averaged over all possible problems every algorithm performs the same. In practice you make *reasonable assumptions* about your data (linear? smooth? local?) and test a few suitable models. This is why Part I teaches many algorithms and a rigorous way to compare them.

## 1.2 Framing HomeLens: the checklist every project starts with

| Question | HomeLens answer |
|---|---|
| What is the business objective? How will the model be used? | Help analysts rank/compare districts; help brokers screen for premium areas. It is a decision-support tool, **not** an appraisal of individual houses. |
| What does the current solution look like? | Manual spreadsheet estimates (our human baseline; typically off by large margins in unfamiliar areas). |
| Supervised / unsupervised / RL? | Supervised (labels: median_house_value). Plus unsupervised extras. |
| Regression or classification? | Regression (value); classification (premium or not); clustering (segments). |
| Batch or online? | Batch; plain retraining suffices. Data is small enough for memory. |

| Question | HomeLens answer |
|---|---|
| Performance measure? | RMSE (penalises big errors, matches the typical regression objective) and MAE (robust, easier to explain). Both in dollars. |
| Assumptions to verify? | Is the downstream user expecting *prices* or *categories*? If categories only (cheap/mid/expensive), classification would have been enough. We checked: both are needed. |

**Choosing RMSE vs MAE.** RMSE is the Euclidean ($\ell_2$) norm of the error vector: RMSE = $\sqrt{(1/m)\ \Sigma\ (\hat{y}^{(i)} - y^{(i)})^2}$. Squaring makes it sensitive to large errors, which is good when big misses are costly and the error distribution is roughly bell-shaped. MAE is the $\ell_1$ norm: $(1/m)\ \Sigma\ |\hat{y}^{(i)} - y^{(i)}|$. It is more robust to outliers. A larger norm index focuses on large values: RMSE ≥ MAE always, and they are close only if errors are similar in size.

## 1.3 Project step 1 — hands on

We load the data, split it, and make two baselines. A **baseline** is the simplest defensible model; it sets the bar. If a sophisticated model does not clearly beat 'predict the average', something is wrong.

**CODE · steps/step01_framing.py · block “setup”**

```
import sys; sys.path.insert(0, "src")
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import MinMaxScaler, PolynomialFeatures
from homelens.data import split, xy
from homelens.utils import set_style, savefig, record, rmse, PALETTE
set_style()
train, test = split()
X_train, y_train = xy(train)
print("train rows:", len(train), "| test rows (locked away):", len(test))
print("target mean = $%.0f, std = $%.0f" % (y_train.mean(), y_train.std()))
```

**OUTPUT · produced by running the block above**

```
train rows: 16512 | test rows (locked away): 4128
target mean = $207005, std = $115701
```

**CODE · steps/step01_framing.py · block “baseline”**

```
# A baseline is the dumbest model that could possibly work. Every real model must beat it.
dummy = DummyRegressor(strategy="mean").fit(X_train, y_train)
base_rmse = rmse(y_train, dummy.predict(X_train))
print("Baseline 1 - predict the average price for everyone: RMSE = $%.0f" % base_rmse)

lin = LinearRegression().fit(X_train[["median_income"]], y_train)
lin_rmse = rmse(y_train, lin.predict(X_train[["median_income"]]))
print("Baseline 2 - one-feature linear model on median_income: RMSE = $%.0f" % lin_rmse)
print("   learned: price = %.0f + %.0f * income" % (lin.intercept_, lin.coef_[0]))
record("baseline_mean_rmse", base_rmse); record("baseline_income_rmse", lin_rmse)
```

**OUTPUT · produced by running the block above**

```
Baseline 1 - predict the average price for everyone: RMSE = $115698
Baseline 2 - one-feature linear model on median_income: RMSE = $84056
   learned: price = 45241 + 41736 * income
```

Predicting the training mean everywhere yields an RMSE of about **$115,698**. This number is simply the standard deviation of the target: a constant predictor's error is the spread of the data. One feature, `median_income`, in a straight line already cuts it to **$84,056**. Every model from here on must be compared with these two numbers.

### Seeing under- and overfitting

To see the bias/variance trade-off clearly we deliberately make learning hard: only **40** training districts, one feature (income), and polynomials of increasing degree. A degree-*d* polynomial has *d* + 1 parameters, so higher degree means more flexibility. We measure error on the 40 points it saw (training) and on 2,000 districts it did not see (unseen).

**CODE · steps/step01_framing.py · block “bias_variance”**

```
# Take only 40 training districts (small data = easy to overfit) and fit polynomials of
# increasing degree. Compare error on the data the model saw vs. on unseen districts.
rng = np.random.RandomState(7)
small = train.sample(40, random_state=7)
# unseen districts, restricted to the income range the model was trained on (no extrapolation)
lo, hi = small["median_income"].min(), small["median_income"].max()
val = train.drop(small.index).query("@lo <= median_income <= @hi").sample(2000, random_state=7)
xs, ys = small[["median_income"]].to_numpy(), small["median_house_value"].to_numpy()
xv, yv = val[["median_income"]].to_numpy(), val["median_house_value"].to_numpy()

degrees = [1, 2, 3, 5, 8, 12, 16, 20]
rows = []
for d in degrees:
    m = make_pipeline(MinMaxScaler((-1, 1)), PolynomialFeatures(d, include_bias=False), LinearRegression())
    m.fit(xs, ys)
    rows.append((d, rmse(ys, m.predict(xs)), rmse(yv, m.predict(xv))))
tab = pd.DataFrame(rows, columns=["degree", "train_RMSE", "unseen_RMSE"]).round(0).astype({"degree": int})
print(tab.to_string(index=False))
best = tab.loc[tab.unseen_RMSE.idxmin(), "degree"]
print("\nLowest error on unseen data at degree", best)
record("bv_table", tab.to_dict("list"))
```

**OUTPUT · produced by running the block above**

```
 degree  train_RMSE  unseen_RMSE
      1    102254.0      85868.0
      2     99903.0      88443.0
      3     82151.0     102172.0
      5     72202.0     104342.0
      8     69066.0     135033.0
     12     63767.0    3554026.0
     16     61731.0   38753389.0
     20     63074.0   19443183.0

Lowest error on unseen data at degree 1
```

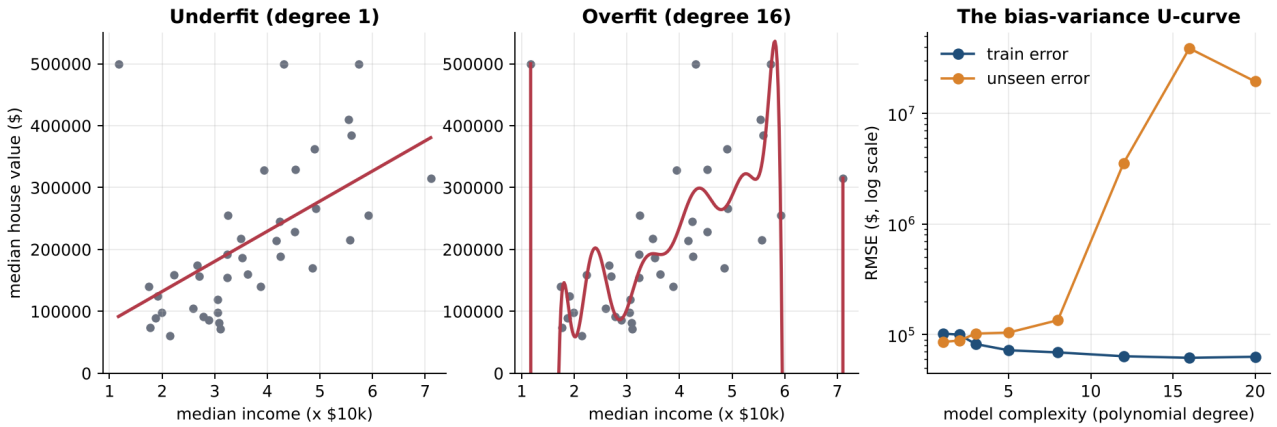



***Figure 1.*** *Left: a straight line underfits (high bias). Middle: a degree-16 polynomial threads the 40 training points but swings wildly in between (high variance). Right: training error keeps falling with complexity while error on unseen data explodes: the signature of overfitting.*

Read the table in three steps. (1) Training RMSE falls monotonically with degree: more flexibility always fits the training points better. (2) Unseen RMSE is best at the simplest degrees and then *explodes* by orders of magnitude. (3) The gap between the two columns *is* the overfitting. Notice that the best model here is the simple one: with noisy data and few points, the extra flexibility only buys noise.

## More data is the best regulariser

**CODE · steps/step01_framing.py · block “data_size”**

```
# More data is often the cheapest cure for overfitting: re-fit the degree-12 model with growing n.
rows = []
pool = train.drop(small.index)
for n in [20, 40, 80, 300, 1000, 5000]:
    s = pool.sample(n, random_state=1)
    m = make_pipeline(MinMaxScaler((-1, 1)), PolynomialFeatures(12, include_bias=False), LinearRegression())
    m.fit(s[["median_income"]].to_numpy(), s["median_house_value"])
    rows.append((n, rmse(s["median_house_value"], m.predict(s[["median_income"]].to_numpy())),
                 rmse(yv, m.predict(xv))))
t2 = pd.DataFrame(rows, columns=["n_train", "train_RMSE", "unseen_RMSE"]).round(0).astype(int)
print(t2.to_string(index=False))
record("datasize_table", t2.to_dict("list"))
```

**OUTPUT · produced by running the block above**

```
 n_train  train_RMSE  unseen_RMSE
      20       60410     13170051
      40       72363        95526
      80       89707        87422
     300       85440        85286
    1000       84622        84986
    5000       83774        84689
```

Same degree-12 model, growing training sets. With 20 points the model is useless; with 300 the training and unseen errors are almost the same; with 5,000 they have essentially converged. A model that overfits on a small sample can be perfectly fine on a large one. This is why the first question when you see overfitting is *can I get more data?*, and why complexity should always be judged relative to the amount of data.

**Common mistakes this step guards against**
• **Judging a model by its training error.** It can always be made small. Only error on unseen data counts. • **No baseline.** An RMSE of $60,000 means nothing until you know the mean-baseline is $115,000. • **Letting validation data leak into training.** Here the validation districts are removed from the training sample before fitting.

## 1.4 Hands-on experiments (do these)

1. Change `PolynomialFeatures` degree values and the training sample size (40 → 400). At what sample size does the degree-8 model stop overfitting? What is the rule of thumb you infer?
2. Replace `MinMaxScaler` with nothing. What goes wrong numerically and why? (Hint: $x^{16}$.)
3. Add `Ridge(alpha=1)` instead of `LinearRegression` to the degree-16 model. Does the unseen error recover? (You will study why in Chapter 4.)
4. Compute the MAE of both baselines. Which is larger, RMSE or MAE, and what does the ratio tell you about the error distribution?

## 1.5 Challenge questions

**Q1. What is the difference between a parameter and a hyper-parameter?**

A **parameter** is learned from data by the training algorithm (a regression coefficient, a tree's split thresholds). A **hyper-parameter** is a setting of the algorithm chosen *before* training (learning rate, tree depth, regularisation strength, k in k-means) and tuned on validation data.

**Q2. Your model has 99% training accuracy and 70% validation accuracy. Diagnose it and list four remedies.**

**Overfitting** (high variance). Remedies: get more training data; simplify the model (fewer features/parameters, lower tree depth); add or increase regularisation; use early stopping, dropout or ensembling; reduce noise (remove outliers, fix labels).

**Q3. Both training and validation errors are high and similar. What now?**

**Underfitting** (high bias). Use a more expressive model, engineer better features, reduce regularisation, train longer. Collecting more data will *not* help much if the model cannot represent the pattern.

**Q4. Why a separate validation set and a test set?**

Choosing hyper-parameters on the test set makes it part of the training process, so its error is an optimistic estimate. The validation set absorbs the repeated comparisons; the test set stays pristine for one final, unbiased estimate.

**Q5. What is sampling bias, and give an example.**

The sample is systematically unlike the population. Example: pooling customer reviews only from users who bothered to review gives a sentiment model that fails on silent users. Fix by representative or stratified sampling and by checking who is missing.

**Q6. Why can a dumb baseline matter more than a clever model?**

It gives the model a meaning. A 91% accuracy on data where 90% of cases belong to one class is a failure. Baselines (mean predictor, majority class, last-period value, a simple linear model) expose such illusions and quantify the value the complex model adds.

**Q7. Name the three kinds of 'learning mode/generalisation' axes and place k-NN on them.**

Supervision × batch/online × instance/model-based. k-NN is supervised, usually batch, and **instance-based** (it stores training points and compares new points to them).

**Q8. When would you prefer MAE over RMSE?**

When the data has many outliers or when errors should be penalised linearly (e.g. delivery-time errors where an hour late is exactly twice as bad as thirty minutes). RMSE is preferred when large errors are disproportionately costly and outliers are rare.

**Q9. What is a train-dev set and what problem does it diagnose?**

A held-out slice of the *training distribution*. If the model does well on train-dev but badly on the (production-like) validation set, the culprit is **data mismatch**, not overfitting.

**Q10. State the No Free Lunch theorem and its practical consequence.**

Averaged over all possible data-generating processes, no algorithm beats any other. Practically, there is no universally best model: make assumptions about the data and compare a few suitable candidates with proper validation.

# Chapter 2 · End-to-End Machine Learning Project

**Project steps 2 and 3** — everything a real project does between 'we have a CSV' and 'we have a validated model': look at the data, build an untouchable test set, explore, preprocess with pipelines, train and cross-validate several models, fine-tune, and analyse errors. Code: `steps/step02_eda.py`, `steps/step03_pipeline_models.py` (and the final evaluation in Step 11, Chapter 10).

## 2.1 Concepts you must own

### The workflow

1. Look at the big picture and frame the problem (Chapter 1).
2. Get the data.
3. Explore and visualise it to gain insights (on the **training set only**).
4. Prepare the data for the algorithms (cleaning, encoding, scaling, feature engineering) **in a pipeline**.
5. Select and train models; compare with cross-validation.
6. Fine-tune hyper-parameters; analyse the errors; evaluate once on the test set.
7. Present, launch, monitor and maintain the system.

### Creating a test set the right way

Set aside the test set **before** you explore. Your brain is a pattern detector: if you study the test data you will subconsciously pick models that suit it (*data snooping bias*). A split must be (a) **random**, (b) **stable**, i.e. the same rows land in the test set every run, which you get from a fixed seed or from hashing a row identifier, and (c) **representative**.

A purely random split of 20,640 rows is usually fine, but when a *single attribute dominates* the target (here median income, correlation ≈ 0.69), a random split can accidentally under- or over-represent some income ranges, biasing your estimate. **Stratified sampling** divides the population into homogeneous subgroups (*strata*) and samples the right proportion from each. We bin income into five categories solely to stratify; the category is never used as a feature.

### Exploring

- **Look** at `head()`, `info()` (types, non-null counts), `describe()` (scale, range, percentiles), and histograms of every attribute. Look for caps, heavy tails, skew, different scales and suspicious values.
- **Visualise geography** with a scatter plot (alpha for density, size = population, colour = target).
- **Correlations**: the standard (Pearson) correlation coefficient measures only *linear* relationships, in $[-1, 1]$. A correlation near 0 does not mean 'no relationship' (an inverted U gives 0). Plot the scatter matrix too.
- **Experiment with attribute combinations**: totals per district are less useful than ratios, such as rooms per household or the share of rooms that are bedrooms.

### Preparing the data: the toolbox

| Problem | Tool | Notes |
|---|---|---|
| Missing values | `SimpleImputer(strategy='median')` (also mean, most_frequent, constant); `KNNImputer`, `IterativeImputer` | Fit on train only; stores the medians so the same values fill the test set. Median is robust to outliers |
| Categorical text | `OneHotEncoder` (nominal), `OrdinalEncoder` (ordered) | One-hot yields a sparse matrix; use `handle_unknown='ignore'` for production safety. Avoid ordinal codes for unordered categories: the model will assume 'close' |

| Problem | Tool | Notes |
|---|---|---|
| Different scales | `MinMaxScaler` (→ [0,1]), `StandardScaler` (mean 0, std 1) | Standardisation is less affected by outliers. Needed by gradient descent, SVMs, k-NN, PCA, regularised models. Trees do not need it |
| Heavy right tail | `np.log`, square root, **bucketising**, RBF similarity | Gets features closer to a bell shape; otherwise most models see a few extreme points dominate |
| Multimodal distributions | Bucketise, or add RBF similarity to the modes | Lets a linear model see 'near the peak' as a feature |
| Transformed targets | `TransformedTargetRegressor` | If you log the label, you must invert predictions; this wrapper does it automatically |
| Geographic structure | `ClusterSimilarity` (k-means + RBF) | A custom transformer in `features.py`: location becomes 'how close to each market centre' |

**Transformer API.** Every transformer has `fit()` (learn from data), `transform()` (apply), and `fit_transform()`. A custom transformer needs only `fit` and `transform`; inheriting `BaseEstimator` gives `get_params/set_params`, and `TransformerMixin` gives `fit_transform`. Hyper-parameters belong in `__init__`, learned values end with an underscore (`kmeans_`), the classic scikit-learn convention.

**Pipelines.** A `Pipeline` chains steps so that `fit` runs each transformer's `fit_transform` in order and then the final estimator's `fit`. `ColumnTransformer` applies *different* pipelines to *different columns* and concatenates the results. The decisive benefit is **leak-free validation**: when you cross-validate a pipeline, every fold re-fits imputers, scalers and encoders on that fold's training part only. If you scale the whole dataset first and cross-validate afterwards, information from the validation fold has already leaked into the scaler.

> **Data leakage: the silent killer**
> Leakage means information from outside the training set reaches the model during training. Typical sources: scaling/imputing before splitting, selecting features using all labels, building time-based features that look into the future, duplicates present in both train and test, and **target-derived features**. Symptom: validation scores that look too good and then collapse in production. **Defence: split first, then do everything inside a Pipeline.**

### Selecting, validating and tuning

- **Cross-validation** gives a mean *and* a standard deviation of the score: a measure of how stable the estimate is. Scikit-learn expects 'higher is better' so RMSE appears as `neg_root_mean_squared_error`.
- A model with **training error ≈ 0** is almost certainly overfitting (a decision tree with no limits memorises the data). Only cross-validation reveals it.
- **Grid search** tries every combination of a grid; **randomised search** samples combinations from distributions: better for many hyper-parameters or continuous ones. Include *preprocessing* choices (number of clusters, imputation strategy) in the search; they are hyper-parameters too.
- **Analyse errors**: feature importances, error by subgroup, and a look at the worst districts. Ask 'what is the model systematically missing?'
- **Evaluate on the test set once.** Report a **confidence interval** for the generalisation error, e.g. a 95% t-interval for the RMSE derived from the squared errors.
- **Launch**: persist the full pipeline with `joblib` (so preprocessing travels with the model), expose it behind an API or batch job, write monitoring for input drift and performance, retrain on fresh data, and keep dated backups of data and models.

## 2.2 Project step 2 — look at the data and protect the test set

**CODE · steps/step02_eda.py · block “setup”**

```
import sys; sys.path.insert(0, "src")
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from homelens.data import load_raw, add_income_cat, split
from homelens.utils import set_style, savefig, record, PALETTE
set_style()
df = load_raw()
```

**CODE · steps/step02_eda.py · block “quick_look”**

```
print(df.head(3).T.to_string())
print()
df.info()
```

**OUTPUT · produced by running the block above**

```
                           0         1         2
longitude            -122.23   -122.22   -122.24
latitude               37.88     37.86     37.85
housing_median_age      41.0      21.0      52.0
total_rooms            880.0    7099.0    1467.0
total_bedrooms         129.0    1106.0     190.0
population             322.0    2401.0     496.0
households             126.0    1138.0     177.0
median_income         8.3252    8.3014    7.2574
median_house_value  452600.0  358500.0  352100.0
ocean_proximity     NEAR BAY  NEAR BAY  NEAR BAY

<class 'pandas.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  str
dtypes: float64(9), str(1)
memory usage: 1.6 MB
```

`info()` tells us the dataset has 20,640 rows and 10 attributes, and that `total_bedrooms` has fewer non-null entries (20,433): **207 missing values**. The only non-numeric column is `ocean_proximity`. (In pandas 3 text columns display as `str`; scikit-learn treats them as categorical text for one-hot encoding.)

**CODE · steps/step02_eda.py · block “describe”**

```
pd.set_option("display.width", 130)
print(df.describe().round(1).T.to_string())
print("\nocean_proximity counts:")
print(df["ocean_proximity"].value_counts().to_string())
```

**OUTPUT · produced by running the block above**

```
                     count      mean       std      min       25%       50%       75%       max
longitude          20640.0    -119.6       2.0   -124.4    -121.8    -118.5    -118.0    -114.3
latitude           20640.0      35.6       2.1     32.5      33.9      34.3      37.7      42.0
housing_median_age 20640.0      28.6      12.6      1.0      18.0      29.0      37.0      52.0
total_rooms        20640.0    2635.8    2181.6      2.0    1447.8    2127.0    3148.0   39320.0
total_bedrooms     20433.0     537.9     421.4      1.0     296.0     435.0     647.0    6445.0
population         20640.0    1425.5    1132.5      3.0     787.0    1166.0    1725.0   35682.0
households         20640.0     499.5     382.3      1.0     280.0     409.0     605.0    6082.0
median_income      20640.0       3.9       1.9      0.5       2.6       3.5       4.7      15.0
median_house_value 20640.0  206855.8  115395.6  14999.0  119600.0  179700.0  264725.0  500001.0

ocean_proximity counts:
ocean_proximity
<1H OCEAN     9136
INLAND        6551
NEAR OCEAN    2658
NEAR BAY      2290
ISLAND           5
```

**CODE · steps/step02_eda.py · block “missing_and_caps”**

```
miss = df.isna().sum()
print("Missing values per column:\n", miss[miss > 0].to_string())
print("\nShare of districts whose price sits exactly at the 500,001 cap: %.2f%%"
      % (100 * (df.median_house_value >= 500001).mean()))
print("Share of districts whose age sits at the 52-year cap:        %.2f%%"
      % (100 * (df.housing_median_age >= 52).mean()))
record("n_missing_bedrooms", int(miss["total_bedrooms"]))
record("pct_price_cap", float(100 * (df.median_house_value >= 500001).mean()))
```

**OUTPUT · produced by running the block above**

```
Missing values per column:
 total_bedrooms    207

Share of districts whose price sits exactly at the 500,001 cap: 4.68%
Share of districts whose age sits at the 52-year cap:        6.17%
```

Two important quirks are visible. **First**, the maximum `median_house_value` is exactly 500,001 and about **4.7%** of districts sit at that value: the data is *censored* (the true value is higher but was recorded as the cap). A model can never learn to predict above the cap; we will quantify this later. **Second**, `housing_median_age` is capped at 52. The business must be told: if the client needs predictions above $500k, we need correct labels (collect them) or must drop capped districts and limit our claims.

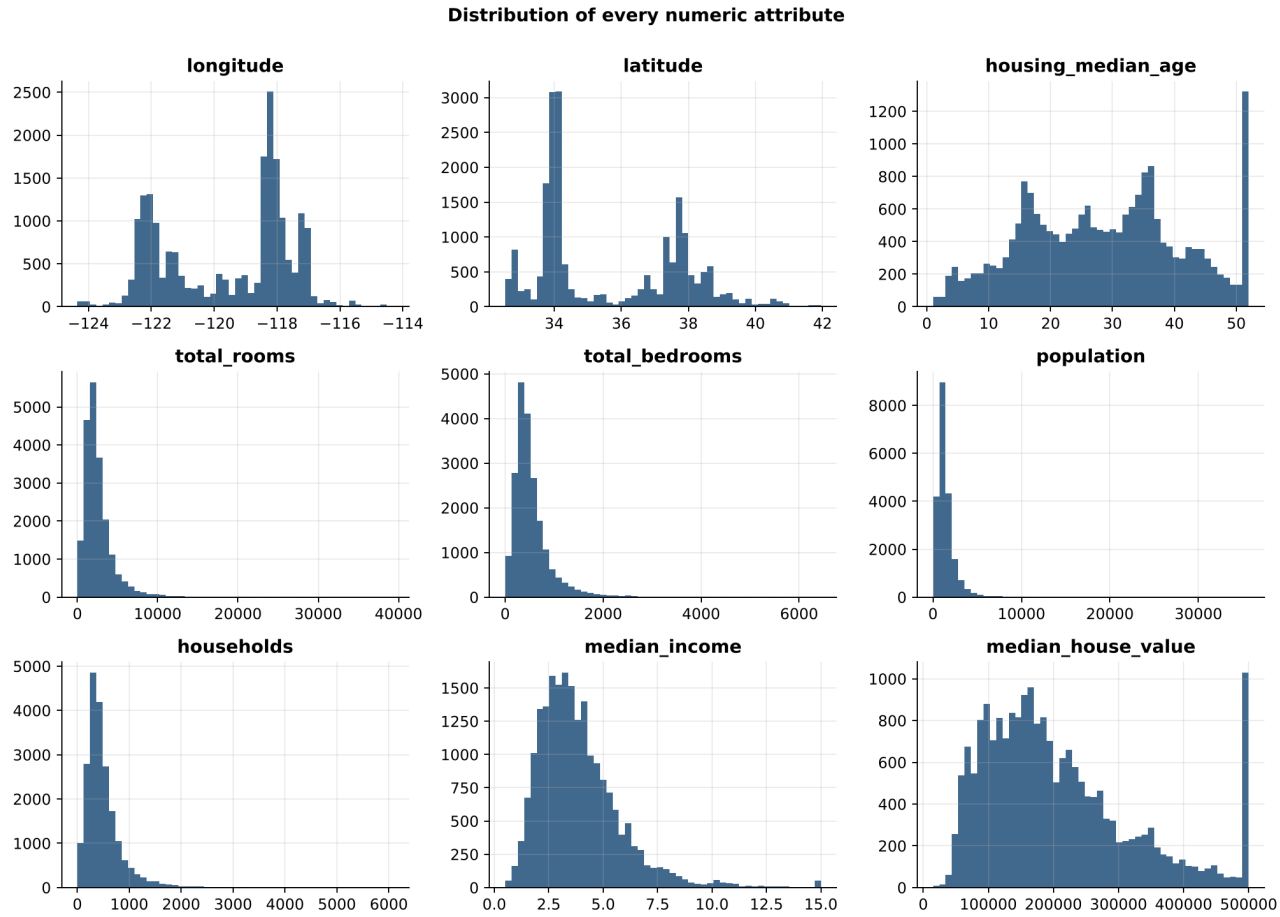



***Figure 2.*** *Histograms of all numeric attributes. Note the spike at the price cap, the age cap at 52, the strong right-skew of rooms/bedrooms/population/households, and very different scales.*

The skew means a few huge districts will dominate un-transformed features: we will log-transform them. The different scales (income 0–15, rooms in the tens of thousands) mean scaling is mandatory for gradient-based and distance-based models.

## A stratified split

**CODE · steps/step02_eda.py · block “split_bias”**

```
# Compare a purely random split with a stratified split on income category.
df["income_cat"] = add_income_cat(df)
rand_train, rand_test = train_test_split(df, test_size=0.2, random_state=42)
strat_train, strat_test = train_test_split(df, test_size=0.2, random_state=42, stratify=df["income_cat"])
cmp_ = pd.DataFrame({
    "Overall %": df["income_cat"].value_counts(normalize=True).sort_index() * 100,
    "Stratified %": strat_test["income_cat"].value_counts(normalize=True).sort_index() * 100,
    "Random %": rand_test["income_cat"].value_counts(normalize=True).sort_index() * 100,
}).round(2)
cmp_["Random err %"] = ((cmp_["Random %"] / cmp_["Overall %"] - 1) * 100).round(2)
cmp_["Strat err %"] = ((cmp_["Stratified %"] / cmp_["Overall %"] - 1) * 100).round(2)
print(cmp_.to_string())
record("split_random_maxerr", float(cmp_["Random err %"].abs().max()))
record("split_strat_maxerr", float(cmp_["Strat err %"].abs().max()))
```

**OUTPUT · produced by running the block above**

```
            Overall %  Stratified %  Random %  Random err %  Strat err %
income_cat
1                3.98          4.00      4.02          1.01         0.50
2               31.88         31.88     32.44          1.76         0.00
3               35.06         35.05     35.85          2.25        -0.03
4               17.63         17.64     16.74         -5.05         0.06
5               11.44         11.43     10.95         -4.28        -0.09
```

The 'Random err %' column shows what happens to the test-set income mix under a purely random split: some income categories are over- or under-represented by up to **5.0%**. With stratification the worst deviation is only **0.50%**. Because income is the strongest predictor, a biased income mix would bias the performance estimate, so this matters. From now on `split()` in `data.py` returns the stratified 80/20 split, and the test part is not looked at again until Step 11.

**CODE · steps/step02_eda.py · block "lock_test"**

```
train, test = split()          # from here on: EXPLORE THE TRAINING SET ONLY
print("train:", train.shape, " test:", test.shape)
explore = train.copy()
```

**OUTPUT · produced by running the block above**

```
train: (16512, 10)  test: (4128, 10)
```

## Visualising geography

**CODE · steps/step02_eda.py · block "geo"**

```
fig, ax = plt.subplots(1, 2, figsize=(13, 5))
ax[0].scatter(explore.longitude, explore.latitude, s=4, alpha=.25, color=PALETTE["blue"])
ax[0].set_title("Where the districts are (density shows population centres)")
sc = ax[1].scatter(explore.longitude, explore.latitude, s=explore.population / 100, alpha=.45,
                   c=explore.median_house_value, cmap="viridis")
plt.colorbar(sc, ax=ax[1], label="median house value ($)")
ax[1].set_title("Bubble size = population, colour = price")
```

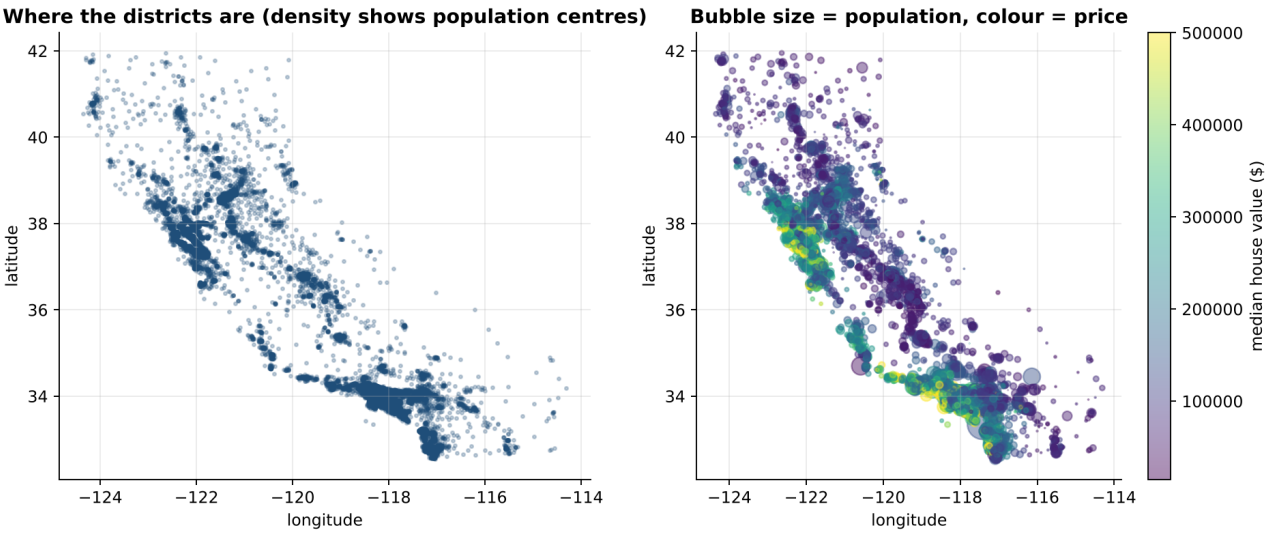



*Figure 3. Left: the density of districts reveals the Bay Area and Los Angeles/San Diego. Right: price is highest near the coast and in the big metros, but 'near the ocean' is not a perfect rule (northern coastal prices are lower), which is why we model location with clusters instead of a single rule.*

## Correlations and attribute combinations

**CODE · steps/step02_eda.py · block “correlations”**

```
num = explore.select_dtypes("number")
corr = num.corr()["median_house_value"].sort_values(ascending=False)
print("Linear correlation with median_house_value (raw attributes):")
print(corr.round(3).to_string())
```

**OUTPUT · produced by running the block above**

```
Linear correlation with median_house_value (raw attributes):
median_house_value    1.000
median_income         0.687
total_rooms           0.135
housing_median_age    0.114
households            0.065
total_bedrooms        0.048
population           -0.027
longitude            -0.047
latitude             -0.143
```

**CODE · steps/step02_eda.py · block “engineered”**

```
explore["rooms_per_house"] = explore.total_rooms / explore.households
explore["bedrooms_ratio"] = explore.total_bedrooms / explore.total_rooms
explore["people_per_house"] = explore.population / explore.households
corr2 = explore.select_dtypes("number").corr()["median_house_value"].sort_values(ascending=False)
print("After adding ratio features:")
print(corr2.round(3).to_string())
record("corr_income", float(corr["median_income"]))
record("corr_bedrooms_ratio", float(corr2["bedrooms_ratio"]))
record("corr_total_rooms", float(corr["total_rooms"]))
```

**OUTPUT · produced by running the block above**

```
After adding ratio features:
median_house_value    1.000
median_income         0.687
rooms_per_house       0.146
total_rooms           0.135
housing_median_age    0.114
households            0.065
total_bedrooms        0.048
people_per_house     -0.022
population           -0.027
longitude            -0.047
latitude             -0.143
bedrooms_ratio       -0.260
```

Income dominates (r = 0.69). Raw `total_rooms` is nearly useless (r = 0.14) because a big district has many rooms whatever its price, but the engineered **bedrooms_ratio** reaches r = -0.26: districts where bedrooms are a small share of the rooms tend to be wealthier. *Ratios express the real phenomenon better than totals.* That insight goes into the pipeline.

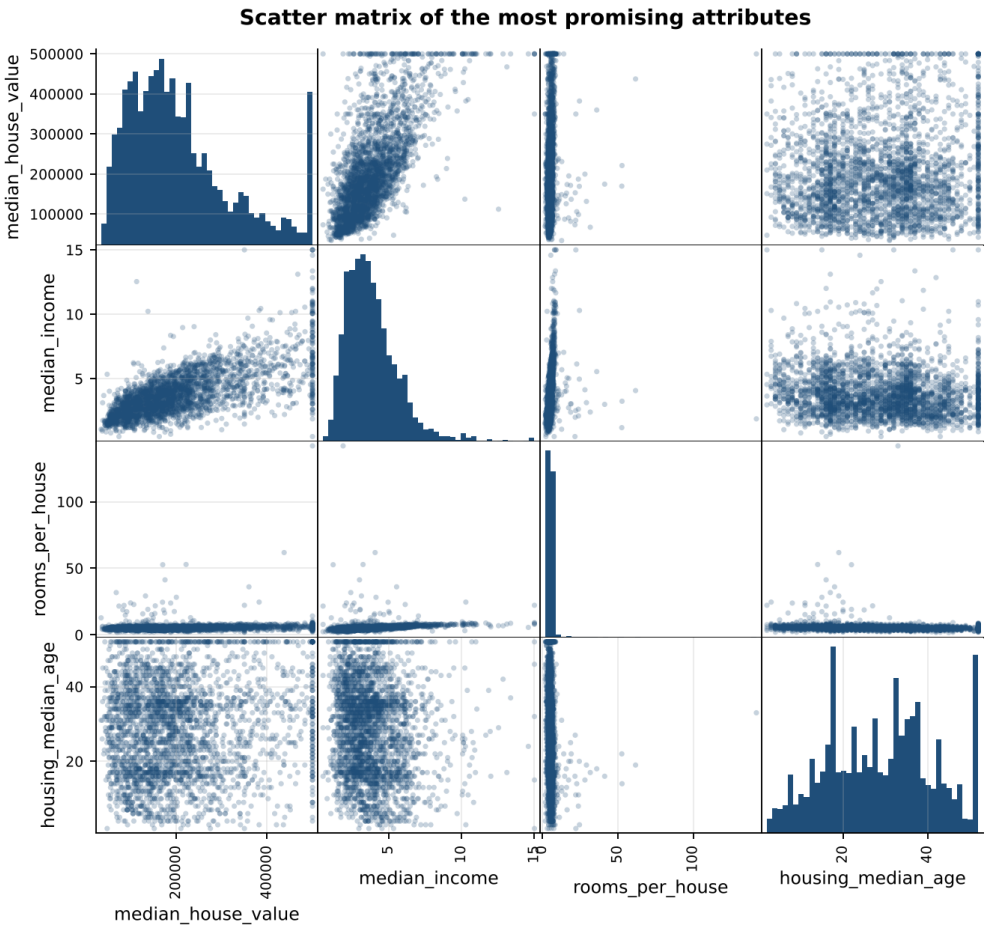



***Figure 4.*** *Scatter matrix of the most promising attributes (sampled for clarity).*

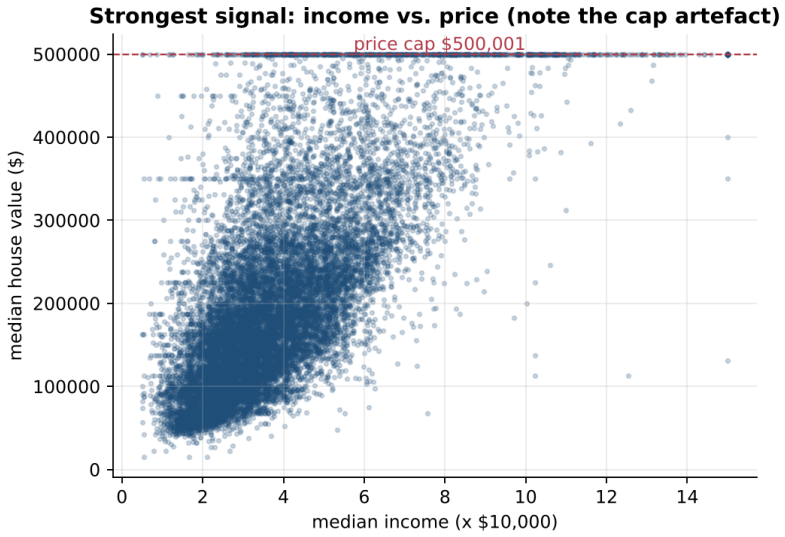



***Figure 5.*** *The income–price relationship is strong and roughly linear, with horizontal lines of points at the price cap (and weaker ones at round values like $350k, $450k): quirks that could mislead a model if not handled.*

# 2.3 Project step 3 — pipelines, models, fine-tuning, error analysis

## The preprocessing pipeline

Here is the pipeline from `features.py`. Study its structure: it is the pattern you will reuse in every project.

**CODE · src/homelens/features.py (abridged)**

```
def build_preprocessing(n_clusters=10, gamma=1.0, use_clusters=True) -> ColumnTransformer:
    transformers = [
        ("bedrooms",         ratio_pipeline(), ["total_bedrooms", "total_rooms"]),
        ("rooms_per_house",  ratio_pipeline(), ["total_rooms", "households"]),
        ("people_per_house", ratio_pipeline(), ["population", "households"]),
        ("log", log_pipeline(), ["total_bedrooms", "total_rooms", "population", "households", "median_income"]),
    ]
    if use_clusters:
        transformers.append(("geo", ClusterSimilarity(n_clusters, gamma), ["latitude", "longitude"]))
    transformers.append(("cat", OneHotEncoder(handle_unknown="ignore"),
        make_column_selector(dtype_exclude="number")))
    default = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())   # for housing_median_age
    return ColumnTransformer(transformers, remainder=default)

def ratio_pipeline():   # impute median -> divide column 0 by column 1 -> standardise
    return make_pipeline(SimpleImputer(strategy="median"), FunctionTransformer(_column_ratio), StandardScaler())
def log_pipeline():     # impute median -> natural log -> standardise
    return make_pipeline(SimpleImputer(strategy="median"), FunctionTransformer(np.log), StandardScaler())
```

Read it as a data-flow diagram. Three ratio features are computed; five heavy-tailed attributes are log-transformed; latitude/longitude become *similarity to each of n k-means market centres* by `ClusterSimilarity`; the category is one-hot encoded; and the one remaining numeric column goes through the default imputer + scaler. Every numeric path ends in standardisation, and every imputer uses the median.

**CODE · the custom transformer (abridged)**

```
class ClusterSimilarity(BaseEstimator, TransformerMixin):
    def __init__(self, n_clusters=10, gamma=1.0, random_state=42): ...
    def fit(self, X, y=None, sample_weight=None):
        self.kmeans_ = KMeans(self.n_clusters, n_init=10, random_state=self.random_state)
        self.kmeans_.fit(X, sample_weight=sample_weight)      # learn the market centres
        return self
    def transform(self, X):
        return rbf_kernel(X, self.kmeans_.cluster_centers_, gamma=self.gamma)   # exp(-gamma * distance^2)
```

The RBF kernel turns distance $d$ to a centre into a similarity $\exp(-\gamma d^2)$ between 0 (far) and 1 (at the centre). A linear model or a tree then sees 'how Bay-Area-like is this district?' as a smooth feature. Because it is *inside* the pipeline, the k-means is refit on each training fold, so there is no leakage.

**CODE · steps/step03_pipeline_models.py · block “setup”**

```
import sys; sys.path.insert(0, "src")
import numpy as np, pandas as pd, matplotlib.pyplot as plt, time
from scipy.stats import randint, uniform
from sklearn.base import clone
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score, RandomizedSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.tree import DecisionTreeRegressor
import joblib
from homelens.data import split, xy
from homelens.features import build_preprocessing
from homelens.utils import set_style, savefig, record, rmse, PALETTE
from homelens.config import ART_DIR, RANDOM_STATE
set_style()
train, _ = split()
X, y = xy(train)
```

**CODE · steps/step03_pipeline_models.py · block “preprocess_demo”**

```
prep = build_preprocessing()
Xt = prep.fit_transform(X)
print("raw shape:", X.shape, "-> transformed shape:", Xt.shape)
names = prep.get_feature_names_out()
print("\nFirst 8 of the", len(names), "output features:")
print(*names[:8], sep="\n")
print("\nAny NaN left after the pipeline?", bool(np.isnan(Xt).any()))
print("Mean/std of first 3 ratio columns (should be ~0/~1):",
      Xt[:, :3].mean(0).round(3), Xt[:, :3].std(0).round(3))
```

**OUTPUT · produced by running the block above**

```
raw shape: (16512, 9) -> transformed shape: (16512, 24)

First 8 of the 24 output features:
bedrooms__ratio
rooms_per_house__ratio
people_per_house__ratio
log__total_bedrooms
log__total_rooms
log__population
log__households
log__median_income

Any NaN left after the pipeline? False
Mean/std of first 3 ratio columns (should be ~0/~1): [-0. -0. -0.] [1. 1. 1.]
```

Nine raw columns became **24** model-ready features, with no NaNs left, and the ratio columns have mean 0 and standard deviation 1, which is exactly what standardisation promises. `get_feature_names_out()` keeps the traceability: `bedrooms__ratio`, `log__median_income`, `geo__geo_cluster_16_sim`, `cat__ocean_proximity_INLAND`...

## Training and cross-validating four models

**CODE · steps/step03_pipeline_models.py · block “cv_models”**

```
# 10-fold cross-validation RMSE for several candidate models, each wrapped with preprocessing.
def cv_rmse(model, folds=10):
    s = -cross_val_score(model, X, y, scoring="neg_root_mean_squared_error", cv=folds, n_jobs=2)
    return s

models = {
    "Mean baseline": DummyRegressor(),
    "Linear regression": LinearRegression(),
    "Decision tree": DecisionTreeRegressor(random_state=RANDOM_STATE),
    "Random forest (100 trees)": RandomForestRegressor(n_estimators=100, random_state=RANDOM_STATE, n_jobs=2),
}
rows = []
for name, est in models.items():
    t0 = time.time()
    pipe = make_pipeline(build_preprocessing(), est)
    folds = 5 if "forest" in name else 10
    s = cv_rmse(pipe, folds)
    train_fit = pipe.fit(X, y)
    tr = rmse(y, train_fit.predict(X))
    rows.append((name, tr, s.mean(), s.std(), time.time() - t0))
    print(f"{name:28s} train RMSE={tr:9.0f} | CV RMSE={s.mean():9.0f} (+/- {s.std():6.0f})
        [{time.time()-t0:4.0f}s]")
cv_table = pd.DataFrame(rows, columns=["model", "train_rmse", "cv_rmse", "cv_std", "secs"])
record("ch2_cv_table", cv_table.round(0).to_dict("list"))
```

**OUTPUT · produced by running the block above**

```
Mean baseline                train RMSE=   115698 | CV RMSE=   115700 (+/-   1744)  [   2s]
Linear regression            train RMSE=    69240 | CV RMSE=    71698 (+/-   7502)  [   1s]
Decision tree                train RMSE=        0 | CV RMSE=    67431 (+/-   3437)  [   2s]
Random forest (100 trees)    train RMSE=    17612 | CV RMSE=    47630 (+/-    899)  [  45s]
```

This table is one of the most important in the whole project; read it slowly.

- **Mean baseline** — CV RMSE $115,700: training and validation agree, and it is just the target's standard deviation.
- **Linear regression** — train $69,240, CV $71,698: errors are similar (no overfitting) but *high*: the model is **underfitting**. A line cannot capture the geography and the non-linear income effect.
- **Decision tree** — train RMSE is **0** (it memorised the 16,512 rows!) but CV RMSE is $67,431: a perfect example of overfitting. Without cross-validation you would have believed it was perfect.
- **Random forest** — train $17,612 versus CV $47,630: it still overfits (gap), but its CV error is by far the best because averaging many decorrelated trees removes variance (Chapter 7).

## Fine-tuning with randomised search

**CODE · steps/step03_pipeline_models.py · block “finetune”**

```
# Randomised search over preprocessing AND model hyper-parameters at the same time.
full = make_pipeline(build_preprocessing(), RandomForestRegressor(n_estimators=60, random_state=RANDOM_STATE,
    n_jobs=2))
param_dist = {
    "columntransformer__geo__n_clusters": randint(5, 40),
    "randomforestregressor__max_features": randint(3, 16),
    "randomforestregressor__min_samples_leaf": randint(1, 6),
}
t0 = time.time()
search = RandomizedSearchCV(full, param_dist, n_iter=10, cv=3, scoring="neg_root_mean_squared_error",
                            random_state=RANDOM_STATE, n_jobs=1)
search.fit(X, y)
print(f"search took {time.time()-t0:.0f}s")
print("Best params:", search.best_params_)
print("Best CV RMSE: %.0f" % -search.best_score_)
res = pd.DataFrame(search.cv_results_).sort_values("rank_test_score")
cols = ["param_columntransformer__geo__n_clusters", "param_randomforestregressor__max_features",
        "param_randomforestregressor__min_samples_leaf", "mean_test_score"]
tab = res[cols].head(6).copy()
tab.columns = ["n_clusters", "max_features", "min_samples_leaf", "CV_RMSE"]
tab["CV_RMSE"] = (-tab["CV_RMSE"]).round(0)
print(tab.to_string(index=False))
record("ch2_best_params", {k.split("__")[-1]: int(v) for k, v in search.best_params_.items()})
record("ch2_best_cv", -search.best_score_)
```

**OUTPUT · produced by running the block above**

```
search took 42s
Best params: {'columntransformer__geo__n_clusters': 34, 'randomforestregressor__max_features': 8,
    'randomforestregressor__min_samples_leaf': 2}
Best CV RMSE: 43227
 n_clusters  max_features  min_samples_leaf  CV_RMSE
         34             8                 2  43227.0
         25             9                 2  43930.0
         23             9                 3  44460.0
         33            13                 5  44740.0
         28             5                 5  44955.0
         29             3                 3  45265.0
```

The search explored the number of geographic clusters, the number of features each split may consider (`max_features`), and `min_samples_leaf`, in **one** pass, *because the preprocessing is part of the pipeline*. The best configuration reached a CV RMSE of **$43,227** (from $47,630), with 34 clusters: more geographic resolution helped. Note the tuning used 3-fold CV and 60 trees for speed; the search is a tool to find a *good region*, not to prove the final number.

> **Practical tuning advice**
> Start with wide ranges and a coarse search; narrow around the best region; use randomised search when you have more than 2–3 hyper-parameters. When the best value sits at the edge of your range, extend the range. Remember that tuned hyper-parameters are tuned to the CV folds: always keep a final untouched test set.

## What did the forest learn? Feature importance

**CODE · steps/step03_pipeline_models.py · block “importance”**

```
best = search.best_estimator_
rf = best.named_steps["randomforestregressor"]
feat = best.named_steps["columntransformer"].get_feature_names_out()
imp = pd.Series(rf.feature_importances_, index=feat).sort_values(ascending=False)
print("Top 12 feature importances (sum of all = %.2f):" % imp.sum())
print(imp.head(12).round(3).to_string())
```

**OUTPUT · produced by running the block above**

```
Top 12 feature importances (sum of all = 1.00):
log__median_income              0.207
cat__ocean_proximity_INLAND     0.085
bedrooms__ratio                 0.067
rooms_per_house__ratio          0.065
people_per_house__ratio         0.062
geo__geo_cluster_16_sim         0.032
geo__geo_cluster_3_sim          0.031
geo__geo_cluster_9_sim          0.030
geo__geo_cluster_18_sim         0.027
geo__geo_cluster_26_sim         0.021
geo__geo_cluster_6_sim          0.018
geo__geo_cluster_24_sim         0.017
```

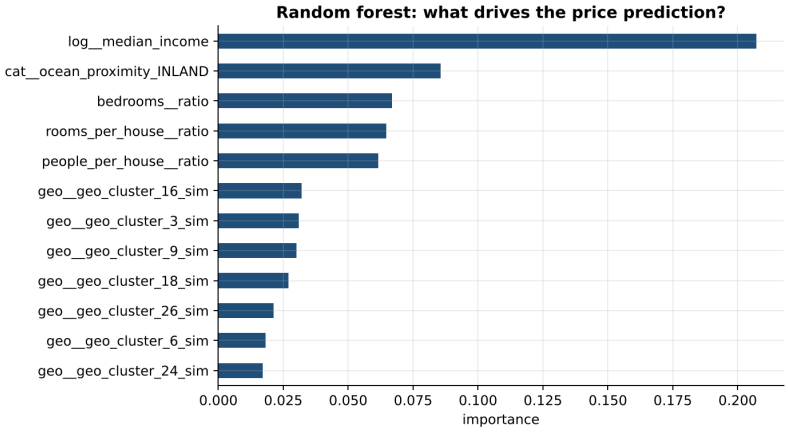



***Figure 6.*** *Income (log) leads; being INLAND is second; the three engineered ratios follow, and several geographic-cluster similarities carry location information.*

Importances sum to 1 and measure how much each feature reduced impurity across the forest. Useful, but treat it as a *hint*, not proof of causation: correlated features share importance, and impurity-based importance favours features with many distinct values. It also suggests pruning candidates: features with ~0 importance can be dropped, simplifying the system.

## Error analysis

**CODE · steps/step03_pipeline_models.py · block “error_analysis”**

```
# Out-of-fold predictions: every district is predicted by a model that never saw it.
from sklearn.model_selection import cross_val_predict
oof = cross_val_predict(clone(best), X, y, cv=5, n_jobs=1)
err = pd.DataFrame({"actual": y, "pred": oof, "abs_err": np.abs(y - oof),
                    "ocean": X.ocean_proximity, "income": X.median_income, "lat": X.latitude, "lon":
                        X.longitude})
print("OOF RMSE = %.0f | OOF MAE = %.0f" % (rmse(y, oof), err.abs_err.mean()))
print("\nError by ocean proximity (MAE, $):")
print(err.groupby("ocean").abs_err.agg(["count", "mean"]).round(0).to_string())
capped = err[err.actual >= 500001]
print("\nCapped districts (price = 500,001): n=%d, mean prediction=%.0f -> models cannot learn beyond the cap" %
      (len(capped), capped.pred.mean()))
record("capped_mean_pred", capped.pred.mean())
```

**OUTPUT · produced by running the block above**

```
OOF RMSE = 42549 | OOF MAE = 26945

Error by ocean proximity (MAE, $):
            count     mean
ocean
<1H OCEAN    7277  28773.0
INLAND       5262  19865.0
ISLAND          2  57374.0
NEAR BAY     1847  31230.0
NEAR OCEAN   2124  34467.0

Capped districts (price = 500,001): n=786, mean prediction=439826 -> models cannot learn beyond the cap
```

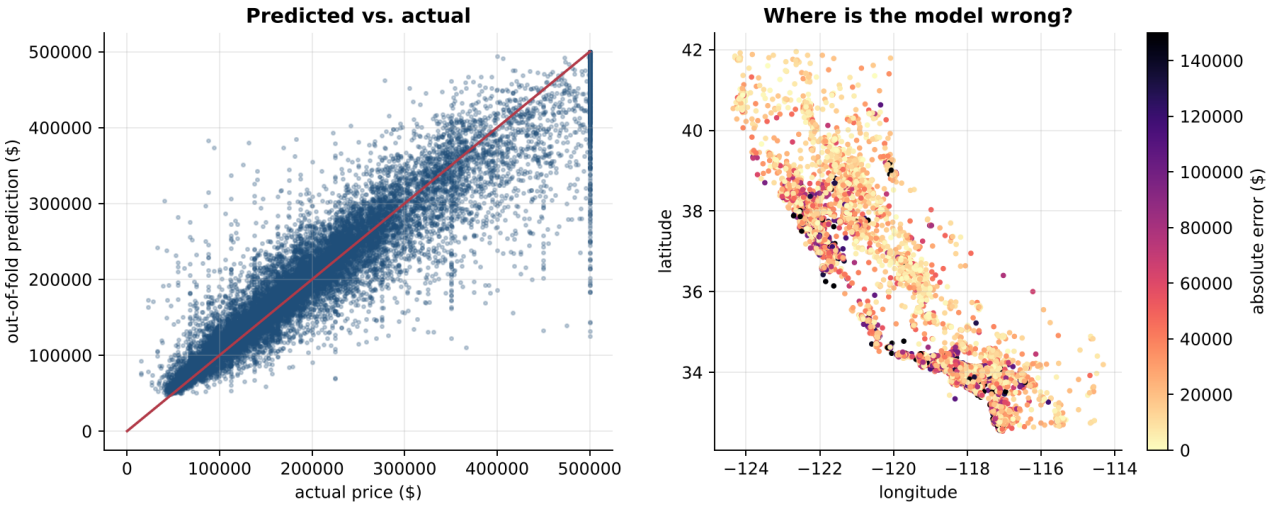



***Figure 7.*** *Left: predictions versus truth hug the diagonal but flatten at the cap. Right: large errors cluster in dense metropolitan areas where prices vary hugely over short distances.*

We used **out-of-fold predictions** (`cross_val_predict`): each district is predicted by a model that never saw it, so these errors are honest. Three findings: (1) coastal and bay districts have larger absolute errors than inland ones, because prices there are higher and more variable; (2) the **capped** districts (true value ≥ $500,001) get a mean prediction of about $439,826: the model cannot see beyond the cap; (3) the geographic error map says 'add finer location information or neighbourhood-level data'. These are the kinds of statements a stakeholder can act on, and they come from error analysis, not from a single number.

## 2.4 Hands-on experiments

1. In `step03`, replace `RandomForestRegressor` by `HistGradientBoostingRegressor` inside the same pipeline. How much does CV RMSE change?
2. Remove the three ratio pipelines from `build_preprocessing`. Re-run the CV. What happens? (Which ratio mattered most?)
3. Deliberately create leakage: scale all of `X` with `StandardScaler` *before* `cross_val_score`. Compare the score with the pipeline version; explain the difference (it will be small here, but explain why it is not zero).
4. Change `OneHotEncoder` to `OrdinalEncoder`. Which models are hurt, and why?
5. Use `TransformedTargetRegressor` with `np.log`/`np.exp` on the price. Does RMSE in dollars improve?

## 2.5 Challenge questions

**Q1. Why must you fit the scaler on the training data only?**

The scaler's mean/std summarise the data it is fitted on. Fitting on all the data puts test-set statistics inside the training procedure (**leakage**), making validation optimistic. Fit on train, then only *transform* validation/test data with those stored values.

**Q2. Explain stratified sampling and when it is needed.**

Divide the population into strata (homogeneous groups) and sample the correct proportion from each, so the sample matches the population on the stratifying attribute. Needed when one attribute is highly predictive and a random split could mis-represent it, especially with small datasets or rare classes.

**Q3. A decision tree has RMSE 0 on training data. Is it a great model?**

Almost certainly not: it has memorised the training set (overfitting). Judge it by cross-validated or test error. Here CV RMSE is $67k, worse than the forest.

**Q4. What is the difference between `fit_transform` on train and `transform` on test?**

`fit_transform` learns parameters (medians, means, category lists, cluster centres) and applies them. `transform` re-uses the *stored* parameters without learning, so new data is treated exactly like the training data.

**Q5. Why log-transform skewed features?**

Heavy tails let a few extreme values dominate scaling, distances and gradient steps. A log pulls the tail in, making the distribution closer to symmetric so standard scaling and linear relationships work better. Alternatives: bucketising, or a power transform.

**Q6. Grid search versus randomised search?**

Grid tries all combinations (cost grows multiplicatively with the number of hyper-parameters) and only values you list. Randomised samples a fixed budget from distributions: more efficient in high dimensions, explores continuous ranges, and the budget is directly controlled.

**Q7. Why use `handle_unknown='ignore'` in the one-hot encoder?**

In production a category unseen in training may appear. With `ignore`, it becomes a row of zeros instead of raising an error that crashes the service.

**Q8. Why does the median beat the mean for imputation here?**

The attributes are heavily skewed with outliers, and the mean is pulled by them; the median is a typical value and robust.

**Q9. What does 'the data is censored at 500,001' imply for the model and for the business?**

Labels are lower bounds for those districts, so the model systematically under-predicts in the most expensive areas and cannot be trusted above the cap. Either obtain true values, drop capped rows and restrict the claimed range, or model the censoring explicitly (survival-style methods).

**Q10. Why persist the entire pipeline instead of only the model?**

At inference, raw data must go through *exactly* the same preprocessing as in training. A saved pipeline guarantees it, eliminating training/serving skew.

**Q11. What would you monitor after deployment?**

Input distributions (drift), prediction distributions, error against delayed ground truth, data-quality checks (missing rate, caps, new categories), latency, and the age of the model with an automatic retraining trigger.

# Chapter 3 · Classification

**Project step 4** — build the HomeLens *premium-district screener*: a district is **premium** if its median value is in the top 25% of the training prices (about $263,900 or more). You will meet the accuracy trap, the confusion matrix, precision/recall, thresholds, ROC and PR curves, multiclass strategies and error analysis. Code: `steps/step04_classification.py`.

## 3.1 Concepts you must own

### Why accuracy is not enough

Accuracy is the fraction of correct predictions. When one class dominates it is useless: a classifier that always says 'not premium' is 75% accurate here and finds *nothing*. The same trap appears in fraud, disease, defects, churn. You need metrics that look at each class's errors separately.

### The confusion matrix and its metrics

| | Predicted negative | Predicted positive |
|---|---|---|
| **Actual negative** | TN (true negative) | FP (false positive) — a false alarm, *type I error* |
| **Actual positive** | FN (false negative) — a miss, *type II error* | TP (true positive) |

| Metric | Formula | Answers the question | Optimise when |
|---|---|---|---|
| **Precision** | TP / (TP + FP) | When it says positive, how often is it right? | False alarms are costly (a broker wastes a day on a bad lead) |
| **Recall** (sensitivity, TPR) | TP / (TP + FN) | Of all real positives, how many did it find? | Misses are costly (missed fraud, missed disease) |
| **$F_1$** | 2·P·R / (P + R) = TP / (TP + (FN + FP)/2) | Harmonic mean: high only if *both* are high | You want a single balance number |
| **Specificity** (TNR) | TN / (TN + FP) | How many negatives were correctly rejected? | Paired with recall in ROC analysis |
| **FPR** | FP / (FP + TN) = 1 − specificity | x-axis of the ROC curve | — |

The harmonic mean punishes imbalance: precision 1.0 with recall 0.1 gives $F_1 \approx 0.18$, not 0.55. Use $F_\beta$ to weight recall β times more than precision.

### The precision/recall trade-off and the threshold

Most classifiers compute a **score** (a decision function value or a probability) and compare it with a **threshold**. Raise the threshold: fewer positives, higher precision, lower recall. Lower it: the reverse. No threshold gives both. Precision can *bounce* when the threshold moves, but recall only decreases when the threshold rises. The business sets the threshold from the relative costs of FP and FN. Tools: `decision_function` or `predict_proba`, then `precision_recall_curve`. Never state 'we reached 99% precision' without the recall at that point.

### ROC curve, AUC and when to prefer PR

The **ROC curve** plots recall (TPR) against the false-positive rate as the threshold varies. A random classifier is the diagonal; a perfect one hugs the top-left corner. **AUC** is the area under the curve (0.5 = random, 1.0 = perfect), and can be read as *the probability that a random positive is scored higher than a random negative*. Prefer the **precision–recall curve** (and average precision) when the positive class is rare or when you care more about false positives than false negatives, since ROC can look optimistic when negatives are plentiful.

### Multiclass strategies

| Strategy | How it works | # binary models for K classes | Notes |
|---|---|---|---|
| Native multiclass | Algorithm handles K classes directly | 1 | Logistic (softmax), random forests, naive Bayes, k-NN, trees |
| **One-versus-the-rest** (OvR / OvA) | One detector per class (class vs the rest); predict the class with the highest score | K | Default for most binary-only learners such as SGD |
| **One-versus-one** (OvO) | One classifier per *pair* of classes; the class winning most duels is chosen | K(K−1)/2 | Each trains on a subset only; preferred for SVMs, which scale badly with training size |

**Error analysis.** Look at the *normalised* confusion matrix (each row divided by its class size) and ask what kinds of mistakes dominate. Then improve with targeted features, more data for confused classes or pre-processing. **Multilabel** classification outputs several binary labels per instance; **multioutput** generalises to several outputs, each possibly multiclass (e.g. de-noising an image pixel by pixel).

## 3.2 Project step 4 — hands on

`load_task()` (in `tasks.py`) returns the training rows with a fitted lightweight pipeline: 16 standardised features, plus the labels *premium* (0/1) and *tier* (0–3 quartiles). The premium threshold is the 75th percentile of the **training** prices.

**CODE · steps/step04_classification.py · block “setup”**

```
import sys; sys.path.insert(0, "src")
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.base import clone
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import (ConfusionMatrixDisplay, accuracy_score, confusion_matrix, f1_score,
                             precision_recall_curve, precision_score, recall_score, roc_auc_score, roc_curve)
from sklearn.model_selection import cross_val_predict, cross_val_score
from sklearn.multiclass import OneVsOneClassifier, OneVsRestClassifier
from sklearn.neighbors import KNeighborsClassifier
from homelens.tasks import load_task
from homelens.utils import set_style, savefig, record, PALETTE
from homelens.config import RANDOM_STATE
set_style()
T = load_task()
X, y = T.X, T.y_premium
print("Premium threshold (75th percentile of TRAIN prices): $%.0f" % T.thr)
print("Class balance: premium = %.1f%%  not premium = %.1f%%" % (100 * y.mean(), 100 * (1 - y.mean())))
print("Feature matrix:", X.shape)
```

**OUTPUT · produced by running the block above**

```
Premium threshold (75th percentile of TRAIN prices): $263900
Class balance: premium = 25.0%  not premium = 75.0%
Feature matrix: (16512, 16)
```

### The accuracy trap

**CODE · steps/step04_classification.py · block “accuracy_trap”**

```
dummy = DummyClassifier(strategy="most_frequent")
acc = cross_val_score(dummy, X, y, cv=3, scoring="accuracy")
print("A model that ALWAYS says 'not premium' scores accuracy = %.3f" % acc.mean())
print("... yet it finds 0 premium districts. Accuracy is misleading on skewed classes.")
record("dummy_accuracy", acc.mean())
```

**OUTPUT · produced by running the block above**

```
A model that ALWAYS says 'not premium' scores accuracy = 0.750
... yet it finds 0 premium districts. Accuracy is misleading on skewed classes.
```

The 'always not premium' classifier gets **75%** accuracy because premium is 25% of the data. This is the number any real classifier must put in perspective.

### A first classifier and its confusion matrix

**CODE · steps/step04_classification.py · block “sgd_confusion”**

```
sgd = SGDClassifier(random_state=RANDOM_STATE, loss="hinge", max_iter=2000, tol=1e-4)
pred = cross_val_predict(sgd, X, y, cv=5)
cm = confusion_matrix(y, pred)
print("Confusion matrix (rows = actual, cols = predicted)")
print(pd.DataFrame(cm, index=["actual not-premium", "actual premium"], columns=["pred not", "pred
    premium"]).to_string())
p, r, f = precision_score(y, pred), recall_score(y, pred), f1_score(y, pred)
print("\nprecision = TP/(TP+FP) = %d/%d = %.3f" % (cm[1, 1], cm[1, 1] + cm[0, 1], p))
print("recall    = TP/(TP+FN) = %d/%d = %.3f" % (cm[1, 1], cm[1, 1] + cm[1, 0], r))
print("F1        = harmonic mean         = %.3f" % f)
print("accuracy  = %.3f" % accuracy_score(y, pred))
record("sgd_precision", p); record("sgd_recall", r); record("sgd_f1", f)
```

**OUTPUT · produced by running the block above**

```
Confusion matrix (rows = actual, cols = predicted)
                    pred not  pred premium
actual not-premium     11719           661
actual premium          1457          2675

precision = TP/(TP+FP) = 2675/3336 = 0.802
recall    = TP/(TP+FN) = 2675/4132 = 0.647
F1        = harmonic mean         = 0.716
accuracy  = 0.872
```

We trained an `SGDClassifier` (a linear model trained by stochastic gradient descent, hinge loss) and used `cross_val_predict` so every prediction is out-of-fold. Reading the matrix: 2,675 premium districts were found (TP), 1,457 were missed (FN), 661 non-premium districts were wrongly flagged (FP). **Precision 0.80** says 80% of flagged districts are truly premium; **recall 0.65** says we found only 65% of all premium districts; $F_1 = 0.72$. Accuracy (0.87) hides the fact that a third of the premium districts are missed.

## Choosing a threshold for the business requirement

The brief demanded **precision ≥ 0.85–0.90**. We obtain scores with `method='decision_function'` and search the precision–recall curve for the lowest threshold that achieves 90% precision.

**CODE · steps/step04_classification.py · block “pr_curve”**

```
scores = cross_val_predict(sgd, X, y, cv=5, method="decision_function")
prec, rec, thr = precision_recall_curve(y, scores)
idx90 = np.argmax(prec >= 0.90)
t90 = thr[idx90]
pred90 = scores >= t90
print("Threshold needed for >=90%% precision: %.3f" % t90)
print("   -> precision = %.3f, recall = %.3f" % (precision_score(y, pred90), recall_score(y, pred90)))
print("Default threshold 0:  precision = %.3f, recall = %.3f" % (precision_score(y, scores >= 0),
    recall_score(y, scores >= 0)))
record("sgd_p90_recall", recall_score(y, pred90))
```

**OUTPUT · produced by running the block above**

```
Threshold needed for >=90% precision: 0.664
   -> precision = 0.900, recall = 0.465
Default threshold 0:  precision = 0.802, recall = 0.647
```

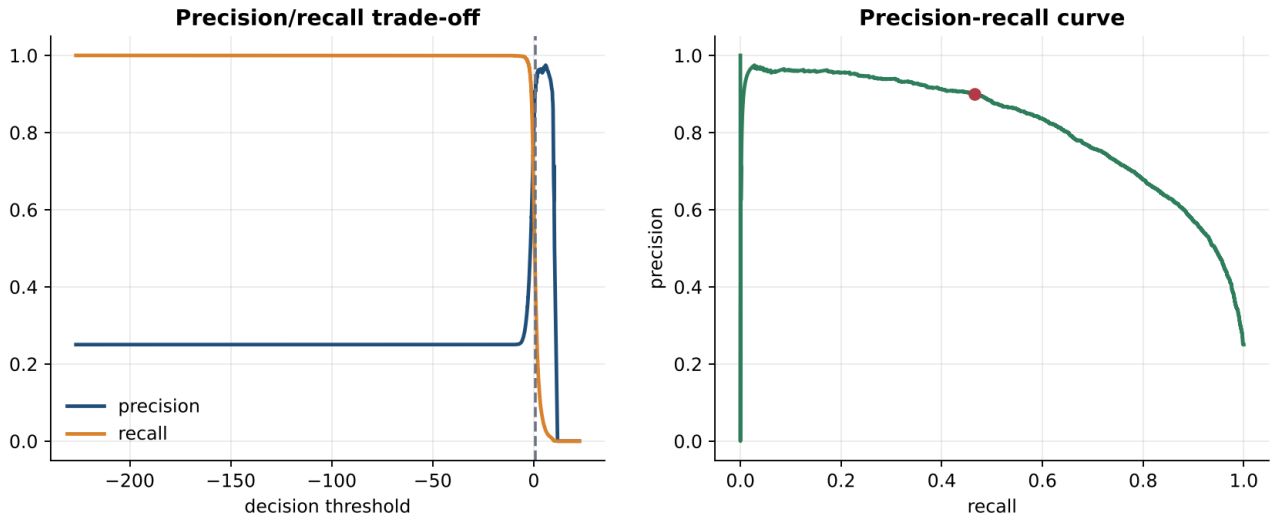



***Figure 8.*** *Left: precision rises and recall falls as the threshold grows (dashed line: the 90%-precision threshold). Right: the precision–recall curve with the chosen operating point.*

Hitting 90% precision with this linear model costs us recall: it falls to **0.46**. The model finds fewer than half of premium districts at that precision. That is the *trade-off made explicit*, and it is the number to show the stakeholders. We can improve both only with a *better model*.

## Comparing models: ROC and average precision

**CODE · steps/step04_classification.py · block “roc_compare”**

```
rf = RandomForestClassifier(n_estimators=100, random_state=RANDOM_STATE, n_jobs=2)
proba_rf = cross_val_predict(rf, X, y, cv=5, method="predict_proba")[:, 1]
auc_sgd, auc_rf = roc_auc_score(y, scores), roc_auc_score(y, proba_rf)
print("ROC AUC  SGD (hinge/linear) = %.4f" % auc_sgd)
print("ROC AUC  Random forest      = %.4f" % auc_rf)
from sklearn.metrics import average_precision_score
print("Average precision (PR-AUC)  SGD = %.4f | RF = %.4f" % (average_precision_score(y, scores),
    average_precision_score(y, proba_rf)))
fig, ax = plt.subplots(figsize=(5.8, 4.8))
for name, s, c in [("SGD linear", scores, PALETTE["orange"]), ("Random forest", proba_rf, PALETTE["blue"])]:
    fpr, tpr, _ = roc_curve(y, s); ax.plot(fpr, tpr, lw=2, color=c, label=f"{name} (AUC {roc_auc_score(y,
        s):.3f})")
ax.plot([0, 1], [0, 1], "k--", lw=1); ax.set_xlabel("false positive rate"); ax.set_ylabel("true positive rate
    (recall)")
ax.set_title("ROC curves"); ax.legend(loc="lower right")
savefig(fig, "fig09_roc")
record("auc_sgd", auc_sgd); record("auc_rf", auc_rf)
prf = proba_rf >= 0.5
print("RF @0.5: precision = %.3f recall = %.3f" % (precision_score(y, prf), recall_score(y, prf)))
record("rf_precision", precision_score(y, prf)); record("rf_recall", recall_score(y, prf))
```

**OUTPUT · produced by running the block above**

```
ROC AUC  SGD (hinge/linear) = 0.9208
ROC AUC  Random forest      = 0.9575
Average precision (PR-AUC)  SGD = 0.8151 | RF = 0.9025
RF @0.5: precision = 0.861 recall = 0.756
```

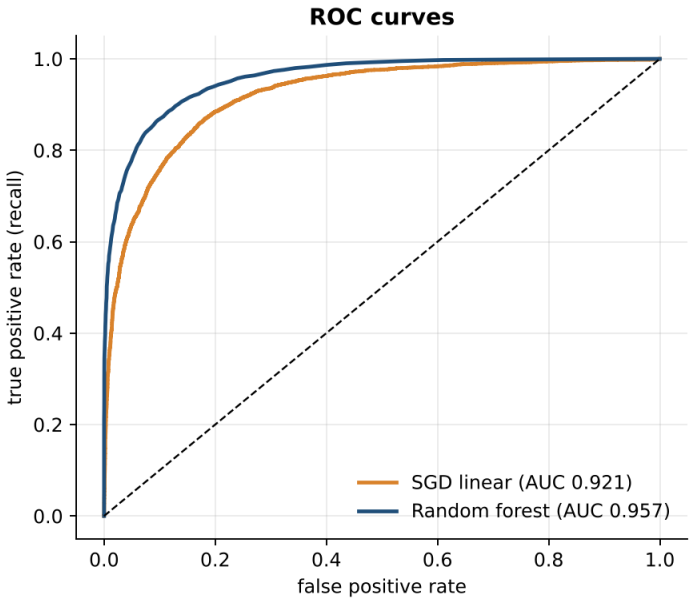



***Figure 9.*** *ROC curves for the linear SGD classifier and a random forest. The forest dominates over the whole range.*

The forest's AUC is **0.957** versus **0.921** for the linear model, and its average precision (the PR-curve area) is far higher (0.90 vs 0.82). At the default threshold it is already better on *both* precision (0.86) and recall (0.76). The non-linear interactions (income × location) matter. Remember: the interpretation 'a random positive outranks a random negative 96% of the time' is the intuition behind AUC.

**ROC or PR here?**
The premium class is 25% of the data (moderately rare) and the business cares about false leads, so **PR curves and average precision** are the primary evidence. We still report ROC-AUC because it is the standard and it is threshold-independent.

## Multiclass: four price tiers

**CODE · steps/step04_classification.py · block “multiclass”**

```
yt = T.y_tier
labels = ["budget", "lower-mid", "upper-mid", "premium"]
print("Tier sizes:", np.bincount(yt))
models = {
    "SGD one-vs-rest (built-in)": SGDClassifier(random_state=RANDOM_STATE, max_iter=2000),
    "SGD one-vs-one (6 binary models)": OneVsOneClassifier(SGDClassifier(random_state=RANDOM_STATE,
        max_iter=2000)),
    "Random forest (native multiclass)": RandomForestClassifier(n_estimators=100, random_state=RANDOM_STATE,
        n_jobs=2),
}
store = {}
for n, m in models.items():
    p_ = cross_val_predict(m, X, yt, cv=5); store[n] = p_
    print(f"{n:36s} accuracy = {accuracy_score(yt, p_):.3f}  macro-F1 = {f1_score(yt, p_,
        average='macro'):.3f}")
record("tier_acc_rf", accuracy_score(yt, store["Random forest (native multiclass)"]))
ovo = OneVsOneClassifier(SGDClassifier(random_state=RANDOM_STATE)).fit(X[:2000], yt[:2000])
print("Number of binary classifiers inside OvO for 4 classes:", len(ovo.estimators_), "= 4*3/2")
print("Number inside OvR for 4 classes:", len(OneVsRestClassifier(SGDClassifier()).fit(X[:2000],
    yt[:2000]).estimators_))
```

**OUTPUT · produced by running the block above**

```
Tier sizes: [4126 4123 4131 4132]
SGD one-vs-rest (built-in)           accuracy = 0.586  macro-F1 = 0.548
SGD one-vs-one (6 binary models)     accuracy = 0.629  macro-F1 = 0.630
Random forest (native multiclass)    accuracy = 0.738  macro-F1 = 0.738
Number of binary classifiers inside OvO for 4 classes: 6 = 4*3/2
Number inside OvR for 4 classes: 4
```

Now predict the quartile tier (budget, lower-mid, upper-mid, premium). `SGDClassifier` is binary, so scikit-learn automatically uses **OvR** (4 detectors); wrapping it in `OneVsOneClassifier` trains **6** (= 4·3/2) pairwise classifiers, which improved accuracy from 0.59 to 0.63 because each pair is easier to separate

linearly. The random forest handles multiclass natively and reaches **0.74** accuracy (chance would be 0.25).

**CODE · steps/step04_classification.py · block "error_analysis"**

```
fig, ax = plt.subplots(1, 2, figsize=(11, 4.4))
pr = store["Random forest (native multiclass)"]
ConfusionMatrixDisplay.from_predictions(yt, pr, display_labels=labels, ax=ax[0], cmap="Blues", colorbar=False)
ax[0].set_title("Counts"); ax[0].grid(False)
ConfusionMatrixDisplay.from_predictions(yt, pr, display_labels=labels, ax=ax[1], cmap="Blues",
                                        normalize="true", values_format=".2f", colorbar=False)
ax[1].set_title("Normalised by true row (=recall)"); ax[1].grid(False)
fig.suptitle("Random forest, 4 price tiers (out-of-fold)", fontweight="bold")
savefig(fig, "fig10_confusion_tiers")
cmn = confusion_matrix(yt, pr)
adjacent = sum(cmn[i, j] for i in range(4) for j in range(4) if abs(i - j) == 1)
far = sum(cmn[i, j] for i in range(4) for j in range(4) if abs(i - j) >= 2)
print("Mistakes landing in an ADJACENT tier: %d | two or more tiers away: %d" % (adjacent, far))
print("=> %.1f%% of all errors are off by only one tier" % (100 * adjacent / (adjacent + far)))
record("adjacent_err_pct", 100 * adjacent / (adjacent + far))
```

**OUTPUT · produced by running the block above**

```
Mistakes landing in an ADJACENT tier: 3937 | two or more tiers away: 386
=> 91.1% of all errors are off by only one tier
```

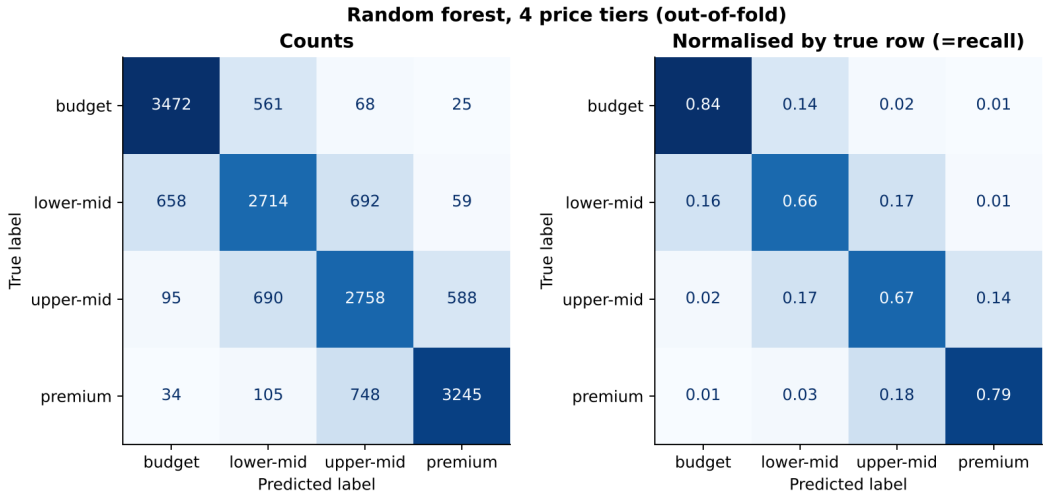



***Figure 10.*** *Confusion matrices for the 4-tier forest: counts (left) and row-normalised (right). The diagonal dominates; almost all errors are one tier away.*

**91%** of all mistakes land in an *adjacent* tier. That is good news for an ordinal target (tiers have an order): most errors are mild. It also suggests the problem is really a *regression* one cut into artificial bins, which is why a regression model with an error bar is more informative for borderline districts.

## Multilabel classification

**CODE · steps/step04_classification.py · block "multilabel"**

```
# Multilabel: each district gets TWO yes/no labels at once.
coastal = T.X_raw["ocean_proximity"].isin(["NEAR BAY", "NEAR OCEAN", "ISLAND"]).to_numpy().astype(int)
Y_multi = np.c_[y, coastal]
knn = KNeighborsClassifier(n_jobs=2)
pm = cross_val_predict(knn, X, Y_multi, cv=3)
print("Labels: [is_premium, is_coastal]")
print("F1 per label:", f1_score(Y_multi, pm, average=None).round(3), "| macro F1 = %.3f" % f1_score(Y_multi, pm,
    average="macro"))
print("Example district -> predicted", pm[0], "actual", Y_multi[0])
```

**OUTPUT · produced by running the block above**

```
Labels: [is_premium, is_coastal]
F1 per label: [0.707 0.992] | macro F1 = 0.849
Example district -> predicted [0 0] actual [0 0]
```

A `KNeighborsClassifier` natively supports multilabel targets, so one model predicts both *is_premium* and *is_coastal*. For multilabel metrics, `average='macro'` gives each label equal weight, while `'weighted'` weights by label frequency. (Coastal is easy to predict from longitude/latitude and the proximity category, which is trivially related to it: $F_1$ 0.99. That label is almost leaked from a feature, a useful

reminder to check whether a label is derivable from the inputs.)

**Common mistakes in classification**
• Reporting accuracy on imbalanced data. • Picking the threshold on the test set. • Using `predict` (0.5 threshold) when the cost of errors is asymmetric. • Reading ROC-AUC as 'quality' for a rare class without checking PR. • Forgetting that **cross-validated predictions** (not training predictions) are needed for any curve you use to choose a threshold. • Using stratified folds? `StratifiedKFold` keeps class ratios in each fold (the default for classifiers in `cross_val_*`).

## 3.3 Hands-on experiments

1. Change the premium quantile from 0.75 to 0.95 (very rare class). Re-run: how do accuracy, precision, recall and the PR curve change? Which metric becomes the most informative?
2. Add `class_weight='balanced'` to the forest. What happens to precision and recall at the default threshold?
3. Compute `fbeta_score` with β = 0.5 and β = 2 at the 90%-precision threshold and say which stakeholder each favours.
4. Plot the ROC curve of the *tier* classifier using OvR probabilities (`roc_auc_score(..., multi_class='ovr')`).
5. Find the 20 districts for which the forest is most confidently wrong. What do they have in common?

## 3.4 Challenge questions

**Q1. Define precision and recall in words and formulas. Which one would you maximise for cancer screening, and for a spam filter?**

Precision = TP/(TP+FP): share of alarms that are real. Recall = TP/(TP+FN): share of real cases found. Cancer screening → high recall (missing a case is worse; follow-up tests weed out false alarms). Spam filter → high precision (blocking a real email is worse than letting spam through).

**Q2. Why can't you have both 100% precision and 100% recall just by moving the threshold?**

The threshold only trades one for the other along a curve defined by the model's scores. Only a better model (better-separated scores) shifts the whole curve.

**Q3. Your classifier has 98% accuracy and 0% recall on the positive class. What happened?**

The positives are rare (≈2%) and the model always predicts negative. Accuracy is misleading; evaluate with recall/precision/PR-AUC, rebalance, or change the threshold/loss.

**Q4. What does AUC = 0.5 mean? AUC = 0.9?**

0.5: no discriminative power (random ranking). 0.9: a randomly chosen positive is scored above a randomly chosen negative 90% of the time.

**Q5. Explain OvR versus OvO; which is cheaper for 10 classes and an SVM?**

OvR trains 10 classifiers on the *full* data; OvO trains 45 on pairs, each seeing only two classes' data. Because SVM training cost grows super-linearly with sample size, OvO's smaller subsets can be cheaper overall; scikit-learn uses OvO for SVMs.

**Q6. How do you choose the decision threshold in practice?**

Estimate cost per FP and per FN (or a minimum precision requirement), use out-of-fold scores to trace the PR (or cost) curve, pick the threshold that minimises expected cost or meets the requirement, freeze it, and only then evaluate once on the test set.

**Q7. Why does `cross_val_predict` matter for threshold selection?**

Scores on training data are overly confident (the model has seen the labels). Out-of-fold scores resemble what the model will output on new data, so the chosen threshold generalises.

**Q8. What is the difference between multilabel and multiclass?**

Multiclass: exactly one of K classes per instance. Multilabel: any subset of K labels (each yes/no) per instance, e.g. all people recognised in a photo.

**Q9. A confusion matrix shows large off-diagonal counts between two classes. What would you do?**

Inspect the examples, add features that separate them, gather more data for them, tweak preprocessing, or consider merging classes or using a pairwise (OvO) specialist. Error analysis guides the intervention.

**Q10. Why might PR curves be better than ROC when positives are rare?**

ROC's x-axis is FP/(all negatives): with many negatives, even many false positives give a small FPR so the curve looks great. Precision uses FP relative to predicted positives, so it reveals the practical burden of false alarms.

# Chapter 4 · Training Models

**Project step 5** — open the black box. You will solve linear regression three ways (normal equation, batch gradient descent, stochastic/mini-batch), read learning curves, regularise, stop early, and implement logistic and softmax regression. This chapter is the **conceptual bridge to neural networks**: everything about gradient descent here is exactly what Keras does later. Code: `steps/step05_training_models.py`.

## 4.1 Concepts you must own

### Linear regression

The prediction is a weighted sum of the features plus a bias: $\hat{y} = \theta_0 + \theta_1 x_1 + \ldots + \theta_n x_n = \theta \cdot x$ (add a constant feature $x_0 = 1$ so the bias is part of θ). Training means finding the θ that minimises the cost, here the **MSE** (mean squared error): $\mathrm{MSE}(\theta) = (1/m)\ \Sigma\ (\theta \cdot x^{(i)} - y^{(i)})^2$. In matrix form $\mathrm{MSE}(\theta) = (1/m)\|X\theta - y\|^2$. The cost is a **convex** bowl: no local minima, a single global minimum.

### Solving it directly: the normal equation

Setting the gradient to zero gives the closed form $\theta = (X^T X)^{-1} X^T y$. Scikit-learn's `LinearRegression` actually uses the **SVD-based pseudo-inverse** $\theta = X^+ y$, which also works when $X^T X$ is singular (redundant features) and is numerically safer. Complexity: about $O(n^{2.4})$–$O(n^3)$ in the *number of features* n (slow for tens of thousands of features), but **linear in the number of instances m**. Predictions are cheap.

### Gradient descent: the engine of modern ML

When n is huge, or when there is no closed form (logistic regression, neural nets), we *walk downhill*: start from random θ, compute the gradient of the cost, step in the opposite direction, repeat. For MSE the gradient is $\nabla_\theta \mathrm{MSE} = (2/m)\ X^T(X\theta - y)$ and the update is $\theta \leftarrow \theta - \eta\ \nabla_\theta \mathrm{MSE}$, where **η is the learning rate**.

| Variant | Gradient computed on | Characteristics | Use |
|---|---|---|---|
| **Batch GD** | The whole training set at every step | Smooth, stable, slow per step; cannot use data too big for memory | Small/medium convex problems |
| **Stochastic GD (SGD)** | One random instance at a time | Very noisy, jumps around the minimum; can escape local minima/plateaus; works out-of-core. Needs a **learning schedule** (decaying η) to settle | Huge data, online learning |
| **Mini-batch GD** | A small random batch (e.g. 32–512) | Compromise; vectorised on GPUs; the standard for deep learning | Almost all neural network training |

> **Three facts about gradient descent you must be able to state**
> 1. **The learning rate matters hugely**: too small converges painfully slowly; too large overshoots and *diverges*. 2. **Features must be on similar scales**; otherwise the cost surface is an elongated valley and GD zig-zags and needs many more iterations (the normal equation does not care). 3. **An epoch** is one pass through the whole training set; stop when the gradient norm is below a tolerance (convergence) or, better, when validation error stops improving (early stopping).

### Polynomial regression, and the bias/variance trade-off

A linear model can fit non-linear data if you add powers of the features (`PolynomialFeatures`): the model stays *linear in its parameters*. With n features and degree d you get $(n + d)!/(d!\ n!)$ features, which explodes combinatorially. More flexible models fit better but risk overfitting. A model's generalisation error decomposes into three parts:

| Component | Meaning | Cause | Reduce by |
|---|---|---|---|
| **Bias** | Wrong assumptions, e.g. 'it is linear' | Underfitting | More flexible model, better features |
| **Variance** | Excessive sensitivity to small changes in the training data | Overfitting (too many degrees of freedom) | More data, regularisation, simpler model, ensembling |
| **Irreducible error** | Noise in the data itself | Measurement error, missing information | Clean the data; cannot be modelled away |

Increasing complexity *increases variance and reduces bias*. The art is the sweet spot. **Learning curves** (training and validation error as a function of training-set size) diagnose which side you are on: if both curves plateau close together and *high*, the model underfits (more data will not help; need a better model); if there is a *gap* between a low training error and a higher validation error, it overfits (more data helps).

### Regularised linear models

| Model | Cost function (α controls strength) | Effect | Notes |
|---|---|---|---|
| **Ridge** ($\ell_2$) | MSE + $\alpha \cdot \frac{1}{2} \Sigma \theta_j^2$ | Shrinks all weights towards 0; none exactly 0 | Closed form exists; **scale features first**; bias $\theta_0$ is not regularised |
| **Lasso** ($\ell_1$) | MSE + $\alpha \cdot \Sigma \lvert\theta_j\rvert$ | Drives unimportant weights **exactly to 0**: automatic feature selection (sparse model) | Can be erratic with correlated features |
| **Elastic net** | MSE + $r \cdot \alpha \cdot \Sigma\lvert\theta_j\rvert + (1-r) \cdot \alpha/2 \cdot \Sigma\theta_j^2$ | Mix of both (r = l1_ratio) | Safe default over lasso when n > m or features correlate |

Why does $\ell_1$ give zeros? The $\ell_1$ penalty's gradient has constant magnitude, so it pushes small weights all the way to zero, whereas the $\ell_2$ gradient shrinks with the weight and never quite arrives. **Early stopping** is regularisation by stopping: for gradient-based learners, validation error falls and then rises as the model starts to overfit; stop at the minimum (and keep the best weights).

### Logistic regression (a classifier!)

Compute a linear score $t = \theta \cdot x$, then squash it with the **sigmoid** $\sigma(t) = 1/(1 + e^{-t})$ to get a probability $\hat{p} = \sigma(\theta \cdot x)$. Predict class 1 if $\hat{p} \geq 0.5$, i.e. when $\theta \cdot x \geq 0$ — the decision boundary is a *hyperplane*. The cost is the **log loss**: $J(\theta) = -(1/m) \Sigma [\, y \log \hat{p} + (1 - y) \log(1 - \hat{p}) \,]$, which punishes confident wrong answers severely. It is convex, has no closed form, and the gradient is strikingly similar to linear regression: $\nabla_\theta J = (1/m)\, X^T(\sigma(X\theta) - y)$. Coefficients have a meaning: $\theta_j$ is the change in the **log-odds** per unit of $x_j$, so $e^{\theta_j}$ multiplies the odds. Scikit-learn regularises by default ($\ell_2$), controlled by **C = 1/α** (larger C = weaker regularisation).

### Softmax regression (multinomial logistic)

For K classes compute a score per class $s_k(x) = \theta_k \cdot x$, then $\hat{p}_k = \exp(s_k) / \Sigma_j \exp(s_j)$: probabilities that sum to 1. Train by minimising the **cross-entropy** $-(1/m)\, \Sigma_i \Sigma_k\, y_k^{(i)} \log \hat{p}_k^{(i)}$. With K = 2 it reduces to logistic regression. This softmax + cross-entropy pair is *exactly* the output layer of most neural network classifiers in Part II. Note that it predicts one class at a time (multiclass, not multilabel).

## 4.2 Project step 5 — hands on

We use four standardised numeric features (income, house age, bedroom ratio, latitude). Outliers are **clipped at ±4 standard deviations**: a handful of extreme rows can otherwise make mini-batch gradients explode, a real-world gotcha. The target is expressed in units of $100,000 to keep gradients friendly.

**CODE · steps/step05_training_models.py · block “setup”**

```
import sys; sys.path.insert(0, "src")
import numpy as np, pandas as pd, matplotlib.pyplot as plt, time
from sklearn.linear_model import (ElasticNet, Lasso, LinearRegression, LogisticRegression, Ridge, SGDRegressor)
from sklearn.model_selection import learning_curve, train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.metrics import log_loss
from homelens.tasks import load_task
from homelens.utils import set_style, savefig, record, rmse, PALETTE
from homelens.config import RANDOM_STATE
set_style()
T = load_task()
cols = ["num__median_income", "num__housing_median_age", "num__bedrooms_ratio", "num__latitude"]
idx = [T.feature_names.index(c) for c in cols]
Xn = np.clip(T.X[:, idx], -4, 4)    # 4 standardised features, outliers clipped at +/-4 std (keeps SGD stable)
y100 = T.y_value.to_numpy() / 1e5   # price in units of $100,000 (keeps gradient numbers friendly)
Xb = np.c_[np.ones(len(Xn)), Xn]    # add the bias column x0 = 1
print("Design matrix with bias:", Xb.shape)
```

**OUTPUT · produced by running the block above**

```
Design matrix with bias: (16512, 5)
```

## Solution 1: the normal equation

**CODE · steps/step05_training_models.py · block “normal_equation”**

```
# theta = (X^T X)^(-1) X^T y  -- the closed-form solution
theta_ne = np.linalg.inv(Xb.T @ Xb) @ Xb.T @ y100
theta_pinv = np.linalg.pinv(Xb) @ y100              # SVD-based, safe when X^T X is singular
lr = LinearRegression().fit(Xn, y100)
print("%-22s %s" % ("normal equation:", np.round(theta_ne, 4)))
print("%-22s %s" % ("pseudo-inverse (SVD):", np.round(theta_pinv, 4)))
print("%-22s %s" % ("sklearn LinearRegression:", np.round(np.r_[lr.intercept_, lr.coef_], 4)))
print("Max difference between the three:", np.abs(theta_ne - np.r_[lr.intercept_, lr.coef_]).max().round(10))
print("RMSE in $: %.0f" % (100000 * rmse(y100, Xb @ theta_ne)))
for c, w in zip(["bias"] + cols, theta_ne):
    print(f"   {c:26s} {w * 1e5:+10.0f} $ per +1 std-dev")
```

**OUTPUT · produced by running the block above**

```
normal equation:       [ 2.081   1.0788  0.2105  0.3511 -0.0465]
pseudo-inverse (SVD):  [ 2.081   1.0788  0.2105  0.3511 -0.0465]
sklearn LinearRegression: [ 2.081   1.0788  0.2105  0.3511 -0.0465]
Max difference between the three: 0.0
RMSE in $: 75654
   bias                        +208096 $ per +1 std-dev
   num__median_income          +107877 $ per +1 std-dev
   num__housing_median_age      +21048 $ per +1 std-dev
   num__bedrooms_ratio          +35113 $ per +1 std-dev
   num__latitude                 -4652 $ per +1 std-dev
```

Three different routes (explicit inverse, SVD pseudo-inverse and scikit-learn) give **identical** parameters. The coefficients are interpretable *because the features are standardised*: +1 standard deviation of income raises the predicted price by about $108k, holding the other features fixed. Notice how the bias $\theta_0$ is the mean price, as expected after centring.

## Solution 2: batch gradient descent and the learning rate

**CODE · steps/step05_training_models.py · block “batch_gd”**

```
# Batch gradient descent: theta <- theta - eta * (2/m) X^T (X theta - y)
def batch_gd(Xb, y, eta, n_iter=200):
    m = len(Xb); theta = np.zeros(Xb.shape[1]); hist = []
    for _ in range(n_iter):
        err = Xb @ theta - y
        hist.append(np.mean(err ** 2))
        theta = theta - eta * (2 / m) * Xb.T @ err
    return theta, np.array(hist)

etas = [0.002, 0.02, 0.2, 1.0]
fig, ax = plt.subplots(figsize=(6.8, 4.3))
for eta, c in zip(etas, [PALETTE["grey"], PALETTE["orange"], PALETTE["green"], PALETTE["red"]]):
    th, h = batch_gd(Xb, y100, eta)
    status = "DIVERGES" if (not np.isfinite(h[-1]) or h[-1] > h[0]) else "converges"
    print(f"eta = {eta:<6}: final MSE = {h[-1]:.4g}  |theta - theta_NE| = {np.linalg.norm(th - theta_ne):.4f}
        -> {status}")
    ax.plot(np.clip(h, 0, 1e6), label=f"eta={eta}", color=c, lw=2)
ax.set_yscale("log"); ax.set_xlabel("iteration"); ax.set_ylabel("MSE (log scale)")
ax.set_title("Learning-rate effect on batch gradient descent"); ax.legend()
savefig(fig, "fig11_gd_learning_rate")
th_bgd, h = batch_gd(Xb, y100, 0.2, n_iter=1000)
print("\nAfter 1000 iterations at eta=0.2:  |theta - theta_NE| =", np.linalg.norm(th_bgd - theta_ne).round(8))
```

**OUTPUT · produced by running the block above**

```
eta = 0.002 : final MSE = 1.687  |theta - theta_NE| = 1.2451  -> converges
eta = 0.02  : final MSE = 0.5752  |theta - theta_NE| = 0.0989  -> converges
eta = 0.2   : final MSE = 0.5724  |theta - theta_NE| = 0.0000  -> converges
eta = 1.0   : final MSE = 2.552e+130  |theta - theta_NE| =
    272204396731722482882162834579695717989829383020239006017794342912.0000  -> DIVERGES

After 1000 iterations at eta=0.2:  |theta - theta_NE| = 0.0
```

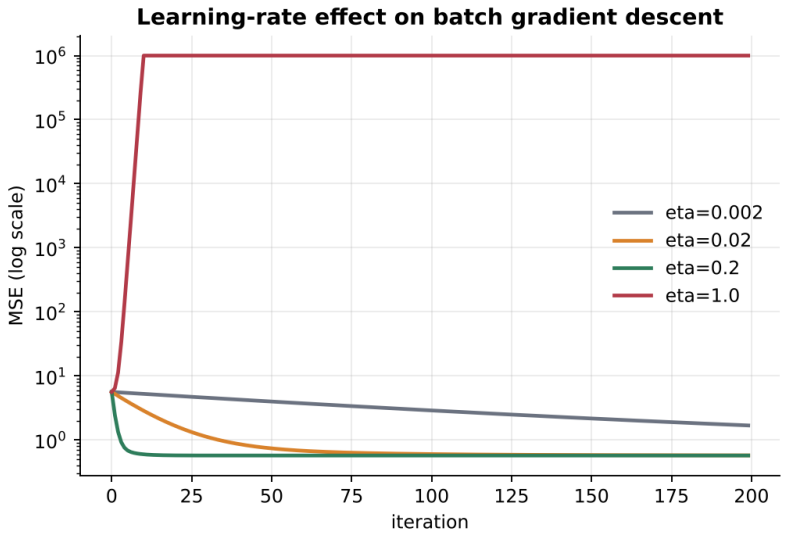



***Figure 11.*** *Batch gradient descent with four learning rates (log y-axis). η = 0.002: still far from the minimum after 200 iterations. η = 0.02: decent. η = 0.2: fastest. η = 1.0:* ***diverges*** *(the curve shoots upward).*

This is the single most useful picture in this chapter. **Too small** a learning rate wastes compute; **too large** overshoots the valley and the loss *grows*. The optimum sits just below the divergence threshold. In deep learning you will tune the very same knob with learning-rate schedulers and finders. After 1,000 iterations at η = 0.2 the result matches the normal equation to four decimal places: gradient descent *is* solving the same problem.

## Solutions 3 and 4: stochastic and mini-batch GD

**CODE · steps/step05_training_models.py · block “sgd_minibatch”**

```
rng = np.random.RandomState(RANDOM_STATE)
def schedule(t, t0=5, t1=50): return t0 / (t + t1)

def sgd(Xb, y, epochs=30):
    m = len(Xb); theta = np.zeros(Xb.shape[1]); path = []
    for epoch in range(epochs):
        for i in range(m):
            k = rng.randint(m)
            g = 2 * Xb[k] * (Xb[k] @ theta - y[k])
            theta = theta - schedule(epoch * m + i) * g
        path.append(np.mean((Xb @ theta - y) ** 2))
    return theta, path

def minibatch(Xb, y, epochs=30, bs=64, eta=0.05):
    m = len(Xb); theta = np.zeros(Xb.shape[1]); path = []
    for epoch in range(epochs):
        order = rng.permutation(m)
        for s in range(0, m, bs):
            b = order[s:s + bs]
            theta = theta - eta * (2 / len(b)) * Xb[b].T @ (Xb[b] @ theta - y[b])
        path.append(np.mean((Xb @ theta - y) ** 2))
    return theta, path

t0 = time.time(); th_sgd, p_sgd = sgd(Xb, y100, epochs=10); t_sgd = time.time() - t0
t0 = time.time(); th_mb, p_mb = minibatch(Xb, y100); t_mb = time.time() - t0
mse_opt = np.mean((Xb @ theta_ne - y100) ** 2)
print("Optimal MSE (normal equation): %.4f" % mse_opt)
print("SGD  (10 epochs, decaying eta):  final MSE %.4f  |  %.1fs  | distance to optimum %.4f" % (p_sgd[-1],
    t_sgd, np.linalg.norm(th_sgd - theta_ne)))
print("Mini-batch (30 epochs, bs=64):   final MSE %.4f  |  %.1fs  | distance to optimum %.4f" % (p_mb[-1], t_mb,
    np.linalg.norm(th_mb - theta_ne)))
```

**OUTPUT · produced by running the block above**

```
Optimal MSE (normal equation): 0.5724
SGD  (10 epochs, decaying eta):  final MSE 0.5724  |  1.0s  | distance to optimum 0.0078
Mini-batch (30 epochs, bs=64):   final MSE 0.5736  |  0.1s  | distance to optimum 0.0344
```

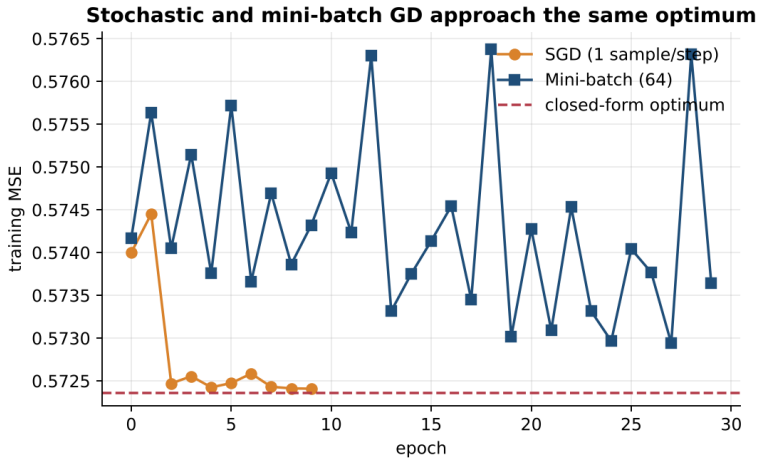



***Figure 12.*** *Both SGD (one instance per step, decaying learning rate) and mini-batch GD (64 per step) approach the closed-form optimum (dashed).*

SGD's noise means it only *hovers* around the minimum; the decaying learning rate (`schedule`) makes the hover tighter over time. Mini-batch GD has less noise per step and, in vectorised form, is the most efficient. Both reach a training MSE within about 0.3% of the optimum. Their parameter vectors are slightly different from the exact solution because the cost surface is flat along some directions (correlated features): the *loss* is optimal even where individual weights are not.

## Learning curves: diagnose before you fix

**CODE · steps/step05_training_models.py · block “learning_curves”**

```
# Same data, three models of different capacity. Watch the gap between the two curves.
f4 = ["num__median_income", "num__housing_median_age", "num__latitude", "num__longitude"]
X4 = np.clip(T.X[:, [T.feature_names.index(c) for c in f4]], -2.5, 2.5)
inc = X4[:, [0]]; yy = T.y_value.to_numpy()
setups = [("Underfit: line on income only", inc, 1), ("Better: line on 4 features", X4, 1), ("Overfit: degree-4
    on 4 features", X4, 4)]
fig, ax = plt.subplots(1, 3, figsize=(13, 3.8), sharey=True)
for a, (ttl, data, deg) in zip(ax, setups):
    model = make_pipeline(PolynomialFeatures(deg, include_bias=False), StandardScaler(), LinearRegression())
    sizes, tr, va = learning_curve(model, data, yy, train_sizes=np.linspace(0.02, 1, 12), cv=5,
                                   scoring="neg_root_mean_squared_error", shuffle=True, random_state=0)
    a.plot(sizes, -tr.mean(1), "o-", label="train", color=PALETTE["blue"]); a.plot(sizes, -va.mean(1), "o-",
        label="validation", color=PALETTE["orange"])
    a.set_title(ttl, fontsize=10); a.set_xlabel("training set size"); a.set_ylim(50000, 140000)
    print(f"{ttl:34s}: final train RMSE = {-tr.mean(1)[-1]:6.0f}, validation RMSE = {-va.mean(1)[-1]:6.0f}, gap
        = {(-va.mean(1)[-1]) - (-tr.mean(1)[-1]):5.0f} | validation at n={sizes[0]}: {-va.mean(1)[0]:.0f}")
ax[0].set_ylabel("RMSE ($)"); ax[0].legend()
savefig(fig, "fig13_learning_curves")
```

**OUTPUT · produced by running the block above**

```
Underfit: line on income only     : final train RMSE =  83501, validation RMSE =  83507, gap =     6 |
    validation at n=264: 83709
Better: line on 4 features        : final train RMSE =  73294, validation RMSE =  73319, gap =    26 |
    validation at n=264: 73975
Overfit: degree-4 on 4 features   : final train RMSE =  65843, validation RMSE =  66191, gap =   348 |
    validation at n=264: 98762
```

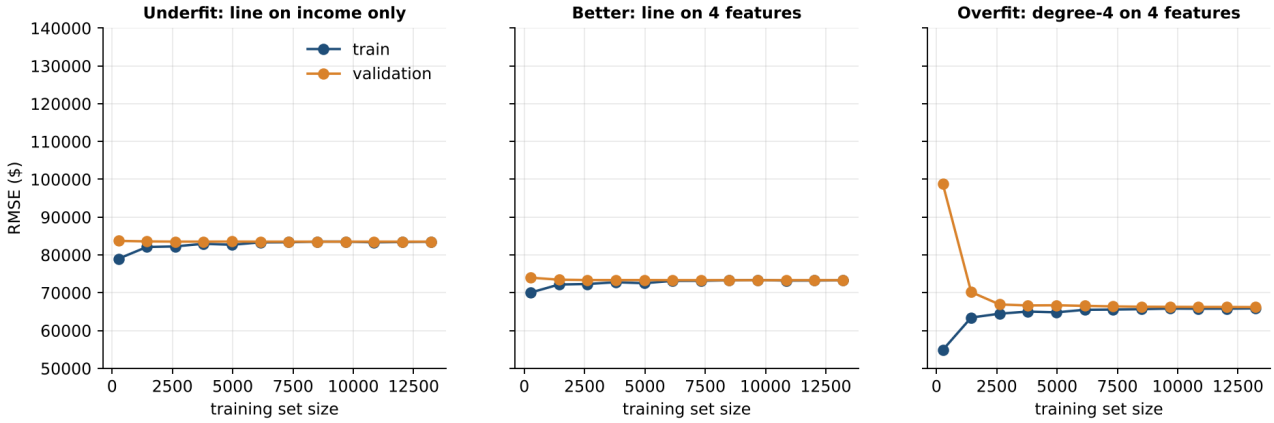



***Figure 13.*** *Learning curves for three models of increasing capacity. Left: a line on income alone: both curves plateau at a high error =* ***underfitting****. Middle: more features lower the plateau. Right: a degree-4 polynomial has a large gap at small n that closes with more data =* ***variance shrinking with data****.*

Use these as a decision tool. Left panel: both errors are high and equal, so *more data is pointless; the model is too simple.* Right panel: with few samples validation error is large (98,762 at n = 264 versus 66,191 at full size), meaning the flexible model is data-hungry; more data or regularisation helps. When you meet a new model, plot its learning curve before deciding what to do next.

## Regularisation in action

**CODE · steps/step05_training_models.py · block “regularisation”**

```
# A tiny training set (150 districts) + a flexible model (degree 4 -> 69 features) = guaranteed overfit.
rng_ = np.random.RandomState(0); perm = rng_.permutation(len(X4)); tr_i, va_i = perm[:150], perm[300:4300]
Xtr, Xva, ytr, yva = X4[tr_i], X4[va_i], yy[tr_i], yy[va_i]
def poly_model(est): return make_pipeline(PolynomialFeatures(4, include_bias=False), StandardScaler(), est)
n_feat = PolynomialFeatures(4, include_bias=False).fit(Xtr).n_output_features_
print("Degree-4 polynomial on 4 features ->", n_feat, "features, but only", len(tr_i), "training rows")
unreg = poly_model(LinearRegression()).fit(Xtr, ytr)
print("Unregularised: train RMSE=%.0f | validation RMSE=%.0f   <- severe overfitting" % (rmse(ytr,
    unreg.predict(Xtr)), rmse(yva, unreg.predict(Xva))))
res = {}
alphas = np.logspace(-1, 4, 16)
for name, mk in [("Ridge (L2)", lambda a: Ridge(alpha=a)), ("Lasso (L1)", lambda a: Lasso(alpha=a,
    max_iter=50000)),
                 ("Elastic net", lambda a: ElasticNet(alpha=a, l1_ratio=0.5, max_iter=50000))]:
    tr_e, va_e, nz = [], [], []
    for a in alphas:
        m = poly_model(mk(a)).fit(Xtr, ytr)
        tr_e.append(rmse(ytr, m.predict(Xtr))); va_e.append(rmse(yva, m.predict(Xva)))
        nz.append(int(np.sum(np.abs(m[-1].coef_) > 1e-8)))
    res[name] = (np.array(tr_e), np.array(va_e), nz)
    b = int(np.argmin(va_e))
    print(f"{name:12s}: best alpha={alphas[b]:8.1f} -> validation RMSE={va_e[b]:.0f} | non-zero coefficients:
        {nz[b]} of {n_feat}")
record("reg_unreg_val", rmse(yva, unreg.predict(Xva))); record("reg_ridge_val", float(res["Ridge
    (L2)"][1].min()))
record("reg_lasso_val", float(res["Lasso (L1)"][1].min())); record("lasso_nonzero", res["Lasso
    (L1)"][2][int(np.argmin(res["Lasso (L1)"][1]))])
```

**OUTPUT · produced by running the block above**

```
Degree-4 polynomial on 4 features -> 69 features, but only 150 training rows
Unregularised: train RMSE=47436 | validation RMSE=302471   <- severe overfitting
Ridge (L2)  : best alpha=   215.4 -> validation RMSE=86139 | non-zero coefficients: 69 of 69
Lasso (L1)  : best alpha=  4641.6 -> validation RMSE=79984 | non-zero coefficients: 14 of 69
Elastic net : best alpha=     2.2 -> validation RMSE=86097 | non-zero coefficients: 69 of 69
```

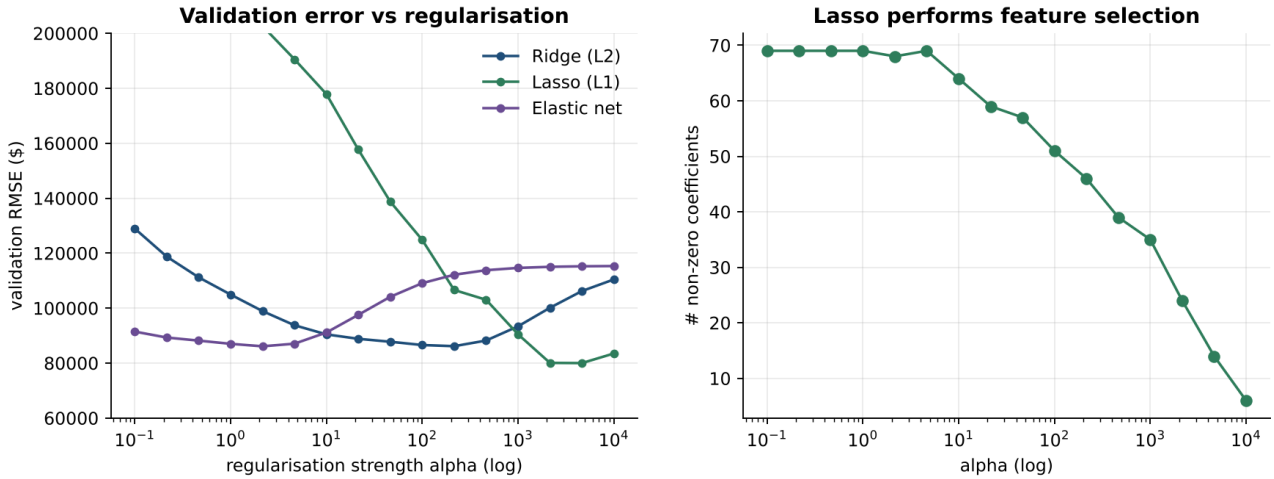



***Figure 14.*** *Left: validation RMSE versus regularisation strength for Ridge, Lasso and Elastic net on an overfit-prone problem. Right: Lasso zeroes out more and more coefficients as α grows.*

The setup is deliberately an *overfit trap*: 150 training rows against 69 polynomial features. Unregularised, the training error is $47k but validation error is **$302,471**: a disaster. **Ridge** brings it to $86,139; **Lasso** to **$79,984** while keeping only **14** of 69 coefficients: it discovered which polynomial terms matter. In both cases the *validation curve is U-shaped in α*: too little regularisation overfits, too much underfits. This is the same U as in Figure 1, mirrored (more regularisation = less complexity).

## Early stopping

**CODE · steps/step05_training_models.py · block “early_stopping”**

```
# Batch/stochastic GD on the SAME overfit-prone problem. Track validation error epoch by epoch.
pre = make_pipeline(PolynomialFeatures(4, include_bias=False), StandardScaler()).fit(Xtr)
Ptr, Pva = pre.transform(Xtr), pre.transform(Xva)
sgd_r = SGDRegressor(penalty=None, learning_rate="constant", eta0=0.001, max_iter=1, tol=None, warm_start=True,
    random_state=RANDOM_STATE)
va_hist, tr_hist = [], []
for epoch in range(400):
    sgd_r.fit(Ptr, ytr / 1e5)                      # target in $100k units keeps the step size sane
    va_hist.append(rmse(yva, 1e5 * sgd_r.predict(Pva))); tr_hist.append(rmse(ytr, 1e5 * sgd_r.predict(Ptr)))
best_epoch = int(np.argmin(va_hist)); best_val = va_hist[best_epoch]
print("Best validation RMSE %.0f reached at epoch %d (train RMSE there: %.0f)" % (best_val, best_epoch,
    tr_hist[best_epoch]))
print("After 400 epochs: validation RMSE %.0f (%+.1f%% worse than best), train RMSE %.0f" % (va_hist[-1], 100 *
    (va_hist[-1] / best_val - 1), tr_hist[-1]))
record("es_best_epoch", best_epoch); record("es_worse_pct", 100 * (va_hist[-1] / best_val - 1))
```

**OUTPUT · produced by running the block above**

```
Best validation RMSE 94537 reached at epoch 24 (train RMSE there: 70909)
After 400 epochs: validation RMSE 105544 (+11.6% worse than best), train RMSE 58617
```

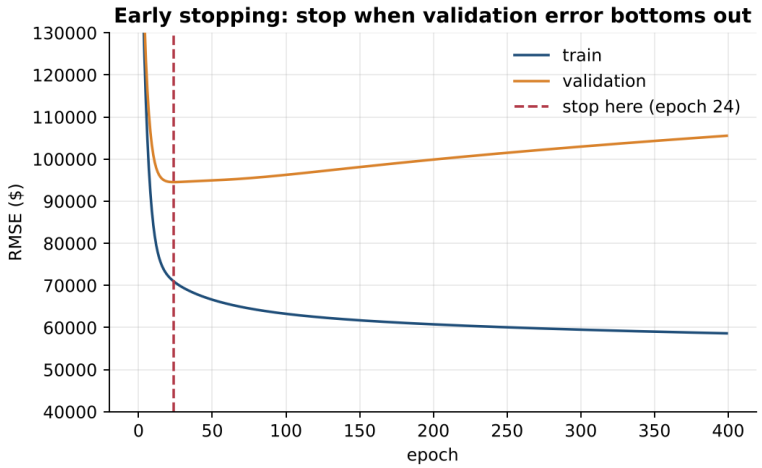



***Figure 15.*** *With SGD, training error keeps decreasing while validation error bottoms out and then rises. Stop at the red line.*

The best validation error occurs at epoch **24**; training on to epoch 400 makes it **12% worse**. In Keras you will use the `EarlyStopping(restore_best_weights=True)` callback that automates precisely this.

## Logistic regression from scratch

**CODE · steps/step05_training_models.py · block “logistic”**

```
# Logistic regression from scratch on 2 features: P(premium) = sigmoid(theta . x)
f2 = [T.feature_names.index("num__median_income"), T.feature_names.index("num__latitude")]
X2 = np.c_[np.ones(len(T.X)), T.X[:, f2]]; y2 = T.y_premium
sigmoid = lambda z: 1 / (1 + np.exp(-z))
theta = np.zeros(3)
for it in range(3000):
    p = sigmoid(X2 @ theta)
    theta -= 0.3 * X2.T @ (p - y2) / len(y2)         # gradient of the log-loss
ll_scratch = log_loss(y2, sigmoid(X2 @ theta))
sk = LogisticRegression(C=1e9, max_iter=1000).fit(X2[:, 1:], y2)
print("from scratch: theta =", theta.round(3), "| log-loss = %.4f" % ll_scratch)
print("scikit-learn: theta =", np.r_[sk.intercept_, sk.coef_[0]].round(3), "| log-loss = %.4f" % log_loss(y2,
    sk.predict_proba(X2[:, 1:])))
print("Accuracy of the 2-feature model: %.3f" % ((sk.predict(X2[:, 1:]) == y2).mean()))
inc_idx = T.feature_names.index("num__median_income")
print("A +1 std-dev rise in income multiplies the odds of being premium by x%.2f" % np.exp(theta[1]))
```

**OUTPUT · produced by running the block above**

```
from scratch: theta = [-1.432  1.728 -0.06 ] | log-loss = 0.3979
scikit-learn: theta = [-1.431  1.728 -0.06 ] | log-loss = 0.3979
Accuracy of the 2-feature model: 0.833
A +1 std-dev rise in income multiplies the odds of being premium by x5.63
```

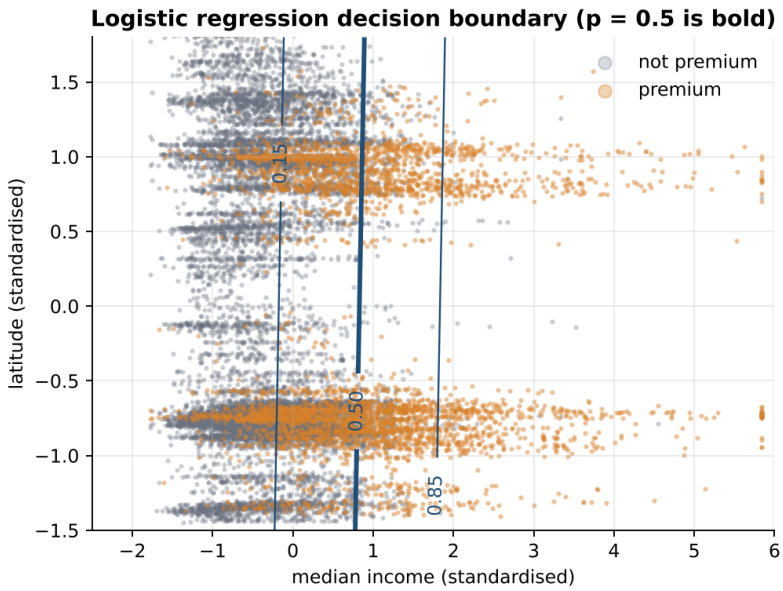



***Figure 16.*** *Logistic regression with two features (income, latitude). The bold line is p = 0.5: the decision boundary is a straight line; the others show p = 0.15 and 0.85.*

Our 6-line gradient-descent loop produces the same coefficients (−1.43, 1.73, −0.06) and the same log-loss (0.398) as scikit-learn: you have now *derived and implemented* logistic regression. The odds interpretation: one standard deviation more income multiplies the odds of being premium by $e^{1.73} \approx 5.6$. The weak latitude coefficient says that, given income, latitude adds little *linearly* (the true geography is non-linear, which is why trees beat this model in Chapter 3).

## Softmax regression

**CODE · steps/step05_training_models.py · block “softmax”**

```
yt = T.y_tier
sm = LogisticRegression(C=10, max_iter=3000).fit(T.X, yt)         # multi-class -> softmax (multinomial) by
    default
print("classes:", sm.classes_, "| coefficient matrix shape (K classes x n features):", sm.coef_.shape)
z = sm.decision_function(T.X[:2])
manual = np.exp(z) / np.exp(z).sum(1, keepdims=True)
print("\nScores z for district 0:", z[0].round(2))
print("softmax by hand        :", manual[0].round(3))
print("sklearn predict_proba  :", sm.predict_proba(T.X[:1])[0].round(3), " (identical)")
print("true tier of district 0:", yt[0])
cv = cross_val_score(LogisticRegression(C=10, max_iter=3000), T.X, yt, cv=5).mean()
print("Softmax regression 5-fold accuracy: %.3f" % cv)
record("softmax_acc", cv)
```

**OUTPUT · produced by running the block above**

```
classes: [0 1 2 3] | coefficient matrix shape (K classes x n features): (4, 16)

Scores z for district 0: [ 4.03  2.11 -0.59 -5.55]
softmax by hand        : [0.864 0.127 0.009 0.   ]
sklearn predict_proba  : [0.864 0.127 0.009 0.   ]  (identical)
true tier of district 0: 0
Softmax regression 5-fold accuracy: 0.640
```

We compute the softmax manually from the class scores and obtain exactly `predict_proba`'s output: district 0 gets 86% budget, 13% lower-mid, 1% upper-mid, ≈0% premium (true tier: 0). A *linear* 4-class model gets 0.64 accuracy, well below the forest's 0.74 from Chapter 3: a reminder that linear boundaries have limits.

**Common mistakes in this chapter**

• Forgetting to scale features before gradient descent or regularisation. • Regularising the bias term. • Using a learning rate by habit instead of plotting the loss curve. • Calling logistic regression 'regression' and applying MSE to it. • Interpreting coefficients of un-standardised or correlated features as importances. • Early stopping on the *training* loss (it never rises!).

## 4.3 Hands-on experiments

1. Without scaling the features, repeat Batch GD with $\eta = 0.2$. What happens, and what $\eta$ now works? Plot the loss.
2. Implement momentum: $v \leftarrow \beta v - \eta \nabla$, $\theta \leftarrow \theta + v$ ($\beta = 0.9$). How many iterations to reach $|\theta - \theta_{NE}| < 0.05$ versus plain GD? (This is the first step toward Adam in Part II.)
3. Add an $\ell_2$ penalty term to your scratch logistic regression gradient (+ $\alpha\theta$ for non-bias terms) and compare with `LogisticRegression(C=1/α)`.
4. Use `GridSearchCV` to tune `ElasticNet`'s `alpha` and `l1_ratio` on the 150-row problem. How many features survive?
5. Replace the clipping by `np.log` on a skewed feature. Is the SGD more stable? Why?

## 4.4 Challenge questions

**Q1. Write the gradient-descent update for linear regression with MSE, and say what each symbol is.**

$\theta \leftarrow \theta - \eta\ (2/m)\ X^T(X\theta - y)$. $\theta$: parameter vector (including bias); $\eta$: learning rate; $m$: training instances; $X$: $m \times (n+1)$ design matrix with a leading column of ones; $y$: targets; $(X\theta - y)$: residual vector.

**Q2. When would you use the normal equation versus gradient descent?**

Normal equation: $n$ (features) small to moderate, need an exact solution, no learning-rate tuning. Gradient descent: very many features, data too large for memory (SGD/mini-batch), models without closed form, online learning.

**Q3. Why does feature scaling speed up gradient descent?**

Unscaled features make the cost surface a long narrow valley; the gradient points across it rather than along it, so GD zig-zags and needs a tiny learning rate. With similar scales the contours are rounder and one learning rate suits all directions.

**Q4. What happens if the learning rate is too large? Too small?**

Too large: overshoots, the loss oscillates or diverges to infinity. Too small: converges very slowly (and may stall on plateaus). Diagnose with a loss-versus-iteration plot.

**Q5. Is the MSE cost of linear regression convex? Of logistic regression's log loss? What does it matter?**

Both are convex, so gradient descent (with a suitable learning rate) reaches the global minimum. Neural networks' losses are *not* convex, so initial conditions and optimiser choices matter more.

**Q6. Explain the bias–variance decomposition and how a learning curve reveals it.**

Error = bias² + variance + irreducible noise. High bias: train and validation errors are both high and converge. High variance: low train error, higher validation error, large gap that shrinks with more data.

**Q7. Why does Lasso produce sparse models and Ridge not?**

The $\ell_1$ penalty's gradient is constant ($\pm\alpha$) so it can push a weight exactly to 0; the $\ell_2$ penalty's gradient is proportional to the weight, so it shrinks weights but never zeroes them. Geometrically, the $\ell_1$ ball has corners on the axes where optima tend to land.

**Q8. Why must you scale features before Ridge or Lasso?**

The penalty treats all weights equally; a feature measured in large units gets a small coefficient and is barely penalised, while one in tiny units gets a big coefficient and is heavily penalised, so regularisation would depend on units.

**Q9. State the formula of the sigmoid and the softmax and how they relate.**

$\sigma(t) = 1/(1 + e^{-t})$. $\text{softmax}(s)_k = e^{s_k}/\Sigma_j e^{s_j}$. For two classes, softmax over $(s, 0)$ equals $\sigma(s)$: logistic regression is the two-class special case.

**Q10. Interpret a logistic-regression coefficient of 0.7.**

Holding other features fixed, a one-unit increase in that feature increases the log-odds by 0.7, i.e. multiplies the odds by $e^{0.7} \approx 2.0$. (Meaningful for comparing features only if they are on comparable scales.)

**Q11. What is early stopping and what is its risk?**

Halting training when validation error stops improving, keeping the best weights. Risk: validation error is noisy (stop too early) and it consumes validation data (so keep a separate test set). Use patience and restore best weights.

**Q12. In SGD, why decrease the learning rate over time?**

Noisy single-sample gradients keep bouncing around the minimum with a large $\eta$. Starting large (fast progress, escape plateaus) and shrinking $\eta$ (settle down) combines speed and precision. Too fast a decay freezes training early.

# Chapter 5 · Support Vector Machines

**Project step 6** — apply large-margin classifiers and regressors to HomeLens. You will see what C and γ really do to a decision boundary, why scaling is non-negotiable, and why kernel SVMs do not scale to big data. Code: `steps/step06_svm.py`.

## 5.1 Concepts you must own

### Large-margin classification

For two linearly separable classes many lines separate them. An SVM chooses the one that stays **as far as possible from the closest training instances of both classes**: it fits the widest possible "street" between the classes. The instances that sit on the edge of the street are the **support vectors**; they alone determine the boundary: adding instances *off* the street changes nothing. The model is $w \cdot x + b$: predict the positive class if it is $\geq 0$. The street's width is $\mathbf{2/\|w\|}$, so *maximising the margin = minimising* $\|w\|$.

### Hard and soft margin

A **hard margin** demands that every instance is outside the street and on the correct side. It works only if the data is linearly separable and is hypersensitive to outliers. The **soft margin** allows *margin violations* with slack variables $\xi^{(i)} \geq 0$ and balances the two goals in one objective: **minimise $\frac{1}{2}\|w\|^2 + C \sum \xi^{(i)}$**.

| Hyper-parameter | Small value | Large value |
|---|---|---|
| **C** (penalty on violations) | Wide street, many violations tolerated, strong regularisation → may **underfit** | Narrow street, few violations tolerated, weak regularisation → may **overfit** |
| **γ** (RBF kernel width: $1/(2\sigma^2)$) | Each instance has a wide influence, boundary smooth → may **underfit** | Each instance has a narrow bell, boundary wiggly and local → may **overfit** |

Equivalent view: the soft-margin cost is the **hinge loss** $\max(0, 1 - t \cdot (w \cdot x + b))$ with $t = \pm 1$, plus an $\ell_2$ penalty. This is why a linear SVM can also be trained by SGD (`SGDClassifier(loss='hinge')`), and why it is a cousin of logistic regression (log loss instead of hinge). Unlike logistic regression an SVM outputs a *score* (distance to the boundary), not a probability; `SVC(probability=True)` adds calibration via an expensive internal cross-validation.

> **SVMs are sensitive to feature scales**
> The margin is measured with a Euclidean distance. A feature with a huge range dominates the geometry and the street becomes narrow in the important directions. **Always standardise** (`StandardScaler`) before an SVM. We prove this experimentally below.

### Non-linear SVMs and the kernel trick

Two ways to handle non-linear data: (1) add polynomial features and fit a linear SVM; (2) use a **kernel**. The **kernel trick** is the key insight: the SVM's dual formulation only needs *dot products* between instances, and a kernel function K(a, b) computes the dot product that the instances *would have* in a higher-dimensional feature space without ever building that space. So you get the effect of thousands or infinite features at the cost of computing K.

| Kernel | K(a, b) | When |
|---|---|---|
| Linear | $a^T b$ | First thing to try, especially with many features or lots of data (use `LinearSVC`, much faster) |
| Polynomial | $(\gamma\, a^T b + r)^d$ | When you believe in polynomial interactions; d and r are hyper-parameters |

| Kernel | K(a, b) | When |
|---|---|---|
| **Gaussian RBF** | $\exp(-\gamma \lVert a - b \rVert^2)$ | The default non-linear choice: acts like a similarity to every training instance |
| Sigmoid, string kernels... | — | Special cases (text, sequences) |

**Rule of thumb:** start with `LinearSVC`; if it is not enough and the data is not huge, try `SVC(kernel='rbf')`; tune C and γ with a grid or randomised search (exponentially spaced values).

### Complexity and what to use

| Class | Time vs. m instances | Needs scaling | Kernel trick | Out-of-core |
|---|---|---|---|---|
| `LinearSVC` | ≈ O(m × n) | yes | no | no |
| `SGDClassifier(loss='hinge')` | O(m × n) | yes | no | **yes** |
| `SVC` | O(m² × n) to O(m³ × n) | yes | **yes** | no |

**SVM regression (SVR)** reverses the objective: fit as many instances as possible *inside* a street of half-width ε. Errors smaller than ε cost nothing (an **ε-insensitive loss**), so adding points inside the tube does not change the model. C again trades flatness for violations; larger ε gives a wider tube and fewer support vectors.

## 5.2 Project step 6 — hands on

Kernel SVMs are expensive, so we work with 12,512 pooled training rows and keep **4,000 rows as an internal hold-out taken from the training data** (the real test set remains locked).

**CODE · steps/step06_svm.py · block “setup”**

```
import sys; sys.path.insert(0, "src")
import numpy as np, pandas as pd, matplotlib.pyplot as plt, time
from sklearn.svm import LinearSVC, SVC, SVR, LinearSVR
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, cross_val_score
from sklearn.metrics import accuracy_score, f1_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from homelens.tasks import load_task
from homelens.utils import set_style, savefig, record, rmse, PALETTE
from homelens.config import RANDOM_STATE
set_style()
T = load_task()
X, y = T.X, T.y_premium
rng = np.random.RandomState(RANDOM_STATE)
perm = rng.permutation(len(X))
hold = perm[:4000]                # an internal hold-out taken from TRAINING data (test set still locked)
pool = perm[4000:]
print("pool:", len(pool), "| internal hold-out:", len(hold))
```

**OUTPUT · produced by running the block above**

```
pool: 12512 | internal hold-out: 4000
```

### A linear SVM versus logistic regression

**CODE · steps/step06_svm.py · block “linear_svm”**

```
t0 = time.time()
lin = LinearSVC(C=1.0, max_iter=20000, random_state=RANDOM_STATE).fit(X[pool], y[pool])
t_svm = time.time() - t0
pr = lin.predict(X[hold])
print("LinearSVC   : accuracy %.3f  F1 %.3f  (fit %.2fs)" % (accuracy_score(y[hold], pr), f1_score(y[hold], pr),
    t_svm))
lr = LogisticRegression(max_iter=3000).fit(X[pool], y[pool]); pl = lr.predict(X[hold])
print("LogisticReg.: accuracy %.3f  F1 %.3f   <- a linear SVM and logistic regression are close cousins" %
    (accuracy_score(y[hold], pl), f1_score(y[hold], pl)))
print("Support vectors are NOT reported for LinearSVC; margin width = 2/||w|| = %.3f" % (2 /
    np.linalg.norm(lin.coef_)))
```

**OUTPUT · produced by running the block above**

```
LinearSVC   : accuracy 0.870  F1 0.714  (fit 0.03s)
LogisticReg.: accuracy 0.871  F1 0.719   <- a linear SVM and logistic regression are close cousins
Support vectors are NOT reported for LinearSVC; margin width = 2/||w|| = 0.855
```

`LinearSVC` fits in about 0.03 s and gets accuracy 0.870, $F_1$ 0.714, essentially tied with logistic regression (0.871 / 0.719): **a linear SVM and logistic regression find nearly the same boundary** because they minimise similar losses with an $\ell_2$ penalty. Pick by what you need: probabilities (logistic) or margin/kernel options (SVM).

## Why scaling matters, demonstrated

**CODE · steps/step06_svm.py · block "scaling_matters"**

```
# SVMs are distance/margin based: unscaled features with very different ranges wreck them.
raw_cols = ["longitude", "latitude", "housing_median_age", "total_rooms", "total_bedrooms", "population",
    "households", "median_income"]
Xraw = T.X_raw[raw_cols].fillna(T.X_raw[raw_cols].median()).to_numpy()
for name, model in [("RBF-SVC, raw features   ", SVC(kernel="rbf", C=1, gamma="scale")),
                    ("RBF-SVC, scaled features", make_pipeline(StandardScaler(), SVC(kernel="rbf", C=1,
                        gamma="scale")))]:
    t0 = time.time(); model.fit(Xraw[pool[:3000]], y[pool[:3000]]); p_ = model.predict(Xraw[hold])
    print(f"{name}: accuracy {accuracy_score(y[hold], p_):.3f}  F1 {f1_score(y[hold], p_):.3f}
        ({time.time()-t0:.1f}s)")
```

**OUTPUT · produced by running the block above**

```
RBF-SVC, raw features   : accuracy 0.752  F1 0.114  (0.3s)
RBF-SVC, scaled features: accuracy 0.878  F1 0.724  (0.2s)
```

Same algorithm, same data, same hyper-parameters. On raw features (where `total_rooms` is in the thousands and `median_income` below 15) the RBF-SVC *essentially predicts 'not premium' for everything*: accuracy 0.752 is the majority-class baseline and $F_1$ is 0.11. After standardisation: accuracy 0.878 and $F_1$ 0.72. Distance-based models without scaling are a classic silent failure.

## What C and γ do to the boundary

**CODE · steps/step06_svm.py · block "boundaries"**

```
# Visualise the effect of the two RBF hyper-parameters on a 2-D slice (income, latitude).
f2 = [T.feature_names.index("num__median_income"), T.feature_names.index("num__latitude")]
X2 = np.clip(X[:, f2], -3, 5); sub = pool[:1500]
gx, gy = np.meshgrid(np.linspace(-3, 5, 220), np.linspace(-3, 5, 220))
fig, axes = plt.subplots(2, 3, figsize=(13, 7.6))
configs = [(0.1, 1), (1, 1), (50, 1), (1, 0.01), (1, 100), (50, 1000)]
for ax, (gamma, C) in zip(axes.ravel(), configs):
    m = SVC(kernel="rbf", gamma=gamma, C=C).fit(X2[sub], y[sub])
    Z = m.decision_function(np.c_[gx.ravel(), gy.ravel()]).reshape(gx.shape)
    ax.contourf(gx, gy, Z > 0, alpha=.15, colors=[PALETTE["grey"], PALETTE["orange"]], levels=[-.5, .5, 1.5])
    ax.contour(gx, gy, Z, levels=[0], colors=[PALETTE["blue"]], linewidths=2)
    ax.scatter(X2[sub][y[sub] == 0, 0], X2[sub][y[sub] == 0, 1], s=5, color=PALETTE["grey"])
    ax.scatter(X2[sub][y[sub] == 1, 0], X2[sub][y[sub] == 1, 1], s=5, color=PALETTE["orange"])
    ax.set_xlim(-3, 5); ax.set_ylim(-3, 5)
    acc = m.score(X2[hold], y[hold])
    ax.set_title(f"gamma={gamma}, C={C}  | n_SV={m.n_support_.sum()} | hold-out acc {acc:.3f}", fontsize=9)
    print(f"gamma={gamma:<5} C={C:<5}: support vectors = {m.n_support_.sum():4d} of {len(sub)} | train acc
        {m.score(X2[sub], y[sub]):.3f} | hold-out acc {acc:.3f}")
fig.suptitle("RBF kernel: gamma controls how local the boundary is, C controls how strictly errors are
    punished", fontweight="bold")
savefig(fig, "fig17_svm_boundaries")
```

**OUTPUT · produced by running the block above**

```
gamma=0.1   C=1    : support vectors =  605 of 1500 | train acc 0.822 | hold-out acc 0.835
gamma=1     C=1    : support vectors =  582 of 1500 | train acc 0.829 | hold-out acc 0.840
gamma=50    C=1    : support vectors =  886 of 1500 | train acc 0.859 | hold-out acc 0.813
gamma=1     C=0.01 : support vectors =  759 of 1500 | train acc 0.749 | hold-out acc 0.748
gamma=1     C=100  : support vectors =  619 of 1500 | train acc 0.831 | hold-out acc 0.839
gamma=50    C=1000 : support vectors =  542 of 1500 | train acc 0.945 | hold-out acc 0.745
```




***Figure 17.*** *RBF-SVM decision boundaries on a 2-D slice (income, latitude) for six (γ, C) settings. Top row: γ = 0.1, 1, 50 at C = 1: increasing γ turns a smooth boundary into small islands around individual points. Bottom row: C = 0.01 (over-regularised, almost no structure), C = 100, and γ = 50 with C = 1000 (pure memorisation).*

Study the numbers beside the picture. **γ = 50, C = 1000** reaches 94.5% *training* accuracy but only 74.5% on the hold-out: it memorised the sample (classic overfitting) and the boundary is a collection of islands. **C = 0.01** underfits (75% on both). The balanced choices around **γ ≈ 1, C ≈ 1–100** achieve about 84% on this 2-D slice. The tell-tale of overfitting in SVMs: *more support vectors surround each point* with large γ, and train accuracy exceeds hold-out accuracy.

**Intuition for γ**
Think of each training instance dropping a Gaussian bump of width 1/√(2γ). Small γ: huge bumps overlap into one smooth landscape. Large γ: tiny isolated bumps. If your model overfits, **reduce γ** (and C); if it underfits, increase them.

## Tuning on all 16 features

**CODE · steps/step06_svm.py · block “grid_search”**

```
sub = pool[:4000]
grid = GridSearchCV(SVC(kernel="rbf"),
                    {"C": [0.3, 1, 3, 10, 30], "gamma": [0.01, 0.03, 0.1, 0.3]}, cv=3, scoring="f1", n_jobs=2)
t0 = time.time(); grid.fit(X[sub], y[sub]); print("Grid search over 20 combos x 3 folds took %.0fs" %
    (time.time() - t0))
print("best params:", grid.best_params_, "| CV F1 = %.3f" % grid.best_score_)
best = grid.best_estimator_
pb = best.predict(X[hold])
print("Hold-out (4000 unseen training rows): accuracy %.3f | F1 %.3f" % (accuracy_score(y[hold], pb),
    f1_score(y[hold], pb)))
print("Support vectors: %d of %d (%.0f%%)" % (best.n_support_.sum(), len(sub), 100 * best.n_support_.sum() /
    len(sub)))
record("svc_best", {k: float(v) for k, v in grid.best_params_.items()}); record("svc_holdout_f1",
    f1_score(y[hold], pb))
table = pd.DataFrame(grid.cv_results_)[["param_C", "param_gamma", "mean_test_score"]].pivot(index="param_C",
    columns="param_gamma", values="mean_test_score").round(3)
print("\nCV F1 by (C rows x gamma cols):\n" + table.to_string())
```

**OUTPUT · produced by running the block above**

```
Grid search over 20 combos x 3 folds took 4s
best params: {'C': 10, 'gamma': 0.03} | CV F1 = 0.762
Hold-out (4000 unseen training rows): accuracy 0.885 | F1 0.753
Support vectors: 1173 of 4000 (29%)

CV F1 by (C rows x gamma cols):
param_gamma   0.01   0.03   0.10   0.30
param_C
0.3          0.568  0.634  0.683  0.657
1.0          0.669  0.711  0.724  0.729
3.0          0.717  0.735  0.746  0.740
10.0         0.737  0.762  0.759  0.742
30.0         0.757  0.757  0.747  0.731
```

A 5 × 4 grid with exponentially spaced values and 3-fold CV (60 fits) found **C = 10, γ = 0.03**, scoring $F_1$ ≈ 0.76 in CV and **0.75** on the 4,000-row hold-out (accuracy 0.885). Look at the pivot table: $F_1$ forms a ridge: when C is small, γ must be larger, and vice versa. C and γ interact, so tune them *together*. 29% of the training rows became support vectors.

## The scalability problem

**CODE · steps/step06_svm.py · block “time_scaling”**

```
rows = []
for n in [1000, 2000, 4000, 8000, 12000]:
    idx = pool[:n]
    t0 = time.time(); SVC(kernel="rbf", C=3, gamma=0.1).fit(X[idx], y[idx]); ts = time.time() - t0
    t0 = time.time(); LinearSVC(max_iter=20000).fit(X[idx], y[idx]); tl = time.time() - t0
    rows.append((n, round(ts, 2), round(tl, 3)))
tt = pd.DataFrame(rows, columns=["n_samples", "kernel_SVC_sec", "LinearSVC_sec"])
print(tt.to_string(index=False))
print("Kernel SVC: doubling n multiplies time by ~%.1fx (between O(m^2) and O(m^3)); LinearSVC by ~%.1fx" %
      (tt.kernel_SVC_sec.iloc[3] / tt.kernel_SVC_sec.iloc[2], tt.LinearSVC_sec.iloc[3] /
          max(tt.LinearSVC_sec.iloc[2], 1e-6)))
record("svc_time_table", tt.to_dict("list"))
```

**OUTPUT · produced by running the block above**

```
 n_samples  kernel_SVC_sec  LinearSVC_sec
      1000            0.01          0.003
      2000            0.04          0.004
      4000            0.12          0.006
      8000            0.44          0.014
     12000            1.30          0.036
Kernel SVC: doubling n multiplies time by ~3.7x (between O(m^2) and O(m^3)); LinearSVC by ~2.3x
```

Doubling the sample size multiplies kernel-SVC training time by roughly 3.5–5× (the exact ratio varies from run to run): between quadratic and cubic. At 12,000 rows it takes ~1 s; at 1 million rows it would take days. `LinearSVC` grows almost linearly and stays in milliseconds. This is the practical limit of kernel SVMs: great on small/medium datasets (up to tens of thousands of instances), impractical beyond.

## SVM regression

**CODE · steps/step06_svm.py · block “svr”**

```
y100 = T.y_value.to_numpy() / 1e5
sub = pool[:4000]
res = []
t0 = time.time(); l = LinearSVR(epsilon=0.1, C=1, max_iter=50000, random_state=RANDOM_STATE).fit(X[pool],
    y100[pool])
res.append(("LinearSVR (epsilon=0.1), 12.5k rows", 1e5 * rmse(y100[hold], l.predict(X[hold])), time.time() -
    t0))
for C, eps in [(1, 0.1), (3, 0.1), (10, 0.1), (3, 0.5)]:
    t0 = time.time(); s = SVR(kernel="rbf", C=C, epsilon=eps, gamma="scale").fit(X[sub], y100[sub])
    res.append((f"SVR rbf C={C} eps={eps}, 4k rows", 1e5 * rmse(y100[hold], s.predict(X[hold])), time.time() -
        t0))
    if (C, eps) == (3, 0.1): n_sv = len(s.support_)
r = pd.DataFrame(res, columns=["model", "hold-out RMSE ($)", "fit sec"]).round(1)
print(r.to_string(index=False))
print("\nSupport vectors in SVR(C=3, eps=0.1): %d of %d (a wider epsilon tube -> fewer SVs)" % (n_sv, len(sub)))
record("svr_best_rmse", float(r["hold-out RMSE ($)"].min()))
```

**OUTPUT · produced by running the block above**

```
                            model  hold-out RMSE ($)  fit sec
LinearSVR (epsilon=0.1), 12.5k rows            68794.1      3.6
       SVR rbf C=1 eps=0.1, 4k rows            58542.6      0.8
       SVR rbf C=3 eps=0.1, 4k rows            57322.2      0.9
      SVR rbf C=10 eps=0.1, 4k rows            56666.0      1.2
       SVR rbf C=3 eps=0.5, 4k rows            57949.2      0.3

Support vectors in SVR(C=3, eps=0.1): 3079 of 4000 (a wider epsilon tube -> fewer SVs)
```

On the internal hold-out, the **RBF-SVR** trained on only 4,000 rows reaches a RMSE of about **$56,666**, better than a `LinearSVR` trained on 12,512 rows ($68,794). Non-linearity matters more than the sample size. (Compare with the forest and gradient-boosting results of Chapter 7: they are better still.) A wider ε tube uses fewer support vectors and trains faster, at a small cost in accuracy.

## 5.3 Hands-on experiments

1. Grid-search `kernel='poly'` with degree 2–4 and compare with RBF. Which wins and how much slower is it?
2. Train `SVC(probability=True)` and compare the timing with `SVC()`. What does the extra time buy?
3. Use `SGDClassifier(loss='hinge', alpha=1/(m*C))` and compare with `LinearSVC`: do the weights match?
4. Plot the number of support vectors against γ. Explain the trend.
5. Run the premium problem with `class_weight='balanced'`. Which of precision and recall moves, and why?

## 5.4 Challenge questions

**Q1. What is a support vector, and what happens if you remove a non-support-vector instance?**

A support vector is a training instance on or inside the margin (or misclassified): it defines the boundary. Removing any other instance does not change the model.

**Q2. Explain the role of C in a soft-margin SVM.**

C weights the penalty for margin violations relative to margin width. Small C → wide margin, more violations, simpler model (regularisation). Large C → narrow margin, fewer violations, risk of overfitting. C is the inverse of regularisation strength.

**Q3. What is the kernel trick?**

Computing the inner product of instances in a high- (even infinite-) dimensional feature space via a kernel function K(a, b) without explicitly mapping to that space, so non-linear boundaries cost about the same as computing pairwise kernels.

**Q4. Your RBF SVM overfits. What do you change?**

Lower γ (smoother boundary) and/or lower C. Check feature scaling, maybe reduce features, and get more data.

**Q5. Why can't you use SVC on 5 million rows?**

Training time scales between $O(m^2)$ and $O(m^3)$ and kernel computations/memory grow quadratically. Use `LinearSVC`/`SGDClassifier`, approximate kernels (Nystroem, random Fourier features), or a different model family (boosting, neural nets).

**Q6. Do SVMs give probabilities?**

Not natively: only a signed distance. `SVC(probability=True)` fits a sigmoid (Platt scaling) using 5-fold CV (slower), and the probabilities can be inconsistent with `predict`.

**Q7. What is the loss function of a linear SVM?**

Hinge loss $\max(0, 1 - t \cdot s)$ ($t = \pm 1$, $s$ = score) plus an $\ell_2$ penalty on w, i.e. $\frac{1}{2}\|w\|^2 + C \sum \text{hinge}$.

**Q8. How does SVM regression differ from linear regression?**

It ignores errors inside an ε-wide tube and penalises only those outside (linearly), so it is robust to small noise and depends only on support vectors. Linear regression penalises squared error everywhere.

**Q9. Why does a linear SVM tend to behave like logistic regression?**

Both are linear models with an $\ell_2$ penalty and convex margin-like losses (hinge versus log). On well-behaved data they find almost identical boundaries, as seen in our experiment.

# Chapter 6 · Decision Trees

**Project step 7** — grow trees on HomeLens, compute a Gini split by hand, tune regularisation, and *see* their instability. Trees are the building block of forests and boosting (Chapter 7), the strongest models in this guide. Code: `steps/step07_trees.py`.

## 6.1 Concepts you must own

### What a tree is

A decision tree is a flowchart of yes/no questions on single features. Each **internal node** tests one feature against a threshold; each **leaf** gives a prediction. Trees need **little preprocessing** (no scaling, no centring; they handle mixed feature types) and are **white-box**: you can read the rules. That is why they are popular for explanations and why they are the base of the best tabular learners.

`predict_proba` for a classification tree returns the **fraction of each class among the training instances in the leaf**; the predicted class is the most frequent one. For regression the leaf returns the **mean target** of its instances.

### CART: how a tree is grown

Scikit-learn uses **CART** (Classification And Regression Trees), a greedy, recursive algorithm. At each node it searches *every feature* and *every threshold* for the split $(k, t_k)$ that minimises the cost $\mathbf{J(k, t_k) = (m_{left}/m)\, G_{left} + (m_{right}/m)\, G_{right}}$, where G measures impurity. It then recurses on both halves until it reaches `max_depth`, cannot find a split that reduces impurity, or violates a stopping rule.

| Impurity | Formula | Notes |
|---|---|---|
| **Gini** (default) | $G = 1 - \Sigma_k\, p_k^{\,2}$ | 0 when a node is pure; slightly faster; tends to isolate the most frequent class |
| **Entropy** | $H = -\Sigma_k\, p_k \log_2 p_k$ | 0 when pure; slightly more balanced trees; results usually nearly identical |
| **MSE** (regression) | $MSE_{node} = (1/m_{node})\, \Sigma\, (\hat{y}_{node} - y^{(i)})^2$ | Leaf predicts the mean; the same J formula with MSE instead of G |

> **CART is greedy**
> It picks the best split *now* and never reconsiders. It does not guarantee the globally optimal tree (finding it is NP-complete), but it is a reasonable solution. **Cost:** prediction $O(\log_2 m)$ (fast); training $O(n \times m \log m)$ because every feature is considered at every node.

### Regularising a tree

A tree with no constraints is **non-parametric**: it grows as complex as the data demands, fitting the training set perfectly (overfitting). Constrain it with hyper-parameters (increase `min_*` or reduce `max_*`):

| Hyper-parameter | Effect |
|---|---|
| `max_depth` | Maximum number of levels. The simplest, most common lever |
| `min_samples_split` / `min_samples_leaf` | Minimum instances to split a node / to exist in a leaf. Larger = smoother, more regularised |
| `max_leaf_nodes`, `max_features` | Cap the leaves; limit the features considered at each split (adds randomness, used by forests) |
| `min_impurity_decrease`, `ccp_alpha` | Only split if impurity drops enough; cost-complexity **pruning** after growing |

### Weaknesses: orthogonal boundaries and instability

- **Axis-aligned splits.** Every split is perpendicular to a feature axis, so a diagonal boundary needs a staircase of many splits. Rotating the data (e.g. with PCA) can change the tree's quality dramatically.
- **High variance.** Small changes in the training data (removing one instance, rotating the axes, a different random seed when ties occur) can produce a very different tree. This instability is exactly what *ensembles of trees* fix by averaging.
- **Extrapolation.** Trees predict constants within regions: they cannot extrapolate a trend beyond the training range, unlike linear models.
- **Overfitting** if unconstrained, and sensitivity to class imbalance (balance or weight the classes).

## 6.2 Project step 7 — hands on

**CODE · steps/step07_trees.py · block “setup”**

```
import sys; sys.path.insert(0, "src")
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.model_selection import cross_val_score, train_test_split, validation_curve
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, export_text, plot_tree
from homelens.tasks import load_task
from homelens.utils import set_style, savefig, record, rmse, PALETTE
from homelens.config import RANDOM_STATE
set_style()
T = load_task()
X, y, names = T.X, T.y_premium, T.feature_names
short = [n.split("__")[-1] for n in names]
Xtr, Xva, ytr, yva = train_test_split(X, y, test_size=0.25, random_state=RANDOM_STATE, stratify=y)
```

### Reading a small tree

**CODE · steps/step07_trees.py · block “shallow_tree”**

```
tree = DecisionTreeClassifier(max_depth=3, random_state=RANDOM_STATE).fit(Xtr, ytr)
print(export_text(tree, feature_names=short, decimals=2))
print("train acc %.3f | validation acc %.3f" % (tree.score(Xtr, ytr), tree.score(Xva, yva)))
```

**OUTPUT · produced by running the block above**

```
|--- median_income <= 0.91
|   |--- people_per_house <= -0.06
|   |   |--- median_income <= -0.26
|   |   |   |--- class: 0
|   |   |--- median_income >  -0.26
|   |   |   |--- class: 1
|   |--- people_per_house >  -0.06
|   |   |--- median_income <= 0.27
|   |   |   |--- class: 0
|   |   |--- median_income >  0.27
|   |   |   |--- class: 0
|--- median_income >  0.91
|   |--- median_income <= 1.29
|   |   |--- people_per_house <= -0.01
|   |   |   |--- class: 1
|   |   |--- people_per_house >  -0.01
|   |   |   |--- class: 0
|   |--- median_income >  1.29
|   |   |--- ocean_proximity_INLAND <= 0.50
|   |   |   |--- class: 1
|   |   |--- ocean_proximity_INLAND >  0.50
|   |   |   |--- class: 1

train acc 0.854 | validation acc 0.844
```

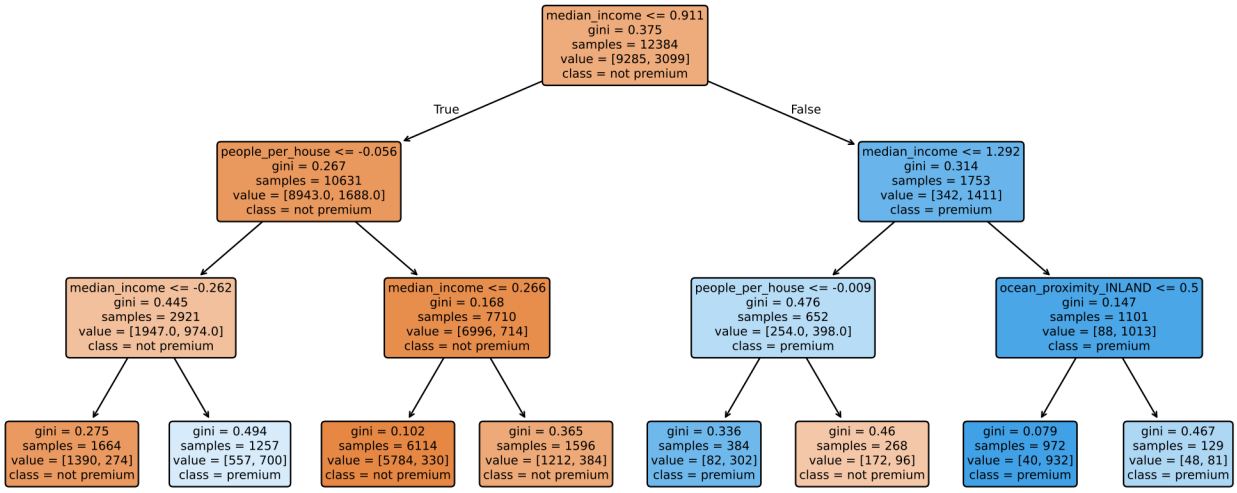



***Figure 18.*** *The depth-3 tree plotted by* `plot_tree`*: each node shows its rule, Gini impurity, sample count and class counts; colours encode the majority class.*

Thresholds are in *standardised* units because the data passed through a `StandardScaler` (trees would be equally happy with raw values). The first rule says: *if median income ≤ 0.91 standard deviations above the mean → go left*. Trace one district through the tree in the figure, and then through the text export: you should be able to read any path as an *if-then rule*. A depth-3 tree already achieves 84% validation accuracy using only a handful of rules.

## Verifying Gini and the CART cost by hand

**CODE · steps/step07_trees.py · block “gini_by_hand”**

```
# Verify the root node's Gini impurity and the CART split cost by hand.
t = tree.tree_
def node_stats(i):
    n = int(t.n_node_samples[i]); frac = t.value[i][0]; frac = frac / frac.sum()   # class proportions
    return n, frac, 1 - np.sum(frac ** 2)                                          # size, proportions, Gini
n_root, p, G_root = node_stats(0)
print("Root node: %d samples, class proportions %s" % (n_root, p.round(3)))
print("Gini(root) = 1 - (p0^2 + p1^2) = 1 - (%.3f^2 + %.3f^2) = %.4f" % (p[0], p[1], G_root))
l, r = t.children_left[0], t.children_right[0]
(nl, _, gl), (nr, _, gr) = node_stats(l), node_stats(r)
J = nl / n_root * gl + nr / n_root * gr
raw_inc = T.X_raw["median_income"]
thr_raw = raw_inc.mean() + t.threshold[0] * raw_inc.std(ddof=0)      # undo the StandardScaler
print("Split: %s <= %.3f (standardised)  =  income <= %.2f in the original units (x $10,000)" %
    (short[t.feature[0]], t.threshold[0], thr_raw))
print("  left : n=%5d  Gini=%.4f" % (nl, gl))
print("  right: n=%5d  Gini=%.4f" % (nr, gr))
print("CART cost J = (%d/%d)*%.4f + (%d/%d)*%.4f = %.4f  -> impurity drops from %.4f to %.4f" % (nl, n_root, gl,
    nr, n_root, gr, J, G_root, J))
record("gini_root", G_root); record("gini_after_split", J); record("root_feature", short[t.feature[0]]);
    record("root_thr_income", thr_raw)
print("\npredict_proba for one district:", tree.predict_proba(Xva[:1]).round(3), "(= class fractions in its
    leaf)")
```

**OUTPUT · produced by running the block above**

```
Root node: 12384 samples, class proportions [0.75 0.25]
Gini(root) = 1 - (p0^2 + p1^2) = 1 - (0.750^2 + 0.250^2) = 0.3752
Split: median_income <= 0.911 (standardised)  =  income <= 5.61 in the original units (x $10,000)
  left : n=10631  Gini=0.2671
  right: n= 1753  Gini=0.3141
CART cost J = (10631/12384)*0.2671 + (1753/12384)*0.3141 = 0.2738  -> impurity drops from 0.3752 to 0.2738

predict_proba for one district: [[0.946 0.054]] (= class fractions in its leaf)
```

The root has 12,384 training rows, 25% premium: Gini = $1 - (0.75^2 + 0.25^2)$ = **0.3752**. CART tried every feature and threshold and found that splitting on **median_income at ≈ 5.61** (units of $10,000, i.e. $56,100) gives a weighted child impurity of **0.2738**: the largest decrease available. The right child (1,753 rows, income above the threshold) is still impure (0.314) because not every high-income district is premium: location matters. This calculation is the *whole* algorithm repeated recursively. `predict_proba`

for a district in a leaf with 94.6% non-premium returns [0.946, 0.054]: leaf fractions, not magic.

## An unconstrained tree overfits

**CODE · steps/step07_trees.py · block "overfit_tree"**

```
full = DecisionTreeClassifier(random_state=RANDOM_STATE).fit(Xtr, ytr)
print("Unconstrained tree: depth=%d, leaves=%d, train acc=%.3f, validation acc=%.3f" %
      (full.get_depth(), full.get_n_leaves(), full.score(Xtr, ytr), full.score(Xva, yva)))
record("full_tree_depth", full.get_depth()); record("full_tree_leaves", full.get_n_leaves())
record("full_tree_val", full.score(Xva, yva))
```

**OUTPUT · produced by running the block above**

```
Unconstrained tree: depth=24, leaves=1059, train acc=1.000, validation acc=0.851
```

With no limits the tree reaches **depth 24 and 1,059 leaves**, 100% training accuracy but only 0.851 validation accuracy. Almost one leaf per 12 training rows: it has memorised the noise.

## Regularisation curves

**CODE · steps/step07_trees.py · block "regularise"**

```
depths = np.arange(1, 26)
tr_s, va_s = validation_curve(DecisionTreeClassifier(random_state=RANDOM_STATE), X, y, param_name="max_depth",
    param_range=depths, cv=5)
leaf_range = [1, 2, 5, 10, 20, 50, 100, 200, 400]
tr_l, va_l = validation_curve(DecisionTreeClassifier(random_state=RANDOM_STATE), X, y,
    param_name="min_samples_leaf", param_range=leaf_range, cv=5)
bd = depths[va_s.mean(1).argmax()]
print("Best max_depth = %d (CV accuracy %.3f)" % (bd, va_s.mean(1).max()))
bl = leaf_range[va_l.mean(1).argmax()]
print("Best min_samples_leaf = %d (CV accuracy %.3f)" % (bl, va_l.mean(1).max()))
print(pd.DataFrame({"min_samples_leaf": leaf_range, "train_acc": tr_l.mean(1).round(3), "cv_acc":
    va_l.mean(1).round(3)}).to_string(index=False))
record("tree_best_depth", int(bd)); record("tree_best_leaf", int(bl)); record("tree_best_acc",
    float(max(va_s.mean(1).max(), va_l.mean(1).max())))
```

**OUTPUT · produced by running the block above**

```
Best max_depth = 10 (CV accuracy 0.879)
Best min_samples_leaf = 20 (CV accuracy 0.885)
 min_samples_leaf  train_acc  cv_acc
                1      1.000   0.859
                2      0.979   0.867
                5      0.949   0.873
               10      0.928   0.880
               20      0.912   0.885
               50      0.895   0.879
              100      0.884   0.878
              200      0.876   0.869
              400      0.859   0.857
```

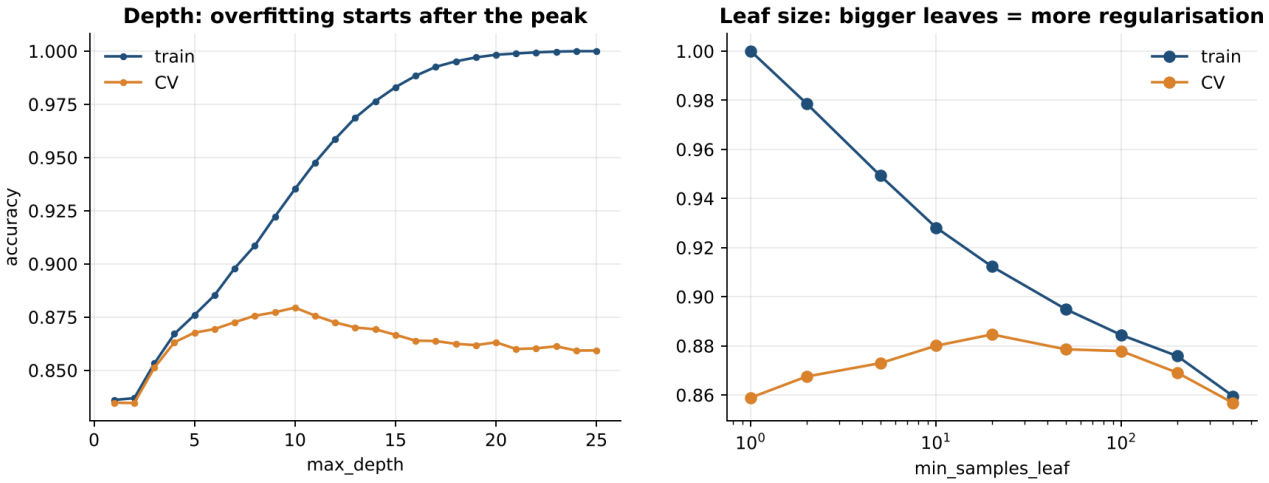



***Figure 19.*** *5-fold CV accuracy versus* `max_depth` *(left) and* `min_samples_leaf` *(right). Training accuracy keeps rising; CV accuracy peaks and then falls.*

The sweet spot is **max_depth ≈ 10** or **min_samples_leaf ≈ 20** (CV accuracy 0.885, versus 0.859 for the unconstrained tree). Again the U-shaped validation curve: *this is the same bias/variance picture, now for trees*. Notice that `min_samples_leaf` can express smoothness more directly than `max_depth`: it

prevents tiny leaves wherever they appear, not just at one depth.

## Instability, demonstrated

**CODE · steps/step07_trees.py · block “instability”**

```
# Trees are sensitive: (a) rotate the data, (b) remove ONE training example.
f2 = [names.index("num__median_income"), names.index("num__latitude")]
X2 = np.clip(X[:, f2], -3, 5)
gx, gy = np.meshgrid(np.linspace(-3, 5, 250), np.linspace(-3, 5, 250))
fig, ax = plt.subplots(1, 3, figsize=(13, 4.3))
def draw(a, model, data, title, grid):
    Z = model.predict(grid).reshape(gx.shape)
    a.contourf(gx, gy, Z, alpha=.2, colors=[PALETTE["grey"], PALETTE["orange"]], levels=[-.5, .5, 1.5])
    a.scatter(data[:, 0][::6], data[:, 1][::6], c=np.where(y[::6] == 1, PALETTE["orange"], PALETTE["grey"]),
        s=3)
    a.set_title(title, fontsize=10); a.set_xlim(-3, 5); a.set_ylim(-3, 5)
base = DecisionTreeClassifier(max_depth=6, min_samples_leaf=20, random_state=RANDOM_STATE).fit(X2, y)
draw(ax[0], base, X2, "Original axes", np.c_[gx.ravel(), gy.ravel()])
ang = np.deg2rad(35); R = np.array([[np.cos(ang), -np.sin(ang)], [np.sin(ang), np.cos(ang)]])
rot = DecisionTreeClassifier(max_depth=6, min_samples_leaf=20, random_state=RANDOM_STATE).fit(X2 @ R, y)
draw(ax[1], rot, X2, "Data rotated 35 deg -> different boundary", np.c_[gx.ravel(), gy.ravel()] @ R)
keep = np.ones(len(X2), bool); keep[np.argmax(X2[:, 0])] = False
one_out = DecisionTreeClassifier(max_depth=6, min_samples_leaf=20, random_state=RANDOM_STATE).fit(X2[keep],
    y[keep])
draw(ax[2], one_out, X2, "One training row removed", np.c_[gx.ravel(), gy.ravel()])
savefig(fig, "fig20_tree_instability")
acc_o, acc_r = base.score(X2, y), rot.score(X2 @ R, y)
print("Accuracy original axes: %.3f | rotated axes: %.3f" % (acc_o, acc_r))
# PCA rotates the data so that axes line up with directions of variance - often stabilises trees
pca_X = PCA().fit_transform(X)
a1 = cross_val_score(DecisionTreeClassifier(max_depth=8, min_samples_leaf=10, random_state=RANDOM_STATE), X, y,
    cv=5).mean()
a2 = cross_val_score(DecisionTreeClassifier(max_depth=8, min_samples_leaf=10, random_state=RANDOM_STATE), pca_X,
    y, cv=5).mean()
print("Full 16-feature tree CV acc: raw axes %.3f vs PCA-rotated axes %.3f" % (a1, a2))
```

**OUTPUT · produced by running the block above**

```
Accuracy original axes: 0.847 | rotated axes: 0.837
Full 16-feature tree CV acc: raw axes 0.878 vs PCA-rotated axes 0.861
```

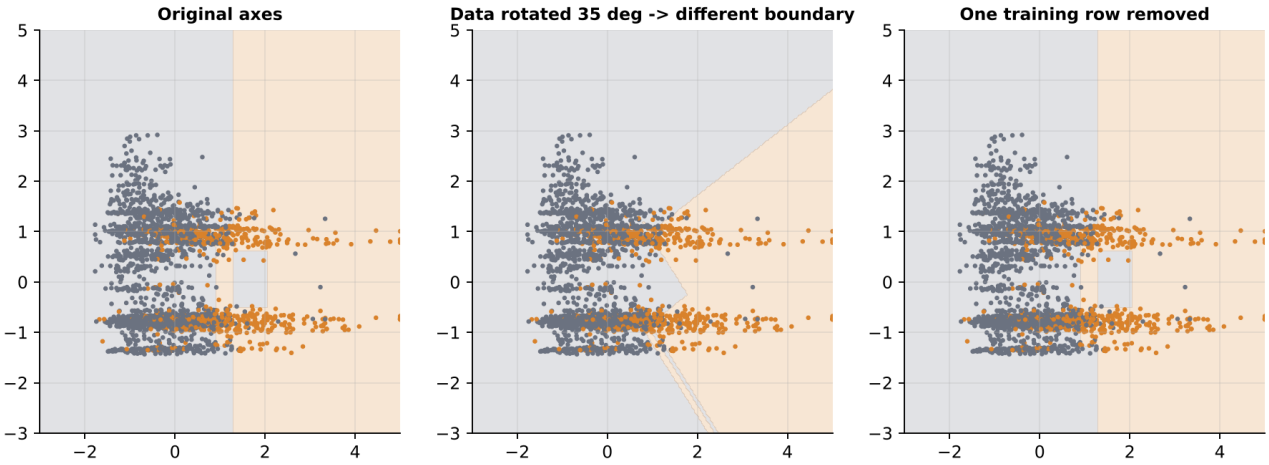



***Figure 20.*** *Left: a depth-6 tree on the original 2-D axes. Middle: the same data rotated 35°: a different staircase boundary and a lower score. Right: remove a single training row and the boundary changes.*

The same tree algorithm gives visibly different decision regions after (a) a rotation and (b) deleting a single training row. Rotating the whole 16-feature dataset with PCA lowered the CV accuracy (0.878 → 0.861) here: PCA is not a free lunch for trees. *This variance is the motivation for Chapter 7*: averaging hundreds of such unstable trees yields a stable, powerful model.

## Regression trees

**CODE · steps/step07_trees.py · block “regression_tree”**

```
inc = T.X_raw[["median_income"]].to_numpy(); yv = T.y_value.to_numpy()
g = np.linspace(inc.min(), 12, 500).reshape(-1, 1)
fig, ax = plt.subplots(1, 3, figsize=(13, 3.9), sharey=True)
for a, kw, ttl in zip(ax, [dict(max_depth=2), dict(max_depth=4), dict()], ["max_depth=2 (4 leaves)",
    "max_depth=4 (16 leaves)", "no limit (overfit)"]):
    m = DecisionTreeRegressor(random_state=RANDOM_STATE, **kw).fit(inc[:3000], yv[:3000])
    a.scatter(inc[:3000], yv[:3000], s=3, alpha=.25, color=PALETTE["grey"]); a.plot(g, m.predict(g),
        color=PALETTE["red"], lw=1.8)
    a.set_title(ttl + " | leaves=%d" % m.get_n_leaves(), fontsize=10); a.set_xlabel("median income")
ax[0].set_ylabel("predicted value ($)")
savefig(fig, "fig21_regression_trees")
m2 = DecisionTreeRegressor(max_depth=2, random_state=RANDOM_STATE).fit(inc, yv)
print(export_text(m2, feature_names=["median_income"], decimals=2))
cvr = -cross_val_score(DecisionTreeRegressor(max_depth=8, min_samples_leaf=10, random_state=RANDOM_STATE), X,
    yv, cv=5, scoring="neg_root_mean_squared_error")
print("Regression tree on all 16 features (depth 8): CV RMSE = %.0f" % cvr.mean())
record("tree_reg_rmse", cvr.mean())
```

**OUTPUT · produced by running the block above**

```
|--- median_income <= 5.09
|   |--- median_income <= 3.07
|   |   |--- value: [135889.56]
|   |--- median_income >  3.07
|   |   |--- value: [209790.49]
|--- median_income >  5.09
|   |--- median_income <= 6.82
|   |   |--- value: [292301.41]
|   |--- median_income >  6.82
|   |   |--- value: [421587.30]

Regression tree on all 16 features (depth 8): CV RMSE = 61342
```

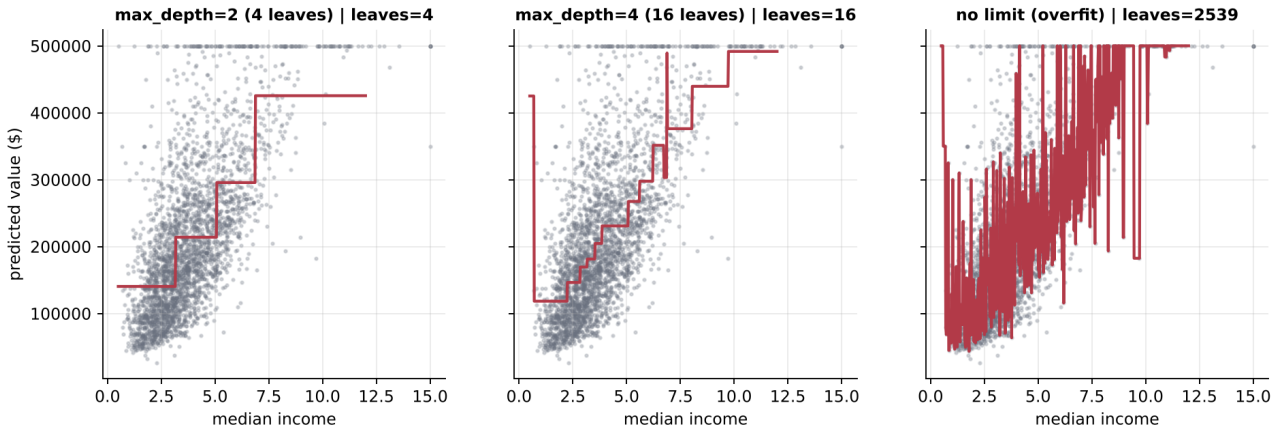



***Figure 21.*** *Regression trees on income only: a depth-2 tree is 4 horizontal steps; depth 4 follows the trend; an unconstrained tree chases individual points.*

The depth-2 tree is readable: income ≤ 3.07 → $135,890; 3.07–5.09 → $209,790; 5.09–6.82 → $292,300; above 6.82 → $421,590. Each leaf predicts the **mean price of its districts**, so the prediction is a staircase and **cannot extrapolate**. Using all 16 features at depth 8, the regression tree reaches a CV RMSE of $61,342: better than linear regression ($68k) but far behind the forest (~$49k), which averages many such trees.

**Common mistakes with trees**
• Trusting training accuracy of an unconstrained tree. • Interpreting a deep tree as 'explainable'; 1,000 leaves are not. • Using impurity-based feature importances as causal truth. • Expecting trees to extrapolate or to benefit from scaling. • Forgetting `random_state`: ties in split quality can make results vary run to run.

## 6.3 Hands-on experiments

1. Use `criterion='entropy'` and compare accuracy and tree shape with Gini. Are the root splits the same?
2. Use `ccp_alpha` (cost-complexity pruning): plot accuracy against alpha from `cost_complexity_pruning_path`.

3. Fit `DecisionTreeClassifier(class_weight='balanced')`. How do precision and recall of the premium class change?
4. Bootstrap 20 trees on resampled training sets, and compare their root splits. How often does the root change? (Preview of bagging.)
5. Plot a regression tree's prediction beyond the training range of income. What do you observe?

## 6.4 Challenge questions

**Q1. Compute the Gini impurity of a node with 30 positives and 10 negatives.**

p = 0.75, 0.25 → G = 1 − (0.5625 + 0.0625) = 0.375.

**Q2. How does CART choose a split?**

For each feature and each candidate threshold it computes the weighted average impurity of the two resulting children, J = (m_l/m)G_l + (m_r/m)G_r, and picks the (feature, threshold) pair with the smallest J. Then it recurses (greedy, top-down).

**Q3. Why are decision trees called 'non-parametric'? Why does that mean they overfit easily?**

The number of parameters is not fixed before training; it grows with the data. Without constraints the structure adapts until it fits the training set perfectly. Regularise via depth/leaf-size limits or pruning.

**Q4. Do trees need feature scaling? Why or why not?**

No. Splits compare a single feature with a threshold; monotone rescaling does not change which instances go left or right.

**Q5. List three ways to regularise a decision tree.**

Limit `max_depth`; raise `min_samples_leaf`/`min_samples_split`; limit `max_leaf_nodes`/`max_features`; prune with `ccp_alpha`.

**Q6. Why are trees unstable and what fixes that?**

Greedy splits depend on small differences in impurity, so small data perturbations change early splits and thus the whole subtree. Averaging many trees (bagging, random forests) reduces the variance.

**Q7. Gini or entropy?**

Usually little difference. Gini is a bit cheaper; entropy can give slightly more balanced trees. Choose by cross-validation if it matters.

**Q8. How does a regression tree make predictions and what shape does the prediction function have?**

Each leaf predicts the mean target of its training instances; the prediction function is piecewise-constant (a staircase), which cannot extrapolate beyond the training range.

**Q9. A tree has 100% training accuracy and 80% test accuracy. What would you do?**

It overfits. Constrain it (e.g. `min_samples_leaf`≥10, `max_depth`), choose the values by cross-validation, or move to a random forest/gradient boosting to cut variance.

**Q10. Why can rotating the data with PCA help or hurt a decision tree?**

Trees split along feature axes; if the discriminating direction is diagonal, rotating so it aligns with an axis simplifies the tree (help). If the original axes were meaningful, PCA mixes them and makes splits harder (hurt), as in our experiment.

# Chapter 7 · Ensemble Learning and Random Forests

**Project step 8** — combine many models into one better model, then run the **model bake-off** that decides what HomeLens ships. Code: `steps/step08_ensembles.py`.

## 7.1 Concepts you must own

An **ensemble** aggregates the predictions of several predictors (the *wisdom of the crowd*). The idea works because errors that are *independent* partly cancel: a thousand voters each correct 51% of the time give a correct majority about 74% of the time (and 92% for five thousand), if they are independent. The catch is **independence**: models trained on the same data make correlated errors. So ensemble methods work to *diversify* their members: different algorithms, different data samples, different features, or sequentially focused training.

| Method | How members differ | Reduces | Key facts |
|---|---|---|---|
| **Voting** | Different algorithms on the same data | Variance, some bias | *Hard*: majority class. *Soft*: average class probabilities (needs `predict_proba`; usually better since confident votes weigh more) |
| **Bagging** (bootstrap aggregating) | Same algorithm, each on a bootstrap sample (with replacement) | **Variance** | Parallelisable. Each tree sees ~63% of rows; the other ~37% (**out-of-bag, OOB**) give a free validation score (`oob_score=True`). *Pasting* = without replacement |
| **Random forest** | Bagged trees + only a random subset of features at each split (`max_features`) | Variance, further decorrelates trees | Strong default on tabular data. Feature importance by average impurity decrease. **Extra-Trees**: random thresholds too: faster, higher bias, lower variance |
| **Boosting** | Models trained *sequentially*; each fixes the previous one's errors | **Bias** (and some variance) | Cannot be parallelised across rounds. Tune `learning_rate` (shrinkage) with `n_estimators`; use early stopping |
| **Stacking** | Heterogeneous models + a **meta-learner** trained on their out-of-fold predictions | Both | A blender learns how much to trust each model. Needs out-of-fold predictions to avoid leakage |

### Boosting in detail

**AdaBoost** trains a weak learner, increases the weights of the instances it got wrong, trains the next learner on the re-weighted data, and so on; the final prediction is a weighted vote (better learners weigh more). **Gradient Boosting** instead fits each new tree to the **residual errors** of the current ensemble: prediction = $\text{tree}_1 + \eta \cdot \text{tree}_2 + \eta \cdot \text{tree}_3 + \ldots$, where $\eta$ is the **learning rate**. For squared error the residuals are exactly the negative gradient of the loss, which is why it is *gradient* boosting. A **small $\eta$ needs more trees but generalises better** (shrinkage as regularisation); `n_iter_no_change` stops adding trees when validation error stalls. **Histogram-based gradient boosting** (`HistGradientBoosting*`, the idea behind XGBoost, LightGBM and CatBoost) bins features into 255 buckets: dramatically faster, supports missing values natively. It is the usual first choice for tabular data.

> **Why bagging cuts variance but boosting cuts bias**
> Averaging many *high-variance, low-bias* learners (deep trees) cancels their noise: variance falls roughly as 1/N for uncorrelated members. Boosting chains *low-variance, high-bias* learners (shallow trees) so each one corrects what the previous missed: bias falls, and with it overall error; but because it fits residuals it can **overfit if run too long** (bagging cannot overfit by adding trees).

# 7.2 Project step 8 — hands on

**CODE · steps/step08_ensembles.py · block “setup”**

```
import sys; sys.path.insert(0, "src")
import numpy as np, pandas as pd, matplotlib.pyplot as plt, time
from scipy.stats import binom
from sklearn.ensemble import (AdaBoostClassifier, BaggingClassifier, ExtraTreesRegressor,
    GradientBoostingRegressor,
                              HistGradientBoostingRegressor, RandomForestClassifier, RandomForestRegressor,
                              StackingRegressor, VotingClassifier)
from sklearn.linear_model import LogisticRegression, Ridge, LinearRegression
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.metrics import accuracy_score, f1_score
from homelens.tasks import load_task
from homelens.utils import set_style, savefig, record, rmse, PALETTE
from homelens.config import RANDOM_STATE
set_style()
T = load_task()
X, y, yv = T.X, T.y_premium, T.y_value.to_numpy()
names = [n.split("__")[-1] for n in T.feature_names]
Xtr, Xva, ytr, yva, vtr, vva = train_test_split(X, y, yv, test_size=0.25, random_state=RANDOM_STATE, stratify=y)
```

## Why the crowd is wise (and when it is not)

**CODE · steps/step08_ensembles.py · block “law_of_large_numbers”**

```
# Why ensembles work: many weak, INDEPENDENT voters beat one. A coin that is right 51% of the time...
fig, ax = plt.subplots(figsize=(6.5, 3.8))
n = np.arange(1, 5001, 20)
for p, c in [(0.51, PALETTE["orange"]), (0.55, PALETTE["blue"]), (0.65, PALETTE["green"])]:
    ax.plot(n, 1 - binom.cdf(n // 2, n, p), label=f"each voter {int(p*100)}% accurate", color=c, lw=2)
ax.set_xlabel("number of independent voters"); ax.set_ylabel("P(majority is right)"); ax.legend();
    ax.set_title("Wisdom of the crowd (needs independence!)")
savefig(fig, "fig22_wisdom_crowd")
for p in [0.51, 0.55, 0.65]:
    print("voters %d%% accurate: majority of 1001 is right with P = %.3f ; majority of 5001: %.3f" %
          (p * 100, 1 - binom.cdf(500, 1001, p), 1 - binom.cdf(2500, 5001, p)))
```

**OUTPUT · produced by running the block above**

```
voters 51% accurate: majority of 1001 is right with P = 0.737 ; majority of 5001: 0.921
voters 55% accurate: majority of 1001 is right with P = 0.999 ; majority of 5001: 1.000
voters 65% accurate: majority of 1001 is right with P = 1.000 ; majority of 5001: 1.000
```

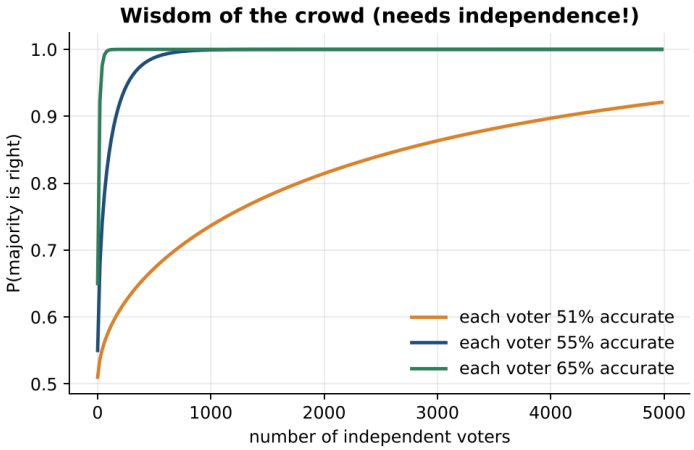



***Figure 22.*** *Probability that a majority of independent voters is right, as the number of voters grows. Weakly accurate voters (51%) need thousands; moderately accurate ones (65%) need only a few dozen.*

## Voting

**CODE · steps/step08_ensembles.py · block “voting”**

```
clfs = [("lr", LogisticRegression(max_iter=3000)), ("rf", RandomForestClassifier(100, random_state=RANDOM_STATE,
    n_jobs=2)),
        ("knn", KNeighborsClassifier(15))]
for n_, c in clfs:
    c.fit(Xtr, ytr); print("%-5s validation accuracy %.4f | F1 %.4f" % (n_, c.score(Xva, yva), f1_score(yva,
        c.predict(Xva))))
for vt in ["hard", "soft"]:
    v = VotingClassifier(clfs, voting=vt, n_jobs=1).fit(Xtr, ytr)
    print("VOTING (%s): validation accuracy %.4f | F1 %.4f" % (vt, v.score(Xva, yva), f1_score(yva,
        v.predict(Xva))))
    record(f"voting_{vt}_acc", v.score(Xva, yva))
```

**OUTPUT · produced by running the block above**

```
lr    validation accuracy 0.8755 | F1 0.7331
rf    validation accuracy 0.9055 | F1 0.7994
knn   validation accuracy 0.8738 | F1 0.7155
VOTING (hard): validation accuracy 0.8932 | F1 0.7648
VOTING (soft): validation accuracy 0.8956 | F1 0.7728
```

Three different models (logistic regression, random forest, k-NN) each score 87–91%. The voting ensembles reach 89.3% (hard) and 89.6% (soft). Notice a subtle, important point: **an ensemble is not guaranteed to beat its best member**. Here the random forest alone (90.6%) beats the vote, because the other two voters are weaker and make correlated errors. Ensembling helps most when members are *about equally good and diverse*.

## Bagging and the free OOB estimate

**CODE · steps/step08_ensembles.py · block “bagging_oob”**

```
bag = BaggingClassifier(DecisionTreeClassifier(min_samples_leaf=5), n_estimators=200, max_samples=0.7,
    bootstrap=True,
                        oob_score=True, n_jobs=2, random_state=RANDOM_STATE).fit(Xtr, ytr)
single = DecisionTreeClassifier(min_samples_leaf=5, random_state=RANDOM_STATE).fit(Xtr, ytr)
print("Single tree  validation accuracy: %.4f" % single.score(Xva, yva))
print("Bagging (200 trees) OOB estimate: %.4f  | actual validation accuracy: %.4f" % (bag.oob_score_,
    bag.score(Xva, yva)))
print("=> the out-of-bag score predicts validation accuracy WITHOUT needing a validation set")
print("A bootstrap sample leaves out ~ %.1f%% of rows (theory: 1/e = 36.8%%)" % (100 *
    np.mean([len(set(range(10000)) - set(np.random.RandomState(i).randint(0, 10000, 10000))) / 10000 for i in
    range(5)])))
record("bag_oob", bag.oob_score_); record("bag_val", bag.score(Xva, yva)); record("single_tree_val",
    single.score(Xva, yva))
```

**OUTPUT · produced by running the block above**

```
Single tree  validation accuracy: 0.8622
Bagging (200 trees) OOB estimate: 0.9033  | actual validation accuracy: 0.9014
=> the out-of-bag score predicts validation accuracy WITHOUT needing a validation set
A bootstrap sample leaves out ~ 36.8% of rows (theory: 1/e = 36.8%)
```

A single tree scores 86.2% on validation. Bagging 200 trees lifts it to **90.1%**: a four-point gain from averaging alone. The **OOB score (90.3%) predicted the validation accuracy almost exactly**, so with bagging you can skip a separate validation set. The check on the bootstrap shows that each sample leaves out ≈ 36.8% of rows (1/e): that is where OOB instances come from.

## Random forests and feature importance

**CODE · steps/step08_ensembles.py · block “random_forest”**

```
rf = RandomForestClassifier(300, min_samples_leaf=3, oob_score=True, n_jobs=2,
    random_state=RANDOM_STATE).fit(Xtr, ytr)
print("Random forest: OOB %.4f | validation accuracy %.4f | F1 %.4f" % (rf.oob_score_, rf.score(Xva, yva),
    f1_score(yva, rf.predict(Xva))))
imp = pd.Series(rf.feature_importances_, index=names).sort_values(ascending=False)
print(imp.head(8).round(3).to_string())
record("rf_clf_val", rf.score(Xva, yva))
fig, ax = plt.subplots(figsize=(6.8, 4.4)); imp.head(10)[::-1].plot.barh(ax=ax, color=PALETTE["blue"]);
    ax.set_title("Random-forest feature importance (classification)")
savefig(fig, "fig23_rf_importance")
# number of trees
accs = []
for k in [1, 3, 10, 30, 100, 300]:
    accs.append((k, RandomForestClassifier(k, min_samples_leaf=3, n_jobs=2, random_state=RANDOM_STATE).fit(Xtr,
        ytr).score(Xva, yva)))
print("Trees -> validation accuracy:", " | ".join(f"{k}:{a:.3f}" for k, a in accs))
```

**OUTPUT · produced by running the block above**

```
Random forest: OOB 0.9073 | validation accuracy 0.9077 | F1 0.8041
median_income             0.279
people_per_house          0.122
bedrooms_ratio            0.102
longitude                 0.097
latitude                  0.082
rooms_per_house           0.074
ocean_proximity_INLAND    0.068
housing_median_age        0.045
Trees -> validation accuracy: 1:0.857 | 3:0.880 | 10:0.901 | 30:0.906 | 100:0.910 | 300:0.908
```

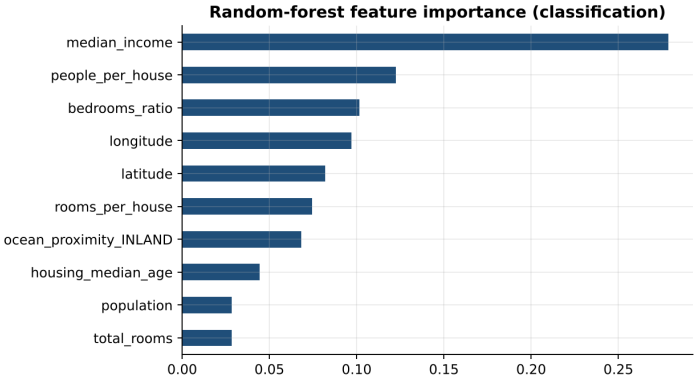



***Figure 23.*** *Random-forest feature importance for the premium classifier.*

The random forest reaches **90.8%** validation accuracy (OOB 90.7%). Accuracy saturates around 100 trees (see the last printed line): *adding trees never overfits*, it just stops helping. Income leads the importances, but now location (longitude, latitude), `people_per_house` and `bedrooms_ratio` all matter: the forest uses interactions that the linear model could not.

## Boosting

**CODE · steps/step08_ensembles.py · block “boosting”**

```
ada = AdaBoostClassifier(DecisionTreeClassifier(max_depth=2), n_estimators=300, learning_rate=0.5,
    random_state=RANDOM_STATE).fit(Xtr, ytr)
print("AdaBoost (300 stumps depth 2): validation accuracy %.4f | F1 %.4f" % (ada.score(Xva, yva), f1_score(yva,
    ada.predict(Xva))))
# Gradient boosting regression with early stopping, compared across learning rates
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
for lr_, c in [(0.02, PALETTE["orange"]), (0.1, PALETTE["blue"]), (0.5, PALETTE["red"])]:
    gb = GradientBoostingRegressor(n_estimators=400, learning_rate=lr_, max_depth=3, subsample=0.8,
        random_state=RANDOM_STATE).fit(Xtr, vtr)
    errs = [rmse(vva, p) for p in gb.staged_predict(Xva)]
    ax[0].plot(errs, label=f"learning_rate={lr_}", color=c, lw=2)
    print("learning_rate=%.2f: best validation RMSE %.0f at %d trees; at 400 trees: %.0f" % (lr_, min(errs),
        int(np.argmin(errs)) + 1, errs[-1]))
ax[0].set_ylim(40000, 90000); ax[0].set_xlabel("number of trees"); ax[0].set_ylabel("validation RMSE ($)");
    ax[0].legend(); ax[0].set_title("Gradient boosting: shrinkage vs. number of trees")
# residual-fitting idea in 3 lines
r1 = vtr - np.full_like(vtr, vtr.mean(), dtype=float)
t1 = DecisionTreeRegressor(max_depth=3).fit(Xtr, r1); pred1 = vtr.mean() + t1.predict(Xtr)
t2 = DecisionTreeRegressor(max_depth=3).fit(Xtr, vtr - pred1); pred2 = pred1 + t2.predict(Xtr)
print("\nBy hand: train RMSE  mean-only %.0f -> +tree1 %.0f -> +tree2 %.0f  (each tree fits the previous
    residuals)" %
      (rmse(vtr, np.full_like(vtr, vtr.mean(), dtype=float)), rmse(vtr, pred1), rmse(vtr, pred2)))
gb_es = GradientBoostingRegressor(n_estimators=2000, learning_rate=0.1, max_depth=3, subsample=0.8,
    n_iter_no_change=15,
                                  validation_fraction=0.15, random_state=RANDOM_STATE).fit(Xtr, vtr)
print("Early stopping picked n_estimators = %d (out of a 2000 budget)" % gb_es.n_estimators_)
record("gb_early_trees", gb_es.n_estimators_)
ax[1].bar(["mean", "+tree 1", "+tree 2"], [rmse(vtr, np.full_like(vtr, vtr.mean(), dtype=float)), rmse(vtr,
    pred1), rmse(vtr, pred2)], color=PALETTE["teal"])
ax[1].set_ylabel("train RMSE ($)"); ax[1].set_title("Boosting by hand: fit the residuals")
savefig(fig, "fig24_boosting")
```

**OUTPUT · produced by running the block above**

```
AdaBoost (300 stumps depth 2): validation accuracy 0.8922 | F1 0.7774
learning_rate=0.02: best validation RMSE 53281 at 400 trees; at 400 trees: 53281
learning_rate=0.10: best validation RMSE 47451 at 400 trees; at 400 trees: 47451
learning_rate=0.50: best validation RMSE 49140 at 164 trees; at 400 trees: 49684

By hand: train RMSE  mean-only 115594 -> +tree1 74985 -> +tree2 68459  (each tree fits the previous residuals)
Early stopping picked n_estimators = 327 (out of a 2000 budget)
```

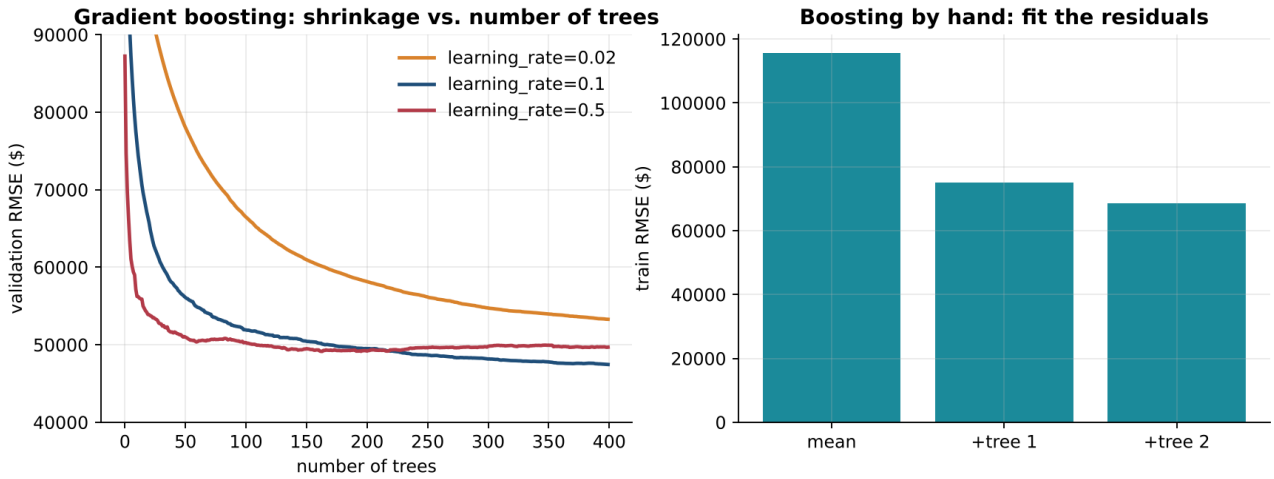



***Figure 24.*** *Left: validation RMSE versus number of trees for three learning rates. A tiny η (0.02) is still improving at 400 trees; η = 0.5 learns fast but bottoms out at a worse error and then drifts up. Right: boosting by hand, each tree fits the previous residuals.*

Study the trade-off on the left panel. **η = 0.1 with 400 trees achieves the best error (RMSE $47.5k)**; η = 0.5 reaches its best (49.1k) after only 164 trees and then slowly overfits; η = 0.02 would need many more than 400 trees. The 'by hand' experiment builds gradient boosting from scratch in four lines: predict the mean (RMSE $115.6k), fit a tree to the residuals (→ $75.0k), fit another to the new residuals (→ $68.5k). **Early stopping** (`n_iter_no_change=15`) picked 327 trees automatically from a 2,000-tree budget: always prefer it to guessing.

## The model bake-off

**CODE · steps/step08_ensembles.py · block "model_bakeoff"**

```
# 5-fold CV RMSE for every regression model on the SAME features. This is the model-selection table.
def cv(model, name, Xm=X, folds=5):
    t0 = time.time()
    s = -cross_val_score(model, Xm, yv, cv=folds, scoring="neg_root_mean_squared_error", n_jobs=2)
    return (name, s.mean(), s.std(), time.time() - t0)
rows = [
    cv(LinearRegression(), "Linear regression"),
    cv(Ridge(alpha=10), "Ridge"),
    cv(DecisionTreeRegressor(max_depth=10, min_samples_leaf=10, random_state=RANDOM_STATE), "Decision tree
        (tuned)"),
    cv(RandomForestRegressor(150, min_samples_leaf=2, max_features=0.5, n_jobs=1, random_state=RANDOM_STATE),
        "Random forest"),
    cv(ExtraTreesRegressor(150, min_samples_leaf=2, max_features=0.7, n_jobs=1, random_state=RANDOM_STATE),
        "Extra-trees"),
    cv(GradientBoostingRegressor(n_estimators=300, learning_rate=0.1, max_depth=4, subsample=0.8,
        random_state=RANDOM_STATE), "Gradient boosting"),
    cv(HistGradientBoostingRegressor(max_iter=500, learning_rate=0.08, max_leaf_nodes=31, early_stopping=True,
        random_state=RANDOM_STATE), "Hist. gradient boosting"),
]
stack = StackingRegressor(
    [("rf", RandomForestRegressor(100, min_samples_leaf=2, max_features=0.5, n_jobs=1,
        random_state=RANDOM_STATE)),
     ("hgb", HistGradientBoostingRegressor(max_iter=400, learning_rate=0.08, random_state=RANDOM_STATE)),
     ("et", ExtraTreesRegressor(100, min_samples_leaf=2, max_features=0.7, n_jobs=1,
         random_state=RANDOM_STATE))],
    final_estimator=Ridge(alpha=1.0), cv=4, n_jobs=2)
rows.append(cv(stack, "Stacking (RF+HGB+ET -> Ridge)", folds=3))
tab = pd.DataFrame(rows, columns=["model", "cv_rmse", "cv_std", "secs"]).round(0)
print(tab.to_string(index=False))
record("bakeoff", tab.to_dict("list"))
fig, ax = plt.subplots(figsize=(8, 4.2)); order = tab.sort_values("cv_rmse", ascending=False)
ax.barh(order.model, order.cv_rmse, xerr=order.cv_std, color=PALETTE["blue"], alpha=.9)
for i, v in enumerate(order.cv_rmse): ax.text(v + 800, i, f"${v:,.0f}", va="center", fontsize=8)
ax.set_xlabel("cross-validated RMSE ($, lower is better)"); ax.set_title("Model bake-off on the same 16

    features"); ax.set_xlim(0, 95000)
savefig(fig, "fig25_bakeoff")
```

**OUTPUT · produced by running the block above**

```
                          model  cv_rmse  cv_std  secs
              Linear regression  68434.0   808.0   0.0
                          Ridge  68428.0   811.0   0.0
         Decision tree (tuned)  59966.0   560.0   0.0
                 Random forest  49225.0   873.0  18.0
                    Extra-trees  49754.0   768.0   9.0
              Gradient boosting  47324.0  1197.0  32.0
       Hist. gradient boosting  46105.0  1027.0   2.0
Stacking (RF+HGB+ET -> Ridge)  46280.0   897.0  37.0
```

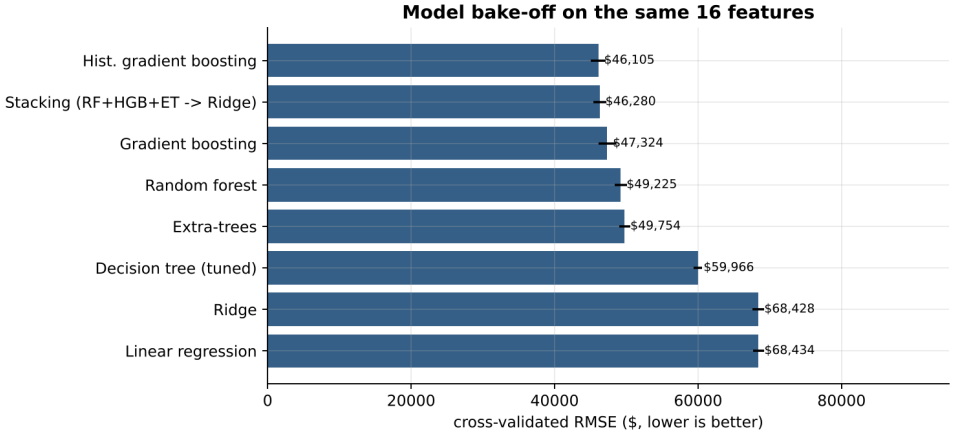



***Figure 25.*** *Cross-validated RMSE for eight regression models on identical features. Lower is better; error bars show the fold-to-fold standard deviation.*

This table is the **model-selection evidence**. Ranking: histogram gradient boosting ($46.1k) ≈ stacking ($46.3k) < gradient boosting ($47.3k) < random forest ($49.2k) ≈ extra-trees ($49.8k) ≪ a tuned single tree ($60.0k) ≪ linear/ridge ($68.4k). Three lessons. (1) **Ensembles of trees beat every single model by a wide margin** (46k vs 60k vs 68k). (2) **The differences among the top four are comparable to the fold standard deviation (~$0.9–1.2k)**: do not over-interpret rank differences within noise. (3) Stacking is 20× slower than histogram boosting for no measurable gain, so we choose the simplest high performer: **HistGradientBoostingRegressor** for the final HomeLens regressor (tuned in Step 11).

**A practical rule for tabular data**
Start with a tuned gradient-boosted tree model and a random forest; compare with a regularised linear model as the sanity baseline; stack/blend only if you need the last 1–2%. Spend your effort on **features and validation** before algorithms.

## 7.3 Hands-on experiments

1. Set `max_features=1.0` in the random forest (plain bagging of trees). Compare CV RMSE with `max_features=0.5`: why does restricting features help?
2. In gradient boosting try `max_depth` 1, 2, 3, 6 at a fixed η. Which depth is best and why does a depth-1 ensemble underfit?
3. Build your own bagging ensemble of 100 `DecisionTreeRegressor`s with `np.random.choice` bootstrap samples and average their predictions: do you match `BaggingRegressor`?
4. Use `StackingRegressor(passthrough=True)`. Does feeding the raw features to the meta-learner help?
5. Create a deliberately correlated ensemble (10 random forests with the same seed). Show that the ensemble gains nothing: independence matters.

## 7.4 Challenge questions

**Q1. Why does averaging many models reduce error? When does it fail?**

If members' errors are not perfectly correlated, averaging cancels part of the noise (variance falls toward $\rho\sigma^2 + (1-\rho)\sigma^2/N$). It fails when errors are strongly correlated (same data, same algorithm, same bias) or when members share a systematic bias: averaging cannot remove bias.

**Q2. What is the out-of-bag score?**

With bootstrap sampling, ~37% of rows are not drawn for a given tree. Evaluating each tree on its OOB rows (and aggregating across trees for each row) gives an unbiased validation estimate without a separate set.

**Q3. What extra randomness does a random forest add to bagging and why?**

Each split considers only a random subset of the features. This de-correlates the trees (they cannot all use the same dominant feature), so averaging reduces variance more.

**Q4. Contrast bagging and boosting.**

Bagging trains models in parallel on bootstrap samples and averages them (variance reduction). Boosting trains sequentially, each model concentrating on the errors of its predecessors (bias reduction); it can overfit and cannot be parallelised across rounds.

**Q5. What does the learning rate do in gradient boosting?**

It scales each tree's contribution. Lower $\eta$ = smaller steps = more trees needed but better generalisation (shrinkage). Choose $\eta$ and n_estimators jointly, using early stopping.

**Q6. Explain gradient boosting for regression in three sentences.**

Start with a constant prediction (the mean). Fit a small tree to the residuals (negative gradient of the squared error), add $\eta$ times its predictions to the ensemble. Repeat; each tree corrects what the ensemble still gets wrong.

**Q7. Hard versus soft voting?**

Hard: each classifier votes for a class, the majority wins. Soft: average predicted probabilities and take the argmax: usually better because it uses confidence, but needs well-calibrated probabilities.

**Q8. What is stacking and what is the leakage risk?**

Train a meta-model on base models' predictions. The meta-model must be trained on **out-of-fold** predictions, otherwise it sees predictions on data the base models were trained on and learns to over-trust them.

**Q9. Random forest versus Extra-Trees?**

Extra-Trees choose split thresholds at random instead of searching the best one: faster, a bit more bias and less variance; sometimes better, sometimes worse; cheap to try.

**Q10. Why choose HistGradientBoosting as the default for this dataset?**

Best CV error in the bake-off, very fast (≈2 s), handles missing values natively, scales to large datasets, and has built-in early stopping. Stacking's gain was within noise.

**Q11. Can a random forest overfit by adding more trees?**

No; more trees only reduce the variance of the average (error plateaus). Overfitting is controlled by tree depth/leaf size, not the number of trees (unlike boosting).

# Chapter 8 · Dimensionality Reduction

**Project step 9** — compress and visualise the HomeLens features: you will compute PCA by hand with the SVD, test whether PCA helps models, try kernel PCA, and visualise price tiers with t-SNE. Code: `steps/step09_dimensionality.py`.

## 8.1 Concepts you must own

### The curse of dimensionality

High-dimensional spaces behave unlike our 2-D/3-D intuition. A random point in a unit hypercube is very likely to be extremely close to a border (in 1,000 dimensions: 86.5% chance of being within 0.001 of a face; see our experiment). Two random points are far apart, and *nearest and farthest neighbours become almost the same distance*. The practical consequences: the training set becomes sparse, predictions rely on extrapolation, distance-based methods degrade, and overfitting risk soars. The number of instances needed for a given density grows **exponentially** with the number of dimensions. Reducing dimensions can speed training, reduce overfitting and enable visualisation, at the cost of losing some information and making the pipeline more complex.

### Two approaches

- **Projection**: most data lies close to a lower-dimensional *subspace* of the high-dimensional space (many features are correlated or nearly constant); project onto it (PCA, random projection).
- **Manifold learning**: the data lies on a curved low-dimensional surface (a *manifold*) twisted in high-dimensional space (a rolled-up Swiss roll); *unroll* it (LLE, Isomap, t-SNE, UMAP). The **manifold assumption** says the task is simpler in the manifold coordinates (not always true).

### PCA: the geometry and the algebra

**PCA** (principal component analysis) finds the axis along which the data has **maximum variance** (preserving the most information, equivalently minimising the mean squared distance between the data and its projection), then the orthogonal axis with the next largest variance, and so on. The unit vectors defining the axes are the **principal components**. Algebraically: centre the data X (subtract the means), compute the **SVD** $X = U\,\Sigma\,V^T$; the columns of V are the principal components; the projection onto the first d components is $\mathbf{X_d = X\,V_d}$, and the explained variance of component j is $\sigma_j^2/\Sigma\sigma^2$.

- **Always centre** (scikit-learn's PCA does it) and usually **standardise** first when features have different units.
- **Explained variance ratio** tells how much information each component keeps. Choose d to retain e.g. 95% (`PCA(n_components=0.95)`) or tune d by the downstream task's score (a hyper-parameter in a Pipeline).
- **Reconstruction**: `inverse_transform` maps back to the original space, with a *reconstruction error* from discarded components. PCA for compression keeps most variance in far fewer dimensions.
- **Variants**: *Randomised PCA* (`svd_solver='randomized'`) is much faster for large m, n when d is small. *Incremental PCA* processes mini-batches (data that does not fit in memory; online). *Kernel PCA* applies the kernel trick for non-linear projections. *Random projection* is a JL-lemma-based cheap method.

### Kernel PCA, LLE, t-SNE

| Method | Idea | Strengths | Limits |
|---|---|---|---|
| **Kernel PCA** | PCA in the implicit feature space of a kernel (RBF, sigmoid…) | Non-linear; unsupervised but can be tuned with a supervised pipeline (grid search on final accuracy) | $O(m^2)$ memory; kernel and γ are hyper-parameters |

| Method | Idea | Strengths | Limits |
|---|---|---|---|
| **LLE** | Express each point as a linear combination of its neighbours, then find a low-d embedding preserving those weights | Unrolls twisted manifolds | Scales poorly with m; sensitive to noise |
| **t-SNE** | Keep similar points close and dissimilar points apart (probabilistic neighbourhood matching) | Beautiful cluster visualisations in 2-D | **Visualisation only**; distances and cluster sizes are not meaningful; no `transform` for new data; perplexity matters |

# 8.2 Project step 9 — hands on

**CODE · steps/step09_dimensionality.py · block "setup"**

```
import sys; sys.path.insert(0, "src")
import numpy as np, pandas as pd, matplotlib.pyplot as plt, time
from sklearn.decomposition import PCA, IncrementalPCA, KernelPCA
from sklearn.ensemble import HistGradientBoostingRegressor, RandomForestRegressor
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.manifold import TSNE
from sklearn.model_selection import GridSearchCV, cross_val_score
from sklearn.pipeline import make_pipeline
from homelens.tasks import load_task
from homelens.utils import set_style, savefig, record, rmse, PALETTE
from homelens.config import RANDOM_STATE
set_style()
T = load_task()
X, y, yv = T.X, T.y_premium, T.y_value.to_numpy()
names = [n.split("__")[-1] for n in T.feature_names]
print("Working with", X.shape[1], "standardised features")
```

**OUTPUT · produced by running the block above**

```
Working with 16 standardised features
```

## The curse, measured

**CODE · steps/step09_dimensionality.py · block "curse"**

```
# In high dimensions, 'near' and 'far' stop meaning anything.
rng = np.random.RandomState(0)
rows = []
for d in [2, 10, 100, 1000, 10000]:
    P = rng.rand(500, d); q = rng.rand(1, d)
    dist = np.linalg.norm(P - q, axis=1)
    rows.append((d, dist.min(), dist.max(), dist.min() / dist.max()))
print(pd.DataFrame(rows, columns=["dimensions", "nearest", "farthest",
    "nearest/farthest"]).round(3).to_string(index=False))
print("-> as d grows the nearest neighbour is almost as far as the farthest one: distance-based methods
    degrade.")
# fraction of the unit hypercube within 0.001 of its surface
print("Share of a random point's volume 'near the border' (within 0.001) in d=1000: %.1f%%" % (100 * (1 - 0.998
    ** 1000)))
```

**OUTPUT · produced by running the block above**

```
 dimensions  nearest  farthest  nearest/farthest
          2    0.012     1.105             0.011
         10    0.759     1.939             0.392
        100    3.429     4.607             0.744
       1000   12.413    13.759             0.902
      10000   40.260    41.679             0.966
-> as d grows the nearest neighbour is almost as far as the farthest one: distance-based methods degrade.
Share of a random point's volume 'near the border' (within 0.001) in d=1000: 86.5%
```

In 2-D the nearest neighbour is ~1% as far as the farthest. In 10,000-D, it is **97% as far**: 'near' no longer exists. This is why k-NN, RBF-SVMs and k-means degrade in very high dimensions, and why feature selection/reduction matters.

## PCA by hand

**CODE · steps/step09_dimensionality.py · block "pca_by_hand"**

```
# PCA = eigen-decomposition of the covariance matrix = SVD of the centred data.
Xc = X - X.mean(0)
U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
var_ratio_manual = S ** 2 / np.sum(S ** 2)
pca = PCA().fit(X)
print("explained variance ratio (manual SVD):", var_ratio_manual[:5].round(4))
print("explained variance ratio (sklearn)   :", pca.explained_variance_ratio_[:5].round(4))
print("max |difference| = %.2e" % np.abs(var_ratio_manual - pca.explained_variance_ratio_).max())
cum = np.cumsum(pca.explained_variance_ratio_)
d95 = int(np.argmax(cum >= 0.95) + 1); d99 = int(np.argmax(cum >= 0.99) + 1)
print("Components needed for 95%% of the variance: %d | for 99%%: %d (of %d)" % (d95, d99, X.shape[1]))
record("pca_d95", d95); record("pca_d99", d99)
loadings = pd.DataFrame(pca.components_[:3].T, index=names, columns=["PC1", "PC2", "PC3"]).round(2)
print("\nWhat do the first components mean? (loadings)")
print(loadings.to_string())
```

**OUTPUT · produced by running the block above**

```
explained variance ratio (manual SVD): [0.3346 0.1817 0.1596 0.0868 0.0783]
explained variance ratio (sklearn)   : [0.3346 0.1817 0.1596 0.0868 0.0783]
max |difference| = 1.94e-16
Components needed for 95% of the variance: 8 | for 99%: 11 (of 16)

What do the first components mean? (loadings)
                            PC1   PC2   PC3
longitude                  0.08 -0.47 -0.48
latitude                  -0.08  0.49  0.48
housing_median_age        -0.22 -0.12  0.13
total_rooms                0.48  0.13 -0.02
total_bedrooms             0.49 -0.01  0.11
population                 0.47 -0.04  0.08
households                 0.49 -0.02  0.12
median_income              0.05  0.36 -0.46
rooms_per_house            0.01  0.37 -0.31
bedrooms_ratio            -0.02 -0.47  0.40
people_per_house          -0.00  0.00 -0.00
ocean_proximity_<1H OCEAN  0.02 -0.11 -0.12
ocean_proximity_INLAND    -0.00  0.07  0.06
ocean_proximity_ISLAND    -0.00 -0.00  0.00
ocean_proximity_NEAR BAY  -0.02  0.06  0.08
ocean_proximity_NEAR OCEAN -0.00 -0.02 -0.02
```

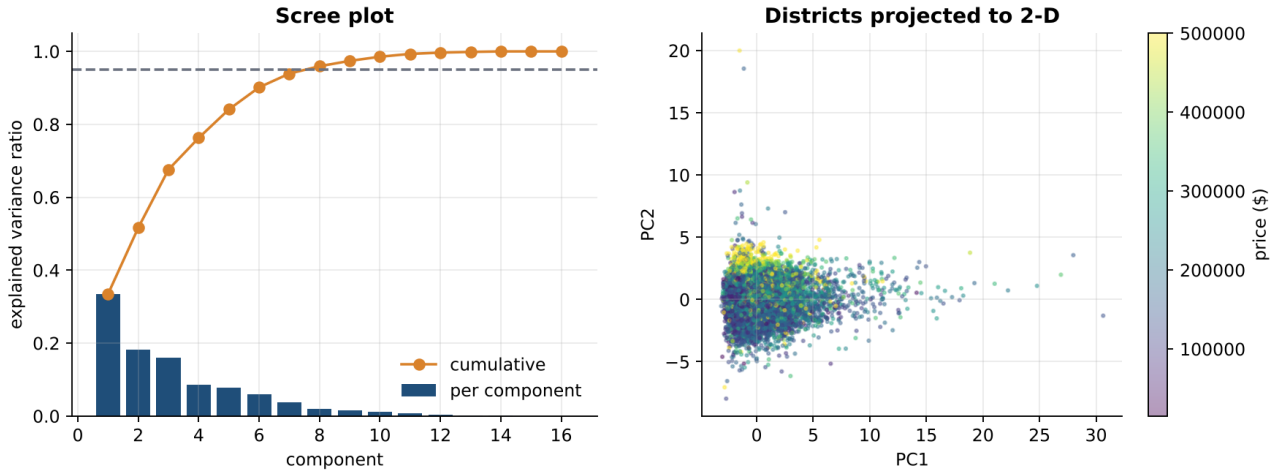



***Figure 26.*** *Left: scree plot with cumulative explained variance (dashed line: 95%). Right: districts projected on the first two principal components, coloured by price.*

We computed PCA **two ways**: the SVD of the centred data by hand, and `sklearn.decomposition.PCA`. Explained variance ratios agree to machine precision (difference $2\times10^{-16}$): PCA *is* the SVD. Our 16 features need **8 components for 95%** and **11 for 99%** of the variance. The loadings table is interpretable: **PC1** is dominated by `total_rooms`, `total_bedrooms`, `population`, `households` (all ≈ 0.47–0.49): a *district size* axis. **PC2** contrasts latitude/ratios with longitude: a *geography* axis (north-west versus south-east). PC1, the biggest source of variance, says almost nothing about price (Figure 26 right: the colours are not ordered along PC1). **High variance ≠ predictive power**. PCA is unsupervised.

## Reconstruction and PCA variants

**CODE · steps/step09_dimensionality.py · block “reconstruction”**

```
for k in [2, 4, 8, 11, 16]:
    p_ = PCA(n_components=k).fit(X); rec = p_.inverse_transform(p_.transform(X))
    print("k=%2d components: reconstruction RMSE (in std-dev units) = %.3f | variance kept %.1f%%" %
          (k, np.sqrt(np.mean((X - rec) ** 2)), 100 * p_.explained_variance_ratio_.sum()))
# incremental and randomised PCA give the same answer, but scale to huge data
ipca = IncrementalPCA(n_components=8, batch_size=2000).fit(X)
rpca = PCA(n_components=8, svd_solver="randomized", random_state=0).fit(X)
full8 = PCA(n_components=8).fit(X)
print("\nIncrementalPCA vs PCA: max |diff| in explained variance = %.2e" % np.abs(ipca.explained_variance_ratio_
    - full8.explained_variance_ratio_).max())
print("Randomised PCA  vs PCA: max |diff| in explained variance = %.2e" % np.abs(rpca.explained_variance_ratio_
    - full8.explained_variance_ratio_).max())
```

**OUTPUT · produced by running the block above**

```
k= 2 components: reconstruction RMSE (in std-dev units) = 0.594 | variance kept 51.6%
k= 4 components: reconstruction RMSE (in std-dev units) = 0.416 | variance kept 76.3%
k= 8 components: reconstruction RMSE (in std-dev units) = 0.172 | variance kept 95.9%
k=11 components: reconstruction RMSE (in std-dev units) = 0.071 | variance kept 99.3%
k=16 components: reconstruction RMSE (in std-dev units) = 0.000 | variance kept 100.0%

IncrementalPCA vs PCA: max |diff| in explained variance = 1.89e-04
Randomised PCA  vs PCA: max |diff| in explained variance = 1.94e-16
```

Reconstruction error falls as components are added; with 11 of 16 components 99.3% of the variance is kept and the reconstruction error is 0.07 standard deviations. Incremental and randomised PCA reproduce the exact solution, differing only by $10^{-4}$ or less in explained variance: use them when data is big.

## Does PCA help a supervised model? (honest test)

**CODE · steps/step09_dimensionality.py · block “pca_before_model”**

```
# Does reducing dimensions before modelling help? Evaluate with proper cross-validation (PCA inside the
    pipeline).
rows = []
for k in [3, 5, 8, 11, 16]:
    for name, est in [("Linear", LinearRegression()), ("Random forest", RandomForestRegressor(60,
        min_samples_leaf=2, max_features=0.6, n_jobs=2, random_state=RANDOM_STATE))]:
        pipe = make_pipeline(PCA(k, random_state=0), est)
        s = -cross_val_score(pipe, X, yv, cv=4, scoring="neg_root_mean_squared_error", n_jobs=1).mean()
        rows.append((k, name, s))
pc = pd.DataFrame(rows, columns=["n_components", "model", "cv_rmse"]).pivot(index="n_components",
    columns="model", values="cv_rmse").round(0)
raw_rf = -cross_val_score(RandomForestRegressor(60, min_samples_leaf=2, max_features=0.6, n_jobs=2,
    random_state=RANDOM_STATE), X, yv, cv=4, scoring="neg_root_mean_squared_error").mean()
print(pc.to_string())
print("\nRandom forest on the ORIGINAL 16 features: CV RMSE = %.0f" % raw_rf)
print("=> At k=16 PCA is only a rotation, yet the forest is already worse: axis-aligned splits lose the
    meaningful axes. Dropping components then removes signal too.")
record("pca_rf_raw", raw_rf); record("pca_rf_k16", float(pc.loc[16, "Random forest"])); record("pca_rf_k8",
    float(pc.loc[8, "Random forest"])); record("pca_lin_k16", float(pc.loc[16, "Linear"]))
```

**OUTPUT · produced by running the block above**

```
model           Linear  Random forest
n_components
3             103330.0        92633.0
5              87724.0        68464.0
8              71598.0        58058.0
11             69424.0        56968.0
16             68747.0        53696.0

Random forest on the ORIGINAL 16 features: CV RMSE = 50112
=> At k=16 PCA is only a rotation, yet the forest is already worse: axis-aligned splits lose the meaningful
    axes. Dropping components then removes signal too.
```

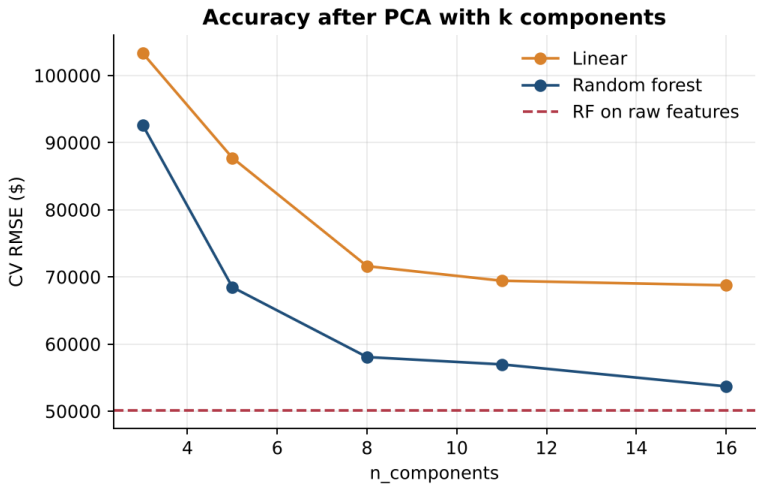



***Figure 27.*** *CV RMSE as a function of PCA components kept before a linear model or a random forest. The dashed line is the forest on the original features.*

Result: *PCA hurt both models*. The forest on the original 16 features has CV RMSE $50,112; with all 16 components (a pure rotation, no information lost) it is already worse ($53,696), and with 8 components (95% variance) it is $58,058. Two reasons: trees rely on axis-aligned, meaningful features (rotation destroys that), and high-variance directions are not the predictive ones. For the linear model, 16 components reproduce the original score (68.7k vs 68.4k), and fewer components lose signal. **Lesson: dimensionality reduction is a hypothesis, and you validate it like any other.** With only 16 well-chosen features we have no need for it. It becomes valuable with hundreds of correlated features, for visualisation, or for denoising.

## Kernel PCA with a supervised grid search

**CODE · steps/step09_dimensionality.py · block “kernel_pca”**

```
# Supervised use: choose the kernel and its hyper-parameters with a grid search on the final classification
    score.
sub = np.random.RandomState(1).permutation(len(X))[:3000]
pipe = make_pipeline(KernelPCA(n_components=6, kernel="rbf"), LogisticRegression(max_iter=3000))
grid = GridSearchCV(pipe, {"kernelpca__gamma": [0.003, 0.01, 0.03, 0.1], "kernelpca__kernel": ["rbf",
    "sigmoid"]}, cv=3, n_jobs=2)
t0 = time.time(); grid.fit(X[sub], y[sub])
print("kernel-PCA + logistic regression grid (3000 rows) took %.0fs" % (time.time() - t0))
print("best:", grid.best_params_, "| CV accuracy %.3f" % grid.best_score_)
lin = cross_val_score(make_pipeline(PCA(6), LogisticRegression(max_iter=3000)), X[sub], y[sub], cv=3).mean()
print("Plain PCA(6) + logistic regression CV accuracy: %.3f" % lin)
record("kpca_acc", grid.best_score_); record("pca6_acc", lin)
```

**OUTPUT · produced by running the block above**

```
kernel-PCA + logistic regression grid (3000 rows) took 3s
best: {'kernelpca__gamma': 0.01, 'kernelpca__kernel': 'rbf'} | CV accuracy 0.839
Plain PCA(6) + logistic regression CV accuracy: 0.844
```

We tuned the kernel and γ by using the **final classifier's CV accuracy** as the criterion (the standard way to evaluate an unsupervised step in a supervised pipeline). The best RBF kernel PCA (γ = 0.01, 6 components) obtained 83.9%, no better than plain PCA with 6 components (84.4%) on this data: non-linearity in the *feature space* adds nothing because the relevant non-linearity is in the *label* structure, which these classifiers can handle after projecting.

## t-SNE for visualisation

**CODE · steps/step09_dimensionality.py · block “tsne”**

```
# t-SNE is for VISUALISATION only: it preserves neighbourhoods, not distances, and cannot transform new data.
sub = np.random.RandomState(2).permutation(len(X))[:3000]
t0 = time.time()
E = TSNE(n_components=2, perplexity=40, init="pca", learning_rate="auto", random_state=0).fit_transform(X[sub])
print("t-SNE on 3000 districts took %.0fs" % (time.time() - t0))
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
Zp = PCA(2).fit_transform(X[sub])
for a, Zs, ttl in [(ax[0], Zp, "PCA (linear)"), (ax[1], E, "t-SNE (non-linear)")]:
    sc = a.scatter(Zs[:, 0], Zs[:, 1], c=T.y_tier[sub], s=5, cmap="viridis"); a.set_title(ttl);
        a.set_xticks([]); a.set_yticks([])
cb = plt.colorbar(sc, ax=ax, ticks=[0, 1, 2, 3], shrink=.8); cb.ax.set_yticklabels(["budget", "lower-mid",
    "upper-mid", "premium"])
savefig(fig, "fig28_tsne")
from sklearn.neighbors import KNeighborsClassifier
for nm, Zs in [("PCA-2D", Zp), ("t-SNE-2D", E)]:
    print("%-9s kNN(15) price-tier accuracy in 2-D: %.3f" % (nm, cross_val_score(KNeighborsClassifier(15), Zs,
        T.y_tier[sub], cv=5).mean()))
```

**OUTPUT · produced by running the block above**

```
t-SNE on 3000 districts took 10s
PCA-2D    kNN(15) price-tier accuracy in 2-D: 0.333
t-SNE-2D  kNN(15) price-tier accuracy in 2-D: 0.570
```

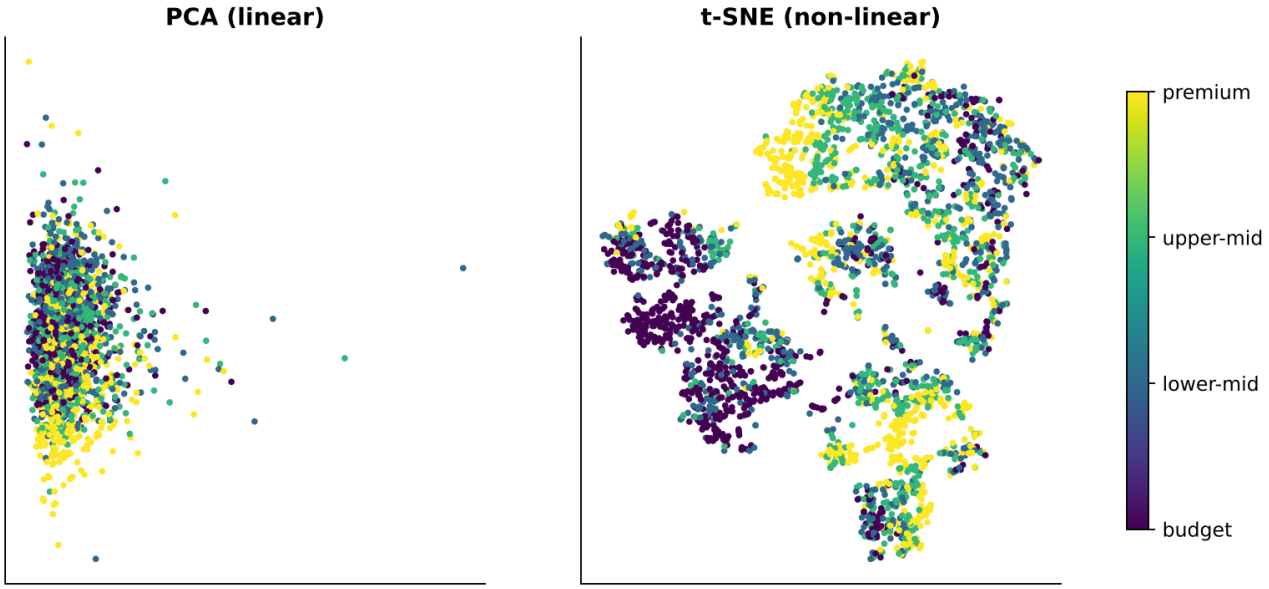



***Figure 28.*** *The same 3,000 districts with PCA (left) and t-SNE (right), coloured by price tier. t-SNE separates the tiers into visibly structured regions; PCA does not.*

t-SNE shows that districts with similar prices form neighbourhoods in the full feature space: a kNN on the 2-D t-SNE coordinates gets 57% tier accuracy versus 33% on the 2-D PCA coordinates. **Caution**: t-SNE was fitted on all 3,000 points at once (it cannot transform new data), so that accuracy is a statement about *visual separability*, not a valid generalisation estimate. Use t-SNE to *look*, never to feed production models.

**Common mistakes in dimensionality reduction**
• Fitting PCA on all data before the train/validation split (leakage): put it in the Pipeline. • Applying PCA without scaling. • Believing 'more variance = more important'. • Interpreting distances/cluster sizes in a t-SNE plot. • Using PCA before tree models by habit.

## 8.3 Hands-on experiments

1. Use `PCA(n_components=0.95)` inside a `Pipeline` with logistic regression and cross-validate. How many components are kept in each fold?
2. Compress a few districts with 4 components and reconstruct them. Which original features are reconstructed worst? Why?
3. Re-run t-SNE with perplexity 5, 40, 200. What changes?

4. Try `LocallyLinearEmbedding(n_neighbors=20)` on the 3,000-row sample. Is the result as clear as t-SNE?
5. Project the standardised data with `GaussianRandomProjection(eps=0.3)` and measure how well pairwise distances are preserved.

## 8.4 Challenge questions

**Q1. What does PCA maximise, and what is the connection to the SVD?**

It finds orthogonal directions of maximal variance (equivalently minimal reconstruction error). With centred $X = U\Sigma V^T$, the columns of V are the principal components and $\sigma^2/(m-1)$ are the variances along them.

**Q2. Why standardise before PCA?**

PCA maximises variance, so a feature measured in large units would dominate the first component purely because of its scale. Standardising puts features on equal footing (unless the original units are meaningful).

**Q3. How do you choose the number of components?**

Keep enough to explain a target share of variance (e.g. 95%), look for an elbow in the scree plot, or treat d as a hyper-parameter tuned by the downstream model's cross-validated performance.

**Q4. Does PCA improve classifier accuracy?**

Not necessarily. It can reduce noise, multicollinearity and cost, but it can also discard predictive low-variance directions. Test it with cross-validation (in our project it lowered accuracy).

**Q5. PCA vs t-SNE?**

PCA is linear, deterministic, invertible, gives a transform for new data and is suitable as a preprocessing step. t-SNE is non-linear and stochastic, preserves local neighbourhoods for visualisation, cannot transform new data, and its global geometry is not meaningful.

**Q6. What is the curse of dimensionality?**

As dimensions grow, data become sparse; distances concentrate; the volume near the surface dominates; required sample size grows exponentially; models overfit and distance-based methods degrade.

**Q7. When use Incremental or Randomised PCA?**

Incremental: data too big for memory or arriving in streams (online). Randomised: large data with small target dimension d, much faster at nearly the same accuracy.

**Q8. What is reconstruction error and how does it relate to explained variance?**

The mean squared distance between the data and its projection mapped back. It equals the sum of the variances of the discarded components: more explained variance → smaller reconstruction error.

**Q9. Why use a pipeline with PCA in cross-validation?**

So that PCA is re-fitted on each training fold. Otherwise validation data influences the learned components (leakage).

# Chapter 9 · Unsupervised Learning Techniques

**Project step 10** — find HomeLens's market segments, use clusters as features, test a semi-supervised shortcut, detect dense urban cores with DBSCAN, and flag anomalous records with a Gaussian mixture. Code: `steps/step10_unsupervised.py`.

## 9.1 Concepts you must own

Most data is unlabelled. Unsupervised learning extracts structure without labels. The three classic tasks: **clustering** (group similar instances), **anomaly detection** (learn what is normal; flag what is not) and **density estimation** (model the probability density function of the data, which also enables anomaly detection). Typical uses: customer segmentation, data analysis, dimensionality reduction, **feature engineering**, anomaly/fraud detection, semi-supervised learning, search by similarity, and image segmentation.

### K-means

Pick k, then iterate: **assign** each instance to the nearest centroid; **update** each centroid to the mean of its instances. It stops when assignments stop changing (guaranteed to converge, but maybe to a *local* optimum). It minimises the **inertia**: the sum of squared distances between instances and their closest centroid. It is fast (roughly linear in m, k, n) but assumes **roughly spherical, similar-sized clusters** and needs **scaled features**. It produces *hard* assignments (`predict`) and distances to all centroids (`transform`).

| Topic | Facts |
|---|---|
| **Initialisation** | Random init can land in poor optima (inertia spread of 89% across seeds in our experiment). **k-means++** (default) picks well-spread initial centroids; `n_init` restarts keep the best inertia |
| **Mini-batch k-means** | Uses mini-batches: faster (and out-of-core) at a slightly worse inertia |
| **Choosing k** | *Inertia* always decreases with k (look for an **elbow**, often ambiguous). The **silhouette coefficient** (b − a)/max(a, b) per instance (a = mean intra-cluster distance, b = mean distance to the nearest other cluster) ranges from −1 to +1; take its mean; higher is better. The silhouette diagram shows cluster sizes and quality |
| **Limits** | Must choose k; fails on elongated, differently sized or non-convex clusters; sensitive to outliers and scale |

### Using clustering

- **Preprocessing / feature engineering**: replace the data with each instance's distance (or similarity) to each centroid. We do this in `ClusterSimilarity`.
- **Semi-supervised learning**: when labels are costly, cluster, label only the *representative instance* of each cluster, then propagate its label to the cluster (or to the instances near the centroid). It works when clusters align with classes.
- **Active learning** (the model asks a human to label the instances it is least certain about), and **image segmentation** (cluster pixel colours).

### DBSCAN, hierarchical clustering

**DBSCAN** defines clusters as continuous regions of **high density**. A *core instance* has at least `min_samples` neighbours within distance `eps`; clusters are chains of core instances and their neighbours; everything else is **noise** (label −1). It finds clusters of arbitrary shape, needs no k, and is robust to outliers, but struggles with clusters of *different densities* and has no `predict` (use a classifier on its output for new points). **Agglomerative (hierarchical) clustering** repeatedly merges the closest

clusters (with a linkage rule such as Ward) building a dendrogram; flexible with connectivity constraints, but memory-heavy.

### Gaussian mixture models (GMM)

A GMM assumes the data are generated from a mixture of k Gaussians with unknown means, covariances and mixing weights, and fits them by **Expectation-Maximisation (EM)**: *E-step* computes each instance's soft responsibility for each cluster; *M-step* re-estimates the parameters using those responsibilities (a soft k-means). It gives **soft clustering** (`predict_proba`), ellipsoidal clusters (`covariance_type`), a generative model (`sample`) and a **density** (`score_samples`). Choose k with the **BIC** (Bayesian information criterion = p·ln m − 2 ln $\hat{L}$) or **AIC** (2p − 2 ln $\hat{L}$), where p is the number of parameters and $\hat{L}$ the maximised likelihood: they reward fit and punish complexity (BIC more strongly).

**Anomaly detection with a density model**: instances in regions of density below a threshold are anomalies. The threshold sets the false-positive/false-negative trade-off (e.g. flag the 2% lowest density). **Novelty detection** assumes a *clean* training set; anomaly detection assumes it may be contaminated. Other tools: Isolation Forest, One-Class SVM, PCA reconstruction error, autoencoders (Part II).

## 9.2 Project step 10 — hands on

**CODE · steps/step10_unsupervised.py · block “setup”**

```
import sys; sys.path.insert(0, "src")
import numpy as np, pandas as pd, matplotlib.pyplot as plt, time
from sklearn.cluster import DBSCAN, KMeans, MiniBatchKMeans, AgglomerativeClustering
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.mixture import GaussianMixture
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from homelens.tasks import load_task
from homelens.features import build_preprocessing
from homelens.utils import set_style, savefig, record, rmse, PALETTE
from homelens.config import RANDOM_STATE
set_style()
T = load_task()
geo = T.X_raw[["longitude", "latitude"]].to_numpy()
X, y, yv = T.X, T.y_premium, T.y_value.to_numpy()
```

### How many market regions? Inertia and silhouette

**CODE · steps/step10_unsupervised.py · block “choose_k”**

```
ks = range(2, 16)
inertia, sil = [], []
for k in ks:
    km = KMeans(k, n_init=10, random_state=RANDOM_STATE).fit(geo)
    inertia.append(km.inertia_); sil.append(silhouette_score(geo, km.labels_, sample_size=4000, random_state=0))
print(pd.DataFrame({"k": list(ks), "inertia": np.round(inertia, 0), "silhouette": np.round(sil,
    3)}).to_string(index=False))
bestk = list(ks)[int(np.argmax(sil))]
print("Highest silhouette at k =", bestk, "(there is no single 'right' k - it is a modelling choice)")
record("kmeans_best_k_sil", bestk)
```

**OUTPUT · produced by running the block above**

```
 k  inertia  silhouette
 2  21428.0       0.752
 3  14675.0       0.649
 4  11449.0       0.599
 5   8286.0       0.583
 6   6413.0       0.570
 7   5430.0       0.571
 8   4815.0       0.527
 9   4201.0       0.495
10   3710.0       0.505
11   3408.0       0.520
12   3107.0       0.447
13   2842.0       0.488
14   2492.0       0.458
15   2218.0       0.470
Highest silhouette at k = 2 (there is no single 'right' k - it is a modelling choice)
```

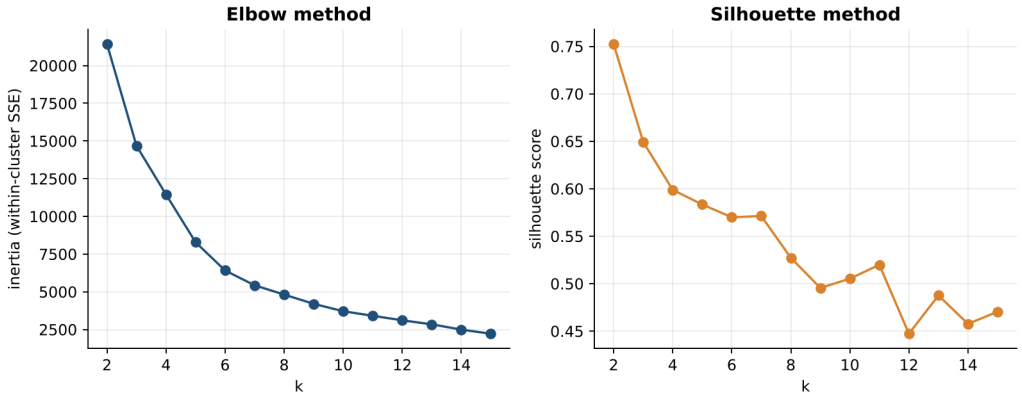



***Figure 29.*** *Elbow and silhouette plots for k-means on (longitude, latitude).*

The inertia curve bends smoothly with no sharp elbow, and the silhouette is highest at k = 2 (north/south California: 0.75), falling after that. **There is no single correct k.** k = 2 is mathematically 'best' but useless for a business that needs regional markets; k ≈ 6–8 balances detail and stability (silhouette ≈ 0.53–0.57). This is a typical situation: the metric guides, domain knowledge decides. We choose **k = 8** for the segmentation report.

## Initialisation matters

**CODE · steps/step10_unsupervised.py · block “kmeans_details”**

```
for init, n_init in [("random", 1), ("k-means++", 1), ("k-means++", 10)]:
    sc = [KMeans(8, init=init, n_init=n_init, random_state=s).fit(geo).inertia_ for s in range(8)]
    print("init=%-10s n_init=%-2d -> inertia over 8 seeds: best %.0f | worst %.0f | spread %.1f%%" %
          (init, n_init, min(sc), max(sc), 100 * (max(sc) / min(sc) - 1)))
t0 = time.time(); KMeans(8, n_init=3, random_state=0).fit(X); t_full = time.time() - t0
t0 = time.time(); mb = MiniBatchKMeans(8, n_init=3, batch_size=1024, random_state=0).fit(X); t_mb = time.time()
    - t0
print("KMeans %.2fs vs MiniBatchKMeans %.2fs on the 16-feature data" % (t_full, t_mb))
km8 = KMeans(8, n_init=10, random_state=RANDOM_STATE).fit(geo)
fig, ax = plt.subplots(figsize=(6, 5.2))
ax.scatter(geo[:, 0], geo[:, 1], c=km8.labels_, s=3, cmap="tab10"); ax.scatter(*km8.cluster_centers_.T,
    marker="X", s=140, c="k", edgecolor="w")
ax.set_xlabel("longitude"); ax.set_ylabel("latitude"); ax.set_title("k-means (k=8) market regions")
savefig(fig, "fig30_kmeans_map")
print("\nAverage price per k-means region:")
reg = pd.DataFrame({"region": km8.labels_, "price": yv}).groupby("region").price.agg(["count", "mean"]).round(0)
print(reg.to_string())
print("-> the clusters are geographic markets; pricing differs strongly between them.")
```

**OUTPUT · produced by running the block above**

```
init=random     n_init=1  -> inertia over 8 seeds: best 4816 | worst 9103 | spread 89.0%
init=k-means++  n_init=1  -> inertia over 8 seeds: best 4871 | worst 5195 | spread 6.7%
init=k-means++  n_init=10 -> inertia over 8 seeds: best 4815 | worst 4815 | spread 0.0%
KMeans 0.06s vs MiniBatchKMeans 0.02s on the 16-feature data

Average price per k-means region:
        count      mean
region
0        1433  164824.0
1        3885  262691.0
2         892   86337.0
3        1466  182986.0
4        5720  242390.0
5         548   91233.0
6        1774  128388.0
7         794  191220.0
-> the clusters are geographic markets; pricing differs strongly between them.
```

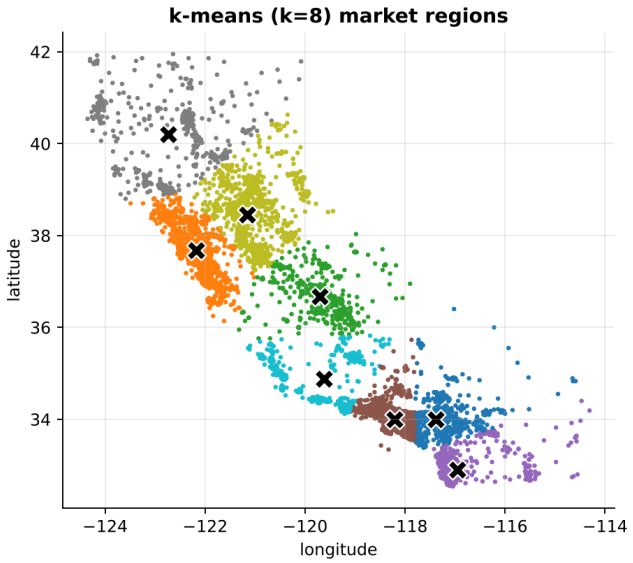



***Figure 30.*** *The eight k-means regions of California (black crosses = centroids).*

With one random initialisation the final inertia varied by **89%** between seeds (some runs were stuck in poor optima); **k-means++** reduced the spread to 6.7%, and 10 restarts gave an identical best result for every seed: the reason `n_init=10` is the right default. The eight regions are geographic markets with very different average prices: from ~$86k to ~$263k, which is a business-ready deliverable by itself.

## Clusters as features: does it help?

**CODE · steps/step10_unsupervised.py · block "clusters_as_features"**

```
# Use clustering as a feature-engineering step (Chapter 2's ClusterSimilarity) - does it help?
rf = RandomForestRegressor(80, min_samples_leaf=2, max_features=0.5, n_jobs=2, random_state=RANDOM_STATE)
res = {}
for lab, uc in [("without geo-cluster features", False), ("with 25 geo-cluster similarity features", True)]:
    pipe = make_pipeline(build_preprocessing(n_clusters=25, use_clusters=uc), rf)
    res[lab] = -cross_val_score(pipe, T.X_raw, yv, cv=4, scoring="neg_root_mean_squared_error", n_jobs=1).mean()
    print("%-42s CV RMSE = %.0f" % (lab, res[lab]))
imp = 100 * (1 - res["with 25 geo-cluster similarity features"] / res["without geo-cluster features"])
print("Improvement: %.1f%%" % imp); record("cluster_feat_gain_pct", imp)
record("rmse_no_geo", res["without geo-cluster features"]); record("rmse_geo", res["with 25 geo-cluster
    similarity features"])
```

**OUTPUT · produced by running the block above**

```
without geo-cluster features               CV RMSE = 49942
with 25 geo-cluster similarity features    CV RMSE = 45074
Improvement: 9.7%
```

With the *same* random forest and CV folds, adding 25 geographic cluster-similarity features lowered the RMSE from **$49,942** to **$45,074**, a **9.7% improvement** from one unsupervised step. It is the single most effective feature we engineered: location is not linear in latitude/longitude, but similarity to market centres is easy for any model to use.

## Semi-supervised learning: an honest negative result

**CODE · steps/step10_unsupervised.py · block "semi_supervised"**

```
# Imagine labelling districts is EXPENSIVE (an expert valuation). We can only afford 20 labels.
rng = np.random.RandomState(0)
n_lab = 20
def draw():                                          # random labelled set (re-draw if only one class shows
    up)
    while True:
        i = rng.choice(len(X), n_lab, replace=False)
        if 0 < y[i].sum() < n_lab: return i
rand_idx = draw()
km = KMeans(n_lab, n_init=5, random_state=0).fit(X)
dist = km.transform(X)
rep_idx = np.argmin(dist, axis=0)                    # the most representative district of each cluster
holdout = np.setdiff1d(np.arange(len(X)), np.r_[rand_idx, rep_idx])
def evaluate(idx, extra=None):
    m = LogisticRegression(max_iter=3000).fit(X[idx], y[idx]); return m.score(X[holdout], y[holdout])
a_rand = np.mean([LogisticRegression(max_iter=3000).fit(X[i], y[i]).score(X[holdout], y[holdout]) for i in
    [draw() for _ in range(20)]])
a_rep = evaluate(rep_idx)
# propagate each representative's label to the whole cluster, train on everything
prop = y[rep_idx][km.labels_]
a_prop = LogisticRegression(max_iter=3000).fit(X, prop).score(X[holdout], y[holdout])
# propagate only to the 20% of points closest to their centroid (cleaner)
close = dist[np.arange(len(X)), km.labels_] <= np.percentile(dist[np.arange(len(X)), km.labels_], 20)
a_prop20 = LogisticRegression(max_iter=3000).fit(X[close], prop[close]).score(X[holdout], y[holdout])
full = LogisticRegression(max_iter=3000).fit(X[holdout[:8000]], y[holdout[:8000]]).score(X[holdout[8000:]],
    y[holdout[8000:]])
print("Accuracy with only %d labels:" % n_lab)
print("  random 20 labelled districts (avg of 20 draws) : %.3f" % a_rand)
print("  20 cluster representatives                     : %.3f" % a_rep)
print("  label propagation to the whole cluster         : %.3f" % a_prop)
print("  propagation to the 20%% closest-to-centroid     : %.3f" % a_prop20)
print("  (reference) all labels, 8000 rows              : %.3f" % full)
purity = sum(max(y[km.labels_ == c].mean(), 1 - y[km.labels_ == c].mean()) * (km.labels_ == c).sum() for c in
    range(n_lab)) / len(y)

print("\nWhy the gain is small: cluster purity (share of districts matching their cluster's majority label) =
    %.3f vs %.3f for always-guess-majority." % (purity, 1 - y.mean()))
print("Cluster-then-label works when clusters align with classes (e.g. handwritten digits). Here price classes
    overlap heavily in feature space.")
record("semi_purity", purity)
record("semi", {"random": a_rand, "rep": a_rep, "prop": a_prop, "prop20": a_prop20, "full": full})
```

**OUTPUT · produced by running the block above**

```
Accuracy with only 20 labels:
  random 20 labelled districts (avg of 20 draws) : 0.789
  20 cluster representatives                     : 0.794
  label propagation to the whole cluster         : 0.789
  propagation to the 20% closest-to-centroid     : 0.759
  (reference) all labels, 8000 rows              : 0.878

Why the gain is small: cluster purity (share of districts matching their cluster's majority label) = 0.782 vs
    0.750 for always-guess-majority.
Cluster-then-label works when clusters align with classes (e.g. handwritten digits). Here price classes overlap
    heavily in feature space.
```

We pretended only 20 labels were affordable and compared: 20 random labels (79%), 20 cluster representatives (79%), label propagation (79%) and 20% closest-to-centroid (76%), against 88% with 8,000 labels. **The trick barely helped.** The diagnostic line explains why: cluster purity is only 0.78, barely above the 0.75 majority baseline: *price classes overlap heavily in feature space*, so the cluster's representative's label is a poor proxy for the whole cluster. Cluster-then-label shines on data where clusters equal classes (the book's handwritten digits). **Reporting a method that did not work, with the reason, is a sign of professionalism**: you now know the assumption to check first (cluster purity).

## DBSCAN: dense urban cores

**CODE · steps/step10_unsupervised.py · block “dbscan”**

```
# Density-based: finds dense urban areas of ANY shape, labels sparse districts as noise (-1).
db = DBSCAN(eps=0.12, min_samples=40).fit(geo)
lab = db.labels_
n_cl = len(set(lab)) - (1 if -1 in lab else 0)
print("DBSCAN found %d dense regions; %.1f%% of districts labelled noise (-1)" % (n_cl, 100 * np.mean(lab ==
    -1)))
sizes = pd.Series(lab[lab >= 0]).value_counts().head(5)
print("Largest regions (label: size):", {int(k): int(v) for k, v in sizes.items()})
fig, ax = plt.subplots(figsize=(6, 5.2))
ax.scatter(geo[lab == -1, 0], geo[lab == -1, 1], s=3, color=PALETTE["grey"], alpha=.5, label="noise")
ax.scatter(geo[lab >= 0, 0], geo[lab >= 0, 1], c=lab[lab >= 0], s=3, cmap="tab20"); ax.legend(markerscale=4)
ax.set_xlabel("longitude"); ax.set_ylabel("latitude"); ax.set_title("DBSCAN (eps=0.12, min_samples=40)")
savefig(fig, "fig31_dbscan")
ag = AgglomerativeClustering(n_clusters=8, linkage="ward").fit(geo[:4000])
print("Hierarchical (Ward) vs k-means agreement on 4000 points: adjusted Rand = %.3f" %
    adjusted_rand_score(ag.labels_, km8.labels_[:4000]))
```

**OUTPUT · produced by running the block above**

```
DBSCAN found 25 dense regions; 11.1% of districts labelled noise (-1)
Largest regions (label: size): {3: 6884, 4: 3564, 1: 1267, 0: 633, 8: 551}
Hierarchical (Ward) vs k-means agreement on 4000 points: adjusted Rand = 0.734
```

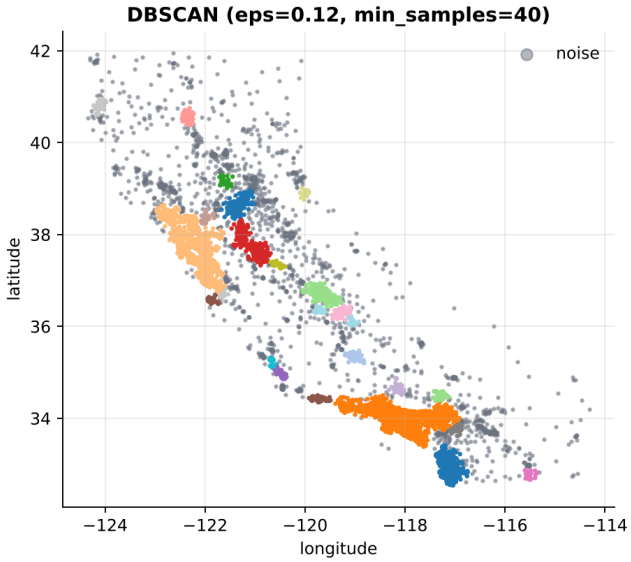



***Figure 31.*** *DBSCAN with eps = 0.12 degrees and min_samples = 40. Grey: noise (rural/sparse districts); coloured: dense regions.*

DBSCAN, with no k, finds 25 dense regions and labels 11% of the districts as noise: the sparse rural and desert areas. The two biggest regions (6,884 and 3,564 districts) are the Los Angeles and Bay Area metros, which k-means would have cut into pieces. Ward hierarchical clustering agrees with k-means moderately (adjusted Rand index 0.73).

## Gaussian mixtures, BIC and anomaly detection

**CODE · steps/step10_unsupervised.py · block “gmm_anomaly”**

```
# Gaussian mixture models: soft clustering + a density estimate we can use for anomaly detection.
Z = StandardScaler().fit_transform(np.c_[np.log(T.X_raw["median_income"]), np.log(T.X_raw["households"]),
    np.log(T.X_raw["population"] / T.X_raw["households"]),
                                        T.X_raw["housing_median_age"], np.log(yv)])
rows = []
for k in range(1, 9):
    g = GaussianMixture(k, n_init=3, random_state=0).fit(Z); rows.append((k, g.bic(Z), g.aic(Z)))
bic = pd.DataFrame(rows, columns=["k", "BIC", "AIC"]).round(0); print(bic.to_string(index=False))
kb = int(bic.loc[bic.BIC.idxmin(), "k"]); print("Lowest BIC at k =", kb)
gm = GaussianMixture(kb, n_init=5, random_state=0).fit(Z)
dens = gm.score_samples(Z)
thr = np.percentile(dens, 2)
anom = dens < thr
print("Flagged the 2%% lowest-density districts as anomalies: %d" % anom.sum())
show = T.X_raw.assign(price=yv, density=dens)[anom].sort_values("density").head(6)[["median_income",
    "households", "population", "housing_median_age", "price", "ocean_proximity"]]
print(show.round(1).to_string())
record("gmm_k", kb); record("n_anomalies", int(anom.sum()))
```

**OUTPUT · produced by running the block above**

```
 k      BIC      AIC
 1 220112.0 219958.0
 2 212309.0 211993.0
 3 209185.0 208707.0
 4 206703.0 206063.0
 5 205966.0 205164.0
 6 205345.0 204381.0
 7 204149.0 203023.0
 8 196759.0 195471.0
Lowest BIC at k = 8
Flagged the 2% lowest-density districts as anomalies: 331
      median_income  households  population  housing_median_age     price ocean_proximity
8040            10.2         6.0      7460.0                45.0  137500.0          INLAND
5583             5.5         7.0      4198.0                36.0   67500.0          INLAND
12205            4.3        13.0      6532.0                46.0  350000.0      NEAR OCEAN
10314            6.1        29.0      6675.0                52.0  225000.0          INLAND
8964             4.2       105.0      8733.0                 5.0  154600.0          INLAND
13366            0.5         4.0         3.0                16.0  350000.0       <1H OCEAN
```

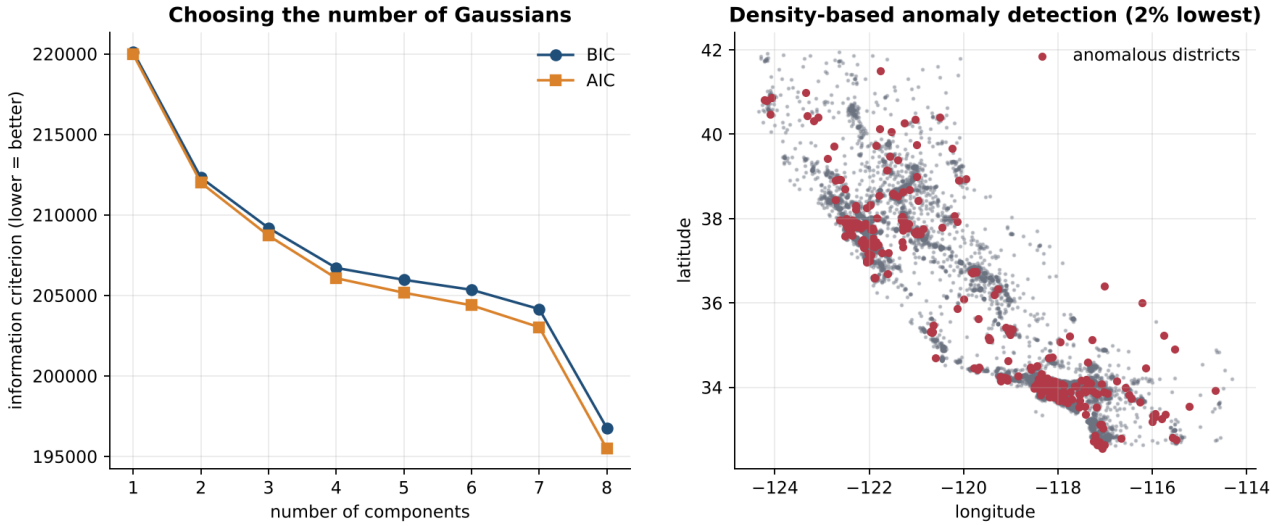



***Figure 32.*** *Left: BIC and AIC versus the number of Gaussian components. Right: the 2% lowest-density districts (red) are scattered statewide.*

Five standardised (log) features describe each district: income, households, people per household, house age and price. BIC keeps decreasing up to k = 8, the edge of our grid, so **the true optimum may be higher; a careful analyst would extend the range** and watch for diminishing returns. We flagged the **331** lowest-density districts. Look at the six strongest anomalies: they are mostly **data errors or special cases**: a district with 6 households and a population of 7,460 (1,243 people per household: a prison or dormitory), another with 4 households and 3 people, and 105 households with 8,733 people. These are the rows data engineers should review *before* training: the same districts that inflated the ratio features' tails. Anomaly detection is as much a data-quality tool as a modelling one.

**Common mistakes in unsupervised learning**
• Not scaling features before k-means/DBSCAN. • Believing the silhouette or the elbow gives 'the' k. • Running k-means once (use `n_init`). • Treating clusters as ground truth rather than a modelling choice; validate by stability and business meaning. • Using DBSCAN with one `eps` on data with very different densities. • Fitting the clustering on test data (leakage) when it feeds a supervised model.

## 9.3 Hands-on experiments

1. Extend the GMM BIC search to k = 20. Where does BIC finally stop improving? Try `covariance_type='diag'` and 'tied'.
2. Run `KMeans` on *standardised* 5-feature data (income, age, ratios, price). Profile each cluster (mean of every feature) and name it as a marketing persona.
3. Vary DBSCAN's `eps` from 0.05 to 0.3. Plot the number of clusters and the noise share. How would you pick eps (hint: k-distance plot)?

4. Use `IsolationForest` instead of the GMM and compare the flagged districts.
5. Raise the label budget in the semi-supervised experiment (20 → 50 → 200). When does the cluster trick start to beat random labelling, if ever?

## 9.4 Challenge questions

**Q1. Describe the k-means algorithm and its objective.**

Initialise k centroids; repeat: assign each point to the nearest centroid, then move each centroid to the mean of its points; until no change. It minimises the within-cluster sum of squared distances (inertia) and converges to a local minimum.

**Q2. Why run k-means several times?**

The result depends on the initialisation; different runs can reach different local optima. Run `n_init` times (with k-means++) and keep the lowest inertia.

**Q3. How can you choose the number of clusters?**

Elbow of inertia, silhouette score/diagram, BIC (for GMMs), stability across resamples, and, above all, whether the clusters are useful for the application.

**Q4. When does k-means fail and what alternative would you use?**

Non-spherical, differently sized/dense or non-convex clusters, many outliers. Use a Gaussian mixture (ellipsoidal), DBSCAN (arbitrary shape, noise), or hierarchical clustering.

**Q5. How does DBSCAN work and what are its two hyper-parameters?**

It labels core points (≥ min_samples neighbours within eps), connects core points that are within eps of each other into clusters, adds border points, and labels the rest noise. eps = neighbourhood radius; min_samples = density requirement.

**Q6. Compare k-means with a Gaussian mixture.**

GMM is a soft, probabilistic generalisation: each cluster has its own mean *and covariance*, instances get membership probabilities, and it provides a density for anomaly detection. k-means is the special case of equal spherical covariances with hard assignments. GMM is slower and can converge to poor solutions/degenerate covariances.

**Q7. BIC versus AIC?**

Both trade log-likelihood against parameter count; BIC = $p \ln m - 2 \ln \hat{L}$ penalises complexity more (for large m), so it tends to choose simpler models than AIC = $2p - 2 \ln \hat{L}$.

**Q8. How can clustering improve a supervised model?**

Add cluster IDs, distances or similarities as features (our +9.7% gain), cluster-then-label for semi-supervised learning, or use cluster-specific models.

**Q9. How would you detect anomalies with a GMM?**

Fit the mixture on normal data, compute log-density with `score_samples`, and flag instances below a density quantile that matches the tolerated false-positive rate.

**Q10. The semi-supervised cluster-then-label trick did not help in our project. Why?**

It assumes clusters are (nearly) pure in the label. Cluster purity was 0.78, close to the 0.75 baseline, so propagated labels were often wrong. Check purity (or label agreement on a handful of points) before using the method.

**Q11. Why must you scale features before k-means?**

Distances drive assignment; unscaled features with large ranges dominate it and clusters then reflect only those features.

# Chapter 10 · Integration: the final model, the one-shot test, and shipping

**Project step 11** — everything now comes together. You tune the winning algorithm, open the locked test set **once**, analyse where the model is weak, and ship a documented, tested artefact. This is the part clients and reviewers actually judge. Code: `steps/step11_final_model.py`, `src/homelens/predict.py`, `tests/test_homelens.py`.

## 10.1 The decision trail

| Decision | Evidence | Chapter |
|---|---|---|
| Use RMSE/MAE in dollars; baseline = mean predictor ($115.7k) | Framing and baselines | 1 |
| Stratified split on income category; test set locked | Random split deviates up to 5% in income mix; stratified 0.5% | 2 |
| Log, ratio and cluster-similarity features inside a Pipeline | Ratios correlate better; clusters cut RMSE by 9.7% | 2, 9 |
| Gradient-boosted trees over linear/SVM/single tree | CV RMSE $46.1k vs $68.4k (linear) vs $60.0k (tree) vs $49.2k (forest) | 4–7 |
| Histogram GB over stacking | Equal within noise; 20× faster; handles missing values | 7 |
| No PCA | PCA lowered CV accuracy/RMSE | 8 |
| Classifier threshold from out-of-fold PR curve | Never from the test set | 3 |

## 10.2 Tuning the final regressor

**CODE · steps/step11_final_model.py · block “setup”**

```
import sys; sys.path.insert(0, "src")
import json, time, numpy as np, pandas as pd, matplotlib.pyplot as plt, joblib
from scipy import stats
from scipy.stats import loguniform, randint
from sklearn.ensemble import HistGradientBoostingClassifier, HistGradientBoostingRegressor
from sklearn.metrics import (mean_absolute_error, precision_recall_curve, precision_score, r2_score,
    recall_score,
                             roc_auc_score, f1_score, confusion_matrix)
from sklearn.model_selection import RandomizedSearchCV, cross_val_predict
from sklearn.pipeline import make_pipeline
from homelens.config import ART_DIR, RANDOM_STATE, TARGET
from homelens.data import make_premium, split, xy, premium_threshold
from homelens.features import build_preprocessing, simple_features_pipeline
from homelens.utils import set_style, savefig, record, rmse, PALETTE
set_style()
train, test = split()
X_train, y_train = xy(train); X_test, y_test = xy(test)
print("train:", X_train.shape, "| test:", X_test.shape, "(touched for the first time in the evaluation block
    below)")
```

**OUTPUT · produced by running the block above**

```
train: (16512, 9) | test: (4128, 9) (touched for the first time in the evaluation block below)
```

**CODE · steps/step11_final_model.py · block “tune_regressor”**

```
# Final regressor = preprocessing (with geo-cluster features) + histogram gradient boosting, tuned by randomised
    search.
pipe = make_pipeline(build_preprocessing(n_clusters=25), HistGradientBoostingRegressor(early_stopping=True,
    random_state=RANDOM_STATE))
space = {
    "columntransformer__geo__n_clusters": randint(10, 45),
    "histgradientboostingregressor__learning_rate": loguniform(0.03, 0.3),
    "histgradientboostingregressor__max_iter": randint(200, 700),
    "histgradientboostingregressor__max_leaf_nodes": randint(15, 80),
    "histgradientboostingregressor__min_samples_leaf": randint(10, 60),
    "histgradientboostingregressor__l2_regularization": loguniform(1e-3, 10),
}
t0 = time.time()
search = RandomizedSearchCV(pipe, space, n_iter=25, cv=4, scoring="neg_root_mean_squared_error",
    random_state=RANDOM_STATE, n_jobs=2)
search.fit(X_train, y_train)
print("Search (25 candidates x 4 folds) took %.0fs" % (time.time() - t0))
bp = {k.split("__")[-1]: (round(float(v), 4) if isinstance(v, float) else int(v)) for k, v in
    search.best_params_.items()}
print("Best params:", bp)
print("Best CV RMSE on the training set: $%.0f" % -search.best_score_)
record("final_cv_rmse", -search.best_score_); record("final_params", bp)
model = search.best_estimator_
```

**OUTPUT · produced by running the block above**

```
Search (25 candidates x 4 folds) took 68s
Best params: {'n_clusters': 44, 'l2_regularization': 1.2164, 'learning_rate': 0.0356, 'max_iter': 430,
    'max_leaf_nodes': 55, 'min_samples_leaf': 37}
Best CV RMSE on the training set: $44003
```

A randomised search of 25 candidates × 4 folds jointly tuned the *preprocessing* (number of geographic clusters) and the booster (learning rate, number of iterations, leaf count, minimum leaf size, $\ell_2$ regularisation), using log-uniform distributions for scale-type parameters. Best CV RMSE: **$44,003**. That figure is our **expectation** for new data, with a fold-to-fold uncertainty around ±$1k. We now stop tuning.

## 10.3 The one-shot test evaluation

**CODE · steps/step11_final_model.py · block “evaluate_test”**

```
# THE ONE-SHOT TEST EVALUATION. No more tuning after this block.
pred = model.predict(X_test)
test_rmse, test_mae, test_r2 = rmse(y_test, pred), mean_absolute_error(y_test, pred), r2_score(y_test, pred)
print("TEST RMSE = $%.0f | MAE = $%.0f | R^2 = %.3f" % (test_rmse, test_mae, test_r2))
sq = (pred - y_test.to_numpy()) ** 2
ci = np.sqrt(stats.t.interval(0.95, len(sq) - 1, loc=sq.mean(), scale=stats.sem(sq)))
print("95%% confidence interval for the generalisation RMSE: [$%.0f, $%.0f]" % tuple(ci))
gap = test_rmse - (-search.best_score_)
print("Gap between CV estimate and test RMSE: %+.0f (%.1f%%) -> small gap = trustworthy validation scheme" %
    (gap, 100 * gap / test_rmse))
dummy = rmse(y_test, np.full(len(y_test), y_train.mean()))
print("Baseline (predict the training mean) test RMSE = $%.0f -> our model removes %.0f%% of the error" %
    (dummy, 100 * (1 - test_rmse / dummy)))
for k, v in dict(test_rmse=test_rmse, test_mae=test_mae, test_r2=test_r2, test_ci_low=ci[0], test_ci_high=ci[1],
    test_baseline_rmse=dummy).items(): record(k, v)
```

**OUTPUT · produced by running the block above**

```
TEST RMSE = $41831 | MAE = $27465 | R^2 = 0.866
95% confidence interval for the generalisation RMSE: [$39984, $43601]
Gap between CV estimate and test RMSE: -2172 (-5.2%) -> small gap = trustworthy validation scheme
Baseline (predict the training mean) test RMSE = $114165 -> our model removes 63% of the error
```

On the 4,128 locked districts: **RMSE $41,831**, MAE $27,465, $R^2$ 0.866. The 95% confidence interval for the generalisation RMSE is **[$39,984, $43,601]**, computed from the squared test errors with a t-interval and then square-rooted. The mean-predictor gets $114,165, so the model removes **63% of the baseline error**. The test RMSE is 5% *lower* than the CV estimate: well within the interval, and the direction is plausible (the final model trains on 100% of the training data versus 75% in each CV fold). **What if the test error had been much worse than CV?** That would signal leakage, distribution shift or over-tuning to the folds. You would investigate; you would **not** go back and tune on the test set (that would burn it).

**Why the test set is sacred**
Every time you look at test results and then change something, information about the test set flows into your decisions. After a few cycles the test score is just another validation score and the estimate is optimistic. If you must iterate after seeing it, get *new* data.

## 10.4 Where is the model weak? Segment analysis

**CODE · steps/step11_final_model.py · block "segment_analysis"**

```
seg = pd.DataFrame({"actual": y_test.to_numpy(), "pred": pred, "ocean": X_test.ocean_proximity.to_numpy(),
    "income": X_test.median_income.to_numpy()})
seg["abs_err"] = (seg.actual - seg.pred).abs(); seg["pct_err"] = 100 * seg.abs_err / seg.actual
seg["income_band"] = pd.cut(seg.income, [0, 2.5, 4, 6, 20], labels=["low", "mid", "upper", "high"])
seg["capped"] = seg.actual >= 500001
print("Error by ocean proximity:")
print(seg.groupby("ocean").agg(n=("abs_err", "size"), MAE=("abs_err", "mean"), MAPE_pct=("pct_err",
    "mean")).round(1).to_string())
print("\nError by income band:")
print(seg.groupby("income_band", observed=True).agg(n=("abs_err", "size"), MAE=("abs_err", "mean"),
    MAPE_pct=("pct_err", "mean")).round(1).to_string())
print("\nCapped districts (true value hidden above $500k): n=%d, MAE=%.0f vs %.0f for the rest" %
      (seg.capped.sum(), seg[seg.capped].abs_err.mean(), seg[~seg.capped].abs_err.mean()))
within = (seg.pct_err <= 20).mean() * 100
print("Share of test districts predicted within +/-20%% of the truth: %.1f%%" % within)
record("within20", within)
```

**OUTPUT · produced by running the block above**

```
Error by ocean proximity:
               n       MAE  MAPE_pct
ocean
<1H OCEAN   1859   29239.1      13.8
INLAND      1289   19879.4      17.6
ISLAND         3  112432.9      26.5
NEAR BAY     443   30792.9      13.7
NEAR OCEAN   534   36358.3      17.0

Error by income band:
                n      MAE  MAPE_pct
income_band
low           987  22090.8      19.1
mid          1526  25541.4      15.5
upper        1143  31067.9      14.1
high          472  36193.8      10.7

Capped districts (true value hidden above $500k): n=179, MAE=53433 vs 26288 for the rest
Share of test districts predicted within +/-20% of the truth: 76.6%
```

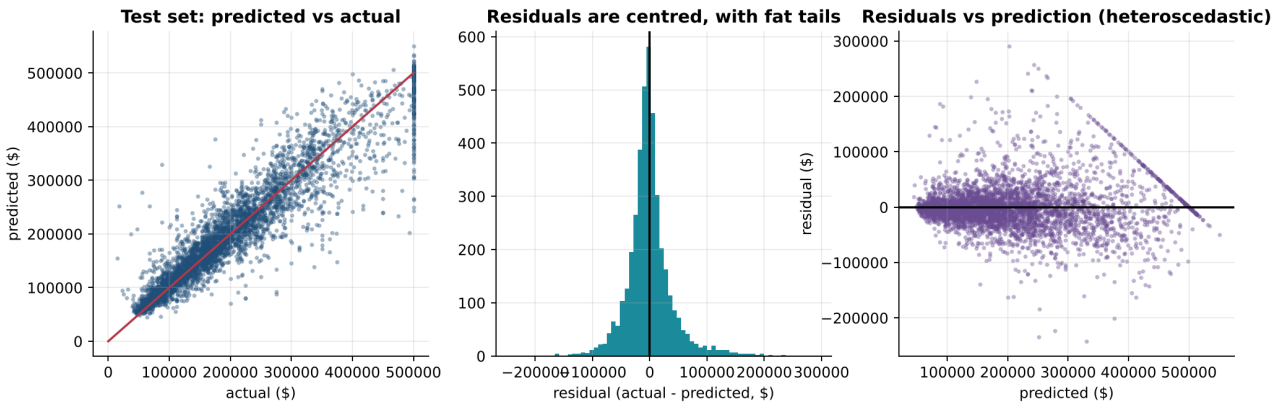



***Figure 33.*** *Test-set diagnostics. Left: predicted vs actual. Middle: residual histogram (centred at 0, fat tails). Right: residuals versus predictions (variance grows with price: heteroscedastic).*

An aggregate RMSE hides important structure. **77%** of districts are predicted within ±20% of the truth. *Absolute* error is larger for expensive/coastal districts (MAE $29–36k) than inland (MAE $20k), but *percentage* error is lowest for high-income districts (10.7% versus 19.1% for low-income): cheap districts are hard in relative terms because a $20k miss on a $90k district is large. The **capped** districts are predicted with an average error of $53k (twice the rest): the model cannot see beyond $500k. The three ISLAND districts have a mean error of $112k: 5 rows in the entire dataset, so the model cannot

learn that category. These findings go in the model card as known limitations.

## 10.5 The premium screener

**CODE · steps/step11_final_model.py · block “premium_classifier”**

```
# Final screening classifier. Threshold chosen on OUT-OF-FOLD training predictions, then frozen.
thr_value = premium_threshold(y_train)
Xs = simple_features_pipeline(); Xtr_s = Xs.fit_transform(X_train); Xte_s = Xs.transform(X_test)
Xtr_s = Xtr_s.toarray() if hasattr(Xtr_s, "toarray") else Xtr_s; Xte_s = Xte_s.toarray() if hasattr(Xte_s,
    "toarray") else Xte_s
ytr_c, yte_c = make_premium(y_train, thr_value), make_premium(y_test, thr_value)
clf = HistGradientBoostingClassifier(learning_rate=0.08, max_iter=400, max_leaf_nodes=31, l2_regularization=1.0,
    early_stopping=True, random_state=RANDOM_STATE)
oof = cross_val_predict(clf, Xtr_s, ytr_c, cv=5, method="predict_proba")[:, 1]
prec, rec, thr = precision_recall_curve(ytr_c, oof)
f1s = 2 * prec * rec / (prec + rec + 1e-12); t_best = thr[np.argmax(f1s[:-1])]
t_prec = thr[np.argmax(prec[:-1] >= 0.90)]
print("Premium threshold = $%.0f" % thr_value)
print("Out-of-fold: best-F1 operating point at probability %.3f ; 90%%-precision point at %.3f" % (t_best,
    t_prec))
clf.fit(Xtr_s, ytr_c); proba = clf.predict_proba(Xte_s)[:, 1]
for name, t in [("max-F1 threshold", t_best), ("90%-precision threshold", t_prec), ("default 0.5", 0.5)]:
    pr = proba >= t
    print("TEST @ %-24s precision %.3f | recall %.3f | F1 %.3f" % (name, precision_score(yte_c, pr),
        recall_score(yte_c, pr), f1_score(yte_c, pr)))
print("TEST ROC-AUC = %.4f" % roc_auc_score(yte_c, proba))
print("Confusion matrix @ max-F1 (rows actual, cols predicted):\n", confusion_matrix(yte_c, proba >= t_best))
record("clf_test_auc", roc_auc_score(yte_c, proba)); pr = proba >= t_best
record("clf_test_precision", precision_score(yte_c, pr)); record("clf_test_recall", recall_score(yte_c, pr));
    record("clf_test_f1", f1_score(yte_c, pr))
record("clf_threshold_prob", float(t_best)); record("premium_value_threshold", thr_value)
```

**OUTPUT · produced by running the block above**

```
Premium threshold = $263900
Out-of-fold: best-F1 operating point at probability 0.441 ; 90%-precision point at 0.607
TEST @ max-F1 threshold         precision 0.886 | recall 0.844 | F1 0.865
TEST @ 90%-precision threshold  precision 0.927 | recall 0.774 | F1 0.844
TEST @ default 0.5              precision 0.904 | recall 0.811 | F1 0.855
TEST ROC-AUC = 0.9724
Confusion matrix @ max-F1 (rows actual, cols predicted):
 [[2938  116]
 [ 168  906]]
```

The classifier is a histogram gradient-boosting classifier. The threshold is chosen on **out-of-fold training predictions**: the max-$F_1$ point (probability 0.44) and the point that gives 90% precision. On the test set the max-$F_1$ operating point reaches **precision 0.89, recall 0.84** (ROC-AUC 0.972); the 90%-precision threshold gives 0.93 precision and 0.77 recall. Compare this to the linear SGD classifier from Chapter 3 (precision 0.80 / recall 0.65 at its default threshold): the acceptance bar of *precision ≥ 0.85* is now met with recall of 0.84, rather than the 0.47 we would have had with the linear model at 90% precision. The business can choose between the operating points knowing exactly what each costs.

## 10.6 Shipping: artefacts, model card, inference, tests

**CODE · steps/step11_final_model.py · block “ship”**

```
model.fit(X_train, y_train)    # (already fitted by search; refit is a no-op safeguard that keeps CV out of the
    artefact)
joblib.dump(model, ART_DIR / "homelens_regressor.joblib")
joblib.dump({"preprocessing": Xs, "classifier": clf, "threshold": float(t_best), "value_threshold": thr_value},
    ART_DIR / "homelens_premium_classifier.joblib")
card = {
    "name": "HomeLens district valuation v1.0", "data": "California 1990 census block groups (20,640 rows)",
    "intended_use": "Rank and screen census districts by typical house value; NOT for pricing an individual
        home.",
    "regressor": "ColumnTransformer(+k-means geo similarity) -> HistGradientBoostingRegressor", "best_params":
        bp,
    "metrics_test": {"rmse": round(test_rmse), "mae": round(test_mae), "r2": round(test_r2, 3), "rmse_95ci":
        [round(float(c)) for c in ci]},
    "classifier_test": {"auc": round(float(roc_auc_score(yte_c, proba)), 4), "precision":
        round(float(precision_score(yte_c, pr)), 3), "recall": round(float(recall_score(yte_c, pr)), 3)},
    "known_limitations": ["Prices capped at $500,001 in the source data", "1990 data - the market has changed",
        "District-level medians, not individual homes",
                          "Weaker in coastal / <1H OCEAN areas", "Will not extrapolate outside the observed
                              feature ranges"],
    "random_state": RANDOM_STATE}
(ART_DIR / "model_card.json").write_text(json.dumps(card, indent=2))
print(json.dumps(card, indent=2)[:1500])
```

**OUTPUT · produced by running the block above**

```
{
  "name": "HomeLens district valuation v1.0",
  "data": "California 1990 census block groups (20,640 rows)",
  "intended_use": "Rank and screen census districts by typical house value; NOT for pricing an individual
      home.",
  "regressor": "ColumnTransformer(+k-means geo similarity) -> HistGradientBoostingRegressor",
  "best_params": {
    "n_clusters": 44,
    "l2_regularization": 1.2164,
    "learning_rate": 0.0356,
    "max_iter": 430,
    "max_leaf_nodes": 55,
    "min_samples_leaf": 37
  },
  "metrics_test": {
    "rmse": 41831,
    "mae": 27465,
    "r2": 0.866,
    "rmse_95ci": [
      39984,
      43601
    ]
  },
... (14 more lines omitted; see reports/outputs/step11_final_model/ship.txt)
```

We persist the entire fitted **pipeline** (preprocessing + model) with `joblib`: raw data in, prediction out, with no possibility of training/serving skew. A **model card** (`artifacts/model_card.json`) documents purpose, data, metrics, limitations and the seed. `predict.py` demonstrates inference on three new districts (including one with a missing bedroom count, which the pipeline imputes):

**OUTPUT of predict.py on three new districts**

```
$ PYTHONPATH=src python -m homelens.predict
   median_income ocean_proximity  predicted_value  premium_probability  is_premium
0           8.33        NEAR BAY         444400.0                0.984        True
1           3.20      NEAR OCEAN         127700.0                0.006       False
2           2.10          INLAND          61200.0                0.002       False
```

**AUTOMATED TESTS**

```
tests/test_homelens.py  (run: pytest -q)
  test_split_is_stratified_and_disjoint ........ income mix of the test set within 0.5% of the full data
  test_split_is_deterministic .................. same seed -> identical split
  test_preprocessing_has_no_nans_and_is_scaled . imputation + scaling behave
  test_preprocessing_handles_unseen_category ... "MARS" does not crash inference
  test_cluster_similarity_shape_and_range ...... custom transformer contract
  test_premium_label_uses_train_threshold_only . 25% of train labelled premium
```

**Deployment and maintenance checklist**

• Wrap `predict.score` behind an API or a scheduled batch job; validate inputs (ranges, missing columns, caps). • Log inputs and predictions. • Monitor: input drift (e.g. PSI per feature), prediction drift, error against delayed ground truth, share of missing/capped values. • Retrain on a schedule or when drift/performance triggers fire; keep dated backups of data, code and model so you can roll back. • Re-run the unit tests and a **shadow comparison** with the previous model before promoting a new one.

## 10.7 Defend your model: the sceptic's interview

Answer these aloud as if a hostile reviewer were asking. They draw on every chapter. Model answers follow.

**Q1. Why should I believe your reported RMSE of about $41.8k?**

Because the test set was split once with stratification and a fixed seed, never used for any decision, and the CV estimate ($44.0k) agrees with it within the 95% CI [$40.0k, $43.6k]. All preprocessing sits inside a pipeline, so no information leaks from validation folds.

**Q2. Your model has $R^2 = 0.87$. Is it good?**

$R^2$ is relative to the mean baseline; what matters is business error: median relative error ~13–19% depending on income band, 77% of districts within ±20%. For *ranking and screening* that is useful; for pricing an individual home it is not (and the model card says so).

**Q3. Why not a neural network?**

On 16k rows of tabular data, gradient-boosted trees usually match or beat neural nets with far less tuning and compute. We will revisit neural nets in Part II and benchmark them on the same split.

**Q4. Your classifier's recall is 0.84. What do the missed 16% look like?**

Mostly borderline districts just under/over the threshold (errors are overwhelmingly adjacent-tier) and capped/high-variance coastal areas. Examining the false negatives is the next error-analysis step.

**Q5. How do you know the model will work next year?**

I don't: this is 1990 data. A deployed version needs monitoring for drift, periodic retraining on fresh data, and an out-of-time test set (train on earlier years, test on later ones).

**Q6. A colleague suggests scaling the full dataset once and then cross-validating, to 'save time'. Respond.**

That leaks validation statistics into training (the scaler sees the validation fold). The effect may be small for scaling but large for target-related steps (feature selection, target encoding). A Pipeline costs nothing and removes the question.

**Q7. What would you do if the business needs predictions above $500k?**

Obtain uncapped labels, or restrict claims to the uncapped range and flag any district predicted near the cap. A censored-target model is another option.

**Q8. Which single change would most improve the model?**

Better location information (neighbourhood-level data, sale prices of individual homes): error maps show large residuals in dense metros where prices vary over short distances; and uncapped labels.

# Appendix A · Master cheat sheets

## A.1 Algorithm comparison (Part I)

| Algorithm | Task | Needs scaling | Strengths | Weaknesses | Key hyper-parameters |
|---|---|---|---|---|---|
| Linear / Ridge / Lasso / Elastic net | Regr. | Yes (esp. regularised) | Fast, interpretable, convex, good baseline | Linear only; sensitive to outliers/collinearity | α, l1_ratio |
| Logistic / softmax regression | Class. | Yes | Probabilities, interpretable coefficients, convex | Linear boundary | C (=1/α), penalty |
| SGD models | Both | Yes | Huge/streaming data, online | Needs η tuning, noisy | loss, α, η schedule, epochs |
| k-NN | Both | Yes | No training, flexible | Slow prediction, curse of dimensionality | k, metric, weights |
| SVM (linear / RBF) | Both | **Yes** | Strong on small/medium data; kernel trick | Poor scaling $O(m^2–m^3)$; no probabilities | C, γ, kernel, ε (SVR) |
| Decision tree | Both | No | Readable, mixed features, no preprocessing | Overfits, unstable, no extrapolation | max_depth, min_samples_leaf |
| Random forest / Extra-Trees | Both | No | Robust, strong default, OOB score, importances | Bigger and slower; no extrapolation | n_estimators, max_features, min_samples_leaf |
| Gradient boosting / HistGB | Both | No | Typically best on tabular data; HistGB handles NaN, fast | More tuning; sequential; can overfit | learning_rate, n_estimators (early stop), max_depth/leaf_nodes, l2 |
| Voting / stacking | Both | Per member | Squeezes extra accuracy | Complexity, small gains | members, meta-learner |
| PCA / Kernel PCA / t-SNE | Reduc. | Yes | Compression, denoising, visualisation | Variance ≠ relevance; t-SNE not for transform | n_components, γ, perplexity |
| k-means / Mini-batch | Cluster | Yes | Fast, simple, `transform` gives features | Needs k, spherical clusters | k, n_init, init |
| DBSCAN | Cluster | Yes | Any shape, noise detection, no k | Varying densities; no predict | eps, min_samples |
| Gaussian mixture | Cluster / density | Yes | Soft clusters, density, anomalies | Local optima, assumes Gaussians | k, covariance_type (BIC) |

## A.2 Metrics cheat sheet

| Problem | Metric | Formula / meaning | Use when |
|---|---|---|---|
| Regression | RMSE | $\sqrt{\text{mean}((\hat{y}-y)^2)}$ | Large errors costly; same units as target |
| Regression | MAE | $\text{mean}\lvert\hat{y}-y\rvert$ | Outliers present; easy to explain |
| Regression | $R^2$ | $1 - SSE/SST$ | Share of variance explained (relative to the mean) |
| Classification | Accuracy | (TP+TN)/all | Balanced classes only |
| Classification | Precision / Recall / $F_1$ | TP/(TP+FP) · TP/(TP+FN) · harmonic mean | Imbalance; costs of FP vs FN differ |
| Classification | ROC-AUC / PR-AUC (AP) | Ranking quality over all thresholds | ROC: balanced; PR: rare positives / false-positive focus |
| Classification | Log loss / cross-entropy | $-\text{mean}[y \log p + (1-y) \log(1-p)]$ | Probability quality; training loss of logistic/softmax |

| Problem | Metric | Formula / meaning | Use when |
|---|---|---|---|
| Clustering | Inertia, silhouette, BIC/AIC, ARI | See Chapter 9 | No labels: internal; labels available: ARI |

## A.3 Symptom → diagnosis → remedy

| Symptom | Likely cause | What to try |
|---|---|---|
| Train error ≪ validation error | Overfitting (variance) | More data; regularise (α↑, C↓, γ↓, depth↓, min_samples_leaf↑); early stopping; ensemble; simpler features |
| Train ≈ validation, both high | Underfitting (bias) | More expressive model; engineer features; α↓; train longer; boosting |
| CV score great, test/production poor | Leakage, sampling bias, drift, tuned-to-validation | Pipelines; check duplicates/time order; fresh holdout; check representativeness |
| Loss diverges / NaN | Learning rate too high, unscaled/outlier features | Lower η; scale; clip outliers; gradient clipping |
| Loss decreases very slowly | η too small or poor conditioning | Increase η; scale features; momentum/Adam (Part II) |
| High accuracy but useless model | Class imbalance | Precision/recall/PR-AUC; resample or weight; threshold tuning |
| SVM predicts one class only | Unscaled features or bad C/γ | StandardScaler; grid-search C, γ on log scale |
| Tree/forest great in CV, bad on new regions | No extrapolation / distribution shift | Check feature ranges; add linear component; widen training data |
| k-means results change between runs | Initialisation | k-means++, n_init ≥ 10, fixed seed |
| Training too slow | Algorithm complexity | Subsample; LinearSVC/SGD/HistGB; mini-batch; randomised search; fewer features (PCA) |

## A.4 Formula sheet (know these cold)

| Name | Formula |
|---|---|
| Linear model / MSE | $\hat{y} = \theta \cdot x$ ; $\mathrm{MSE}(\theta) = (1/m)\,\Sigma\,(\theta \cdot x^{(i)} - y^{(i)})^2$ |
| Normal equation | $\theta = (X^T X)^{-1} X^T y$ (use the pseudo-inverse $X^+ y$ in practice) |
| Gradient (MSE) & update | $\nabla = (2/m)\, X^T(X\theta - y)$ ; $\theta \leftarrow \theta - \eta\,\nabla$ |
| Ridge / Lasso cost | $\mathrm{MSE} + \alpha \cdot \tfrac{1}{2}\Sigma\theta_j^2$ ; $\mathrm{MSE} + \alpha \cdot \Sigma\lvert\theta_j\rvert$ |
| Sigmoid / logistic loss / gradient | $\sigma(t) = 1/(1+e^{-t})$ ; $J = -(1/m)\Sigma[y \log \hat{p} + (1-y)\log(1-\hat{p})]$ ; $\nabla = (1/m)\, X^T(\sigma(X\theta) - y)$ |
| Softmax / cross-entropy | $\hat{p}_k = e^{s_k}/\Sigma_j e^{s_j}$ ; $J = -(1/m)\,\Sigma_i \Sigma_k\, y_k^{(i)} \log \hat{p}_k^{(i)}$ |
| SVM soft margin / margin width | $\min \tfrac{1}{2}\lVert w\rVert^2 + C\Sigma\xi_i$ ; $\text{width} = 2/\lVert w\rVert$ ; RBF $K(a,b) = \exp(-\gamma\lVert a-b\rVert^2)$ |
| Gini / CART cost | $G = 1 - \Sigma p_k^2$ ; $J = (m_l/m)G_l + (m_r/m)G_r$ |
| Precision, recall, $F_1$ | TP/(TP+FP) ; TP/(TP+FN) ; 2PR/(P+R) |
| PCA | $X = U\Sigma V^T$ ; $X_d = X\,V_d$ ; explained variance ratio$_j = \sigma_j^2/\Sigma\sigma^2$ |
| K-means inertia / silhouette | $\Sigma_i \lVert x_i - \mu_{c(i)}\rVert^2$ ; $s = (b - a)/\max(a, b)$ |
| BIC / AIC | $p \ln m - 2 \ln \hat{L}$ ; $2p - 2 \ln \hat{L}$ |
| Bias–variance | $E[(y - \hat{y})^2] = \text{Bias}^2 + \text{Variance} + \text{irreducible noise}$ |

# Appendix B · The mixed challenge round

These cross-chapter scenarios are deliberately open-ended: they train *decisions*, not definitions. Cover the answer, respond aloud, then compare.

**Q1. A bank's fraud rate is 0.2%. A model reports 99.8% accuracy. Your verdict, and what do you measure instead?**

Meaningless: predicting 'no fraud' gets that. Use precision/recall (or PR-AUC), choose a threshold from the cost of a missed fraud versus a false block, and validate with stratified folds on out-of-time data.

**Q2. You have 50 features, 300 rows, and your linear model overfits badly. List three actions in order of preference.**

(1) Regularise (Ridge/Lasso/Elastic net) with α by CV; (2) reduce features (Lasso selection, domain knowledge, PCA inside the pipeline); (3) get more data or use a high-bias model. Avoid flexible models (deep trees, RBF with large γ).

**Q3. You have 20 million rows and 50 numeric features. Which of linear SVM, kernel SVM, random forest, HistGB, k-NN would you test first, and why?**

HistGradientBoosting (fast, scales, strong on tabular), plus an SGD/linear baseline. Kernel SVM ($O(m^2+)$) and k-NN (slow prediction, memory) are impractical; a random forest is feasible but larger and slower than HistGB.

**Q4. Validation accuracy is 95% but after deployment it is 78%. Give five hypotheses and one test for each.**

Leakage (re-audit the pipeline/features); sampling bias or data mismatch (compare feature distributions train vs production); concept drift (compare performance over time); duplicates between train and validation (hash rows); training/serving skew (run the production pipeline on a validation sample and compare predictions).

**Q5. Why might standardising the target help a gradient-descent model but not a random forest?**

GD's step size and loss scale depend on the target's magnitude; standardising keeps gradients in a sane range. Trees split on feature thresholds and predict leaf means; they are invariant to monotone rescaling of the target.

**Q6. Your learning curves show validation error still falling at the largest training size. What does that suggest?**

The model is data-limited (variance); more data will likely help. If the training error is also high, add capacity too.

**Q7. How would you detect and fix overfitting caused by hyper-parameter tuning itself?**

A large gap between CV score and fresh test score. Use nested CV or keep a final test set untouched, limit the search budget, and prefer simpler hyper-parameter regions.

**Q8. When would you prefer Lasso to Ridge and when Elastic net to both?**

Lasso when you believe few features matter and want sparsity; Ridge when many small effects/correlated features; Elastic net when features are correlated *and* you want some selection, or $n < m$.

**Q9. Why is the k in k-means a different kind of decision from the k in k-fold CV?**

k-means' k is a *modelling* choice (number of clusters) with no ground truth; k-fold's k is an *evaluation* choice (bias/variance/cost of the estimate; typically 5 or 10).

**Q10. Explain bagging, boosting and stacking to a manager in one sentence each.**

Bagging: ask many independent experts trained on different samples and average. Boosting: train experts in sequence, each focusing on the previous ones' mistakes. Stacking: hire a referee who learns which expert to trust when.

**Q11. A decision tree on a trend dataset predicts a flat line beyond the last observed point. Why, and what do you do?**

Leaves predict constants (means), so it cannot extrapolate. Use a linear/GAM component, model the detrended residual, or a model that extrapolates.

**Q12. Why can the same feature be useless to a linear model and useful to a forest?**

Linear models need a monotone, additive relationship; trees can use thresholds and interactions (e.g. latitude only matters in combination with income and proximity).

**Q13. Name three ways your pipeline could leak and how you would test for each.**

Preprocessing fitted on all data (move into Pipeline; compare scores), target-derived features (audit feature sources; check importances for suspiciously strong features), duplicates/related rows across splits (group-aware splitting, hash check). A too-good-to-be-true validation score should always trigger an audit.

**Q14. How do you choose between two models whose CV scores differ by less than one standard deviation?**

Treat them as equivalent on accuracy; choose on secondary criteria: simplicity, speed, interpretability, robustness, maintenance cost; or run a paired comparison across the same folds.

**Q15. Why use `StratifiedShuffleSplit`/`stratify=` and what is `GroupKFold` for?**

Stratified: preserve class (or binned-target) proportions in each split. GroupKFold: keep all rows of a group (customer, patient, region) in the same fold to avoid leakage between related samples.

**Q16. Derive why the gradient of the log loss simplifies to $X^T(\sigma(X\theta) - y)/m$.**

With $p = \sigma(z)$, $dp/dz = p(1 - p)$. The loss derivative for one instance w.r.t. z is $-y/p \cdot p(1-p) + (1-y)/(1-p) \cdot p(1-p) = -y(1-p) + (1-y)p = p - y$. By the chain rule $dz/d\theta = x$. Averaging gives $(1/m)X^T(p - y)$.

**Q17. What happens to ridge regression's solution as $\alpha \to 0$ and $\alpha \to \infty$?**

$\alpha \to 0$: the ordinary least-squares solution. $\alpha \to \infty$: all weights (except the bias) shrink to 0, the model predicts the mean (maximum bias).

**Q18. What is the effect of increasing `max_features` in a random forest?**

Trees become more similar and individually stronger: lower bias per tree but higher correlation between trees (less variance reduction). Tune it with CV; smaller values add diversity.

**Q19. You must explain a boosted model's prediction to a regulator. What can you offer?**

Feature importances (permutation importance is more reliable than impurity-based), partial dependence plots, per-prediction attributions (SHAP), a simpler surrogate model (a shallow tree or linear model), and the model card with its limitations.

**Q20. Why does Part II still care about everything in this guide?**

Neural networks are trained with gradient descent on a loss (softmax + cross-entropy for classification, MSE for regression), need scaling, overfit and need regularisation/early stopping, are validated with the same splits, and sit inside the same pipelines. New: automatic differentiation, layers, initialisation, optimisers, and GPUs.

# Appendix C · Bridge to Part II (Neural Networks and Deep Learning)

You are about to learn Keras and TensorFlow. The following table maps each Part II idea to the Part I idea you now understand. If the right-hand column feels solid, Part II will feel like a natural extension.

| Part II idea | Part I foundation (where you built it) |
|---|---|
| A neuron computes σ(w·x + b) | Logistic regression (Chapter 4: you coded it) |
| Multi-layer perceptron, softmax output layer | Softmax regression + cross-entropy (Chapter 4); stacking linear layers with non-linearities gives non-linear boundaries (kernel/polynomial ideas, Chapter 5) |
| Training with `model.fit`: epochs and batches | Mini-batch gradient descent, learning rate, epochs (Chapter 4, Figures 11–12) |
| Backpropagation | The gradient $(1/m)X^{\intercal}(p - y)$, generalised layer by layer by the chain rule; computed automatically (autodiff) |
| Optimisers: momentum, RMSProp, Adam | Plain GD and the learning-rate lessons; you implemented plain GD and (in experiments) momentum |
| Learning-rate schedules and finders | SGD learning schedule (Chapter 4) |
| Regularisation: $\ell_1/\ell_2$, dropout, early stopping | Ridge/Lasso, early stopping (Chapter 4), bagging-like averaging (Chapter 7; dropout ≈ ensemble of subnetworks) |
| Vanishing/exploding gradients, normalisation | Feature scaling (Chapters 2, 4); unstable GD (Figure 11) |
| Train/validation/test, callbacks, hyper-parameter tuning | Chapters 1–2: validation discipline, randomised search |
| Preprocessing layers, `tf.data` | Pipelines and ColumnTransformer (Chapter 2) |
| Autoencoders, GAN-like generative ideas | PCA/reconstruction error (Chapter 8), Gaussian mixtures and density estimation (Chapter 9) |
| Over-parameterised models that still generalise | Bias–variance (Chapters 1, 4), regularisation, ensembling |

## Suggested next step: a bridge experiment

Before opening Part II, run this 30-minute experiment to anchor the transition: using `sklearn.neural_network.MLPRegressor` (a small multi-layer perceptron) on the HomeLens preprocessed features, compare it against the HistGradientBoosting baseline under the same CV folds. Plot the validation curve against `hidden_layer_sizes` and `alpha`. You should *predict*, before running it, that the network will need scaled inputs, will be more sensitive to the learning rate than the boosted trees, and will probably not beat boosting on 16k tabular rows.

## Mastery checklist

- ☐ I can explain train/validation/test sets, CV and data leakage without notes.
- ☐ I can build a Pipeline + ColumnTransformer with a custom transformer from memory.
- ☐ I can write the gradient-descent update and implement linear and logistic regression with NumPy.
- ☐ I can read learning curves and say whether to get more data or a better model.
- ☐ I can select a metric and threshold from a cost description, and defend precision/recall/AUC choices.
- ☐ I can explain C and γ in SVMs and why scaling is mandatory.
- ☐ I can compute Gini and a CART split by hand, and explain tree instability.
- ☐ I can explain bagging, boosting and stacking and say which reduces bias or variance.

- ☐ I can compute PCA via SVD and explain when reduction hurts.
- ☐ I can choose k for k-means, explain DBSCAN and GMM/BIC, and use clustering for features and anomaly detection.
- ☐ I can re-run this entire project, change one design choice, and predict the effect.
- ☐ I scored well on the challenge questions (aim: at least 80% answered fluently).

## A two-week plan

| Days | Focus | Output |
|---|---|---|
| 1–2 | Chapters 1–2 + Steps 1–3; read the code line by line; do experiments | Re-write `features.py` from memory |
| 3–4 | Chapters 3–4 + Steps 4–5 | Implement logistic regression + GD variants without looking |
| 5–6 | Chapters 5–7 + Steps 6–8 | Reproduce the bake-off with a different dataset (e.g. the diabetes dataset) |
| 7–8 | Chapters 8–9 + Steps 9–10 | Explain PCA and GMM to a friend; run the extensions |
| 9–10 | Step 11 + Chapter 10 | Write your own model card for a variant model |
| 11–12 | Challenge questions + mixed round | Answer aloud, mark weak areas, revisit those chapters |
| 13–14 | Capstone (below) + Part II bridge experiment | A new project on a new dataset using the same structure |

## Capstone extensions

- **Out-of-time validation**: simulate drift by training on districts from some regions and testing on others; quantify the degradation.
- **Quantile regression**: use `HistGradientBoostingRegressor(loss='quantile', quantile=0.1/0.9)` to give prediction intervals per district; check their coverage on the test set.
- **Calibration**: plot a reliability curve of the premium classifier and apply `CalibratedClassifierCV`.
- **Permutation importance & partial dependence** (`sklearn.inspection`) to explain the final model.
- **Monitoring simulation**: add noise/shift to features and compute the population stability index (PSI) per feature to practice drift detection.
- **New dataset**: apply the entire workflow (framing → EDA → pipeline → baselines → models → tuning → test → card) to another tabular dataset.

# Appendix D · Glossary

| Term | Meaning |
|---|---|
| **Baseline** | Simple reference model used to judge whether a complex model adds value. |
| **Bias (statistical)** | Error from wrong/simplistic assumptions; causes underfitting. |
| **Bootstrap** | Sampling with replacement to create a resampled training set. |
| **Cross-validation** | Rotating validation folds to estimate generalisation error from the training data. |
| **Data leakage** | Information from validation/test (or the future/the target) reaching training. |
| **Epoch** | One pass through the full training set. |
| **Feature engineering** | Creating/transforming features to expose structure to the model. |
| **Generalisation error** | Expected error on new, unseen data. |
| **Gradient descent** | Iterative optimisation moving parameters against the cost gradient. |
| **Hyper-parameter** | A setting of the learning algorithm, set before training. |
| **Hinge loss** | max(0, 1 − t·s), used by SVMs. |
| **Inertia** | Sum of squared distances of points to their cluster centroid. |
| **Kernel trick** | Computing inner products in a high-dimensional feature space without constructing it. |
| **Learning curve** | Training and validation error as a function of training-set size. |
| **Log loss** | Cross-entropy for binary classification. |
| **Margin** | Width of the street between classes in an SVM. |
| **OOB (out-of-bag)** | Training instances left out of a bootstrap sample, used for validation. |
| **Overfitting** | Model fits training noise: low training error, high validation error. |
| **Pipeline** | A chain of transformations and an estimator fitted as one object. |
| **Precision / Recall** | Share of positive predictions that are right / share of actual positives found. |
| **Regularisation** | Constraining a model to reduce overfitting. |
| **Residual** | Actual minus predicted value. |
| **Stratified sampling** | Sampling that preserves group proportions. |
| **Support vector** | Training instance on/inside the SVM margin that defines the boundary. |
| **Underfitting** | Model too simple to capture the pattern: high error on both train and validation. |
| **Variance (statistical)** | Sensitivity of the model to the specific training sample; causes overfitting. |

## Closing note

If you have *run* the code, predicted the outputs, broken things on purpose and answered the questions aloud, you do not merely recognise Part I: you can use it, debug it and defend it. That is exactly the preparation Part II assumes. Keep the project as a template: **frame → split → explore → pipeline → baseline → compare → tune → evaluate once → explain → ship → monitor.** Every project in Part II, and beyond, follows the same arc.